

Computer Vision 1

3-1: Local Feature, Line and Corner

Master of AI – 2023/2024

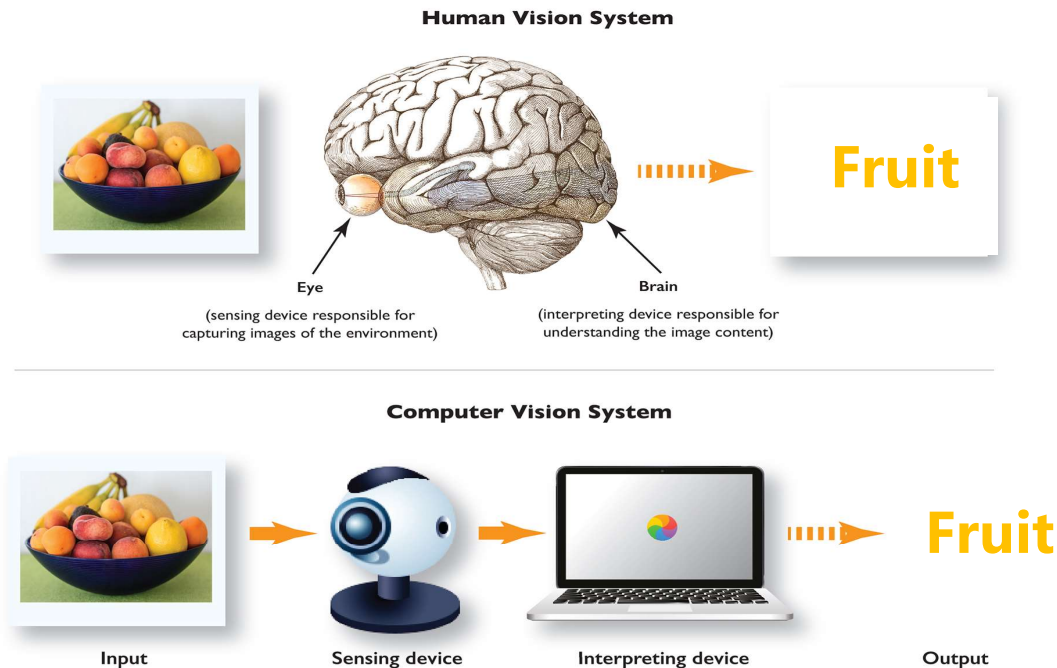
University of Amsterdam



Arnoud Visser & Martin Oswald

The computer vision problem

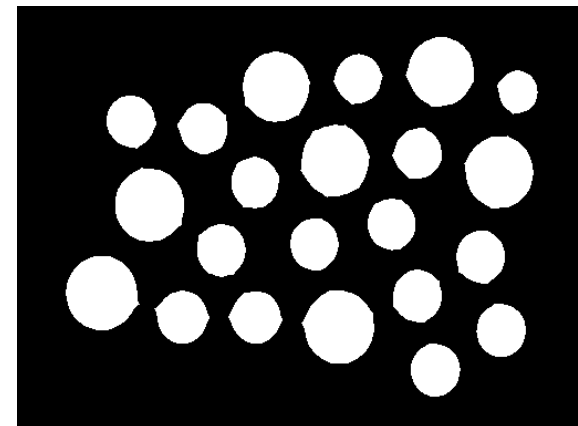
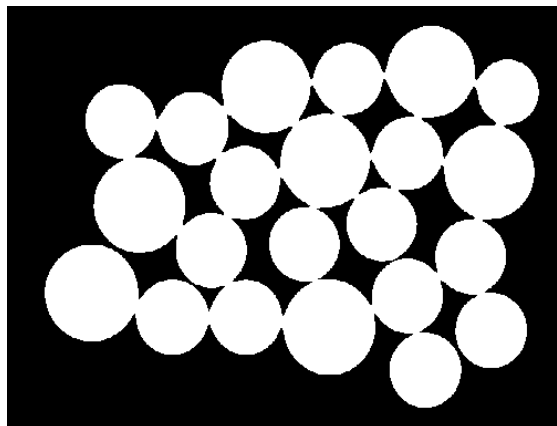
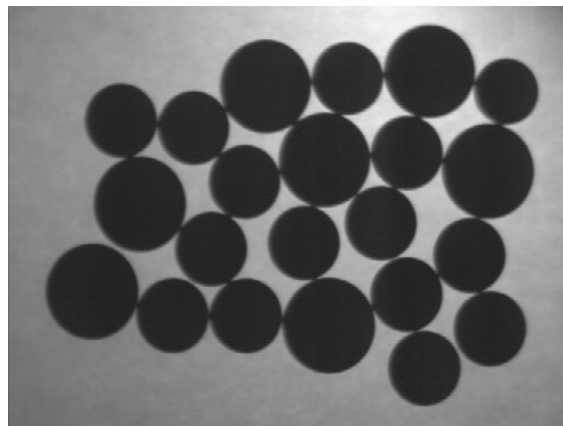
- Make a computer see and understand images



- We know it is physically possible –
we do it every day and effortlessly!

Counting Coins

- Counting coins is difficult because they touch each other!
- Solution: Binarization and Erosion separates them!



Gray-scale morphology

Erosion, dilation, opening and closing can also be defined for grayscale images.

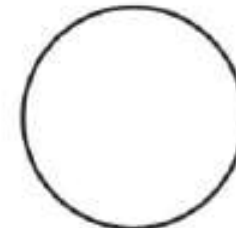
$$f(x, y) : \mathbb{Z}^2 \rightarrow \mathbb{R}$$

$$b(x, y) : \mathbb{Z}^2 \rightarrow \mathbb{R}$$

The structuring element b is used as “probe” to examine a given image for specific properties. Can be non-flat or flat (more common)

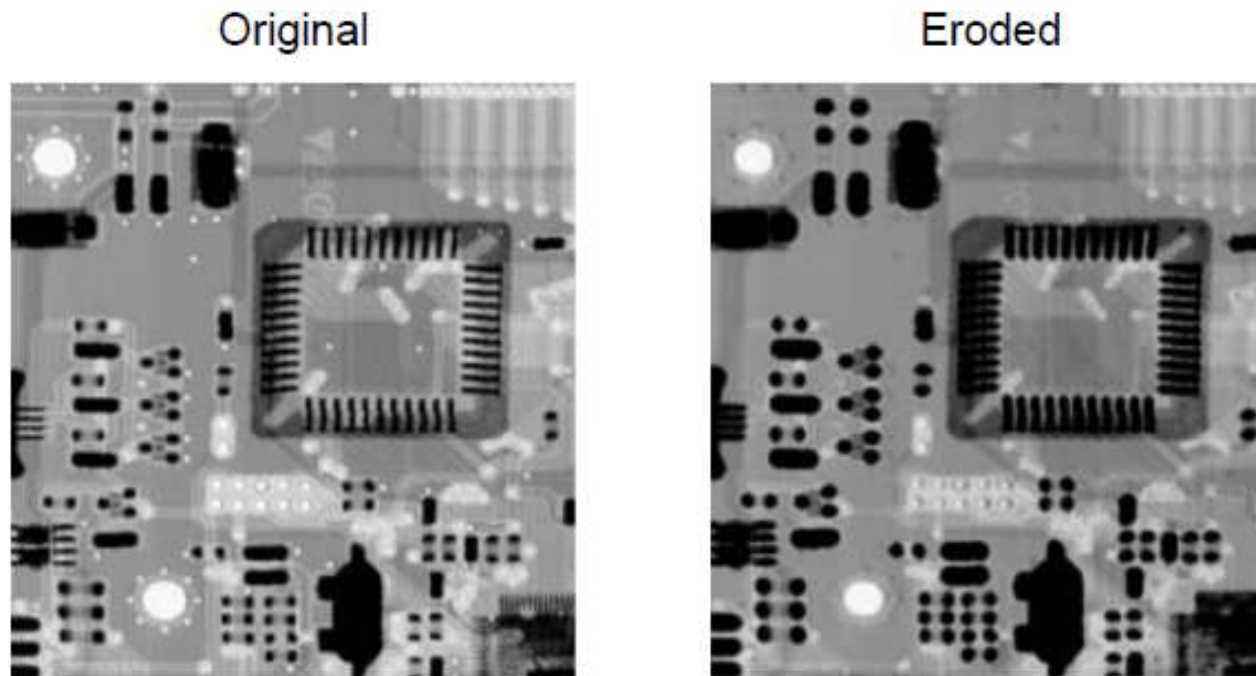


Nonflat SE



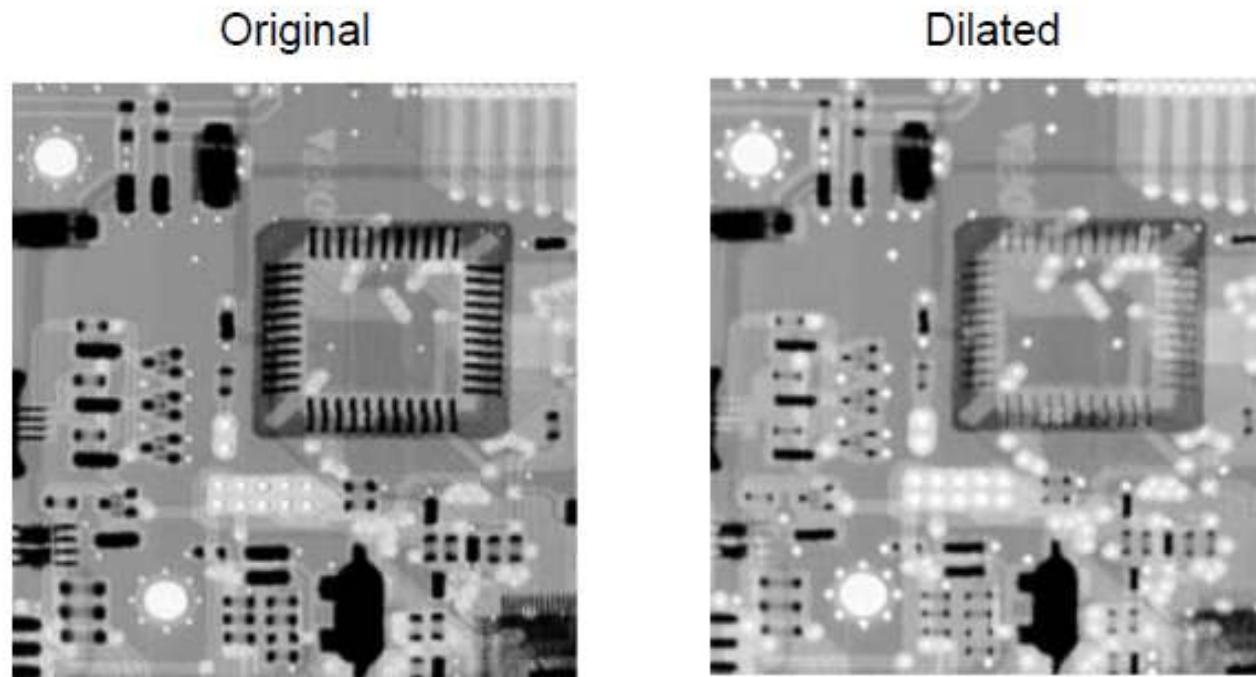
Flat SE

Effect of gray-scale erosion



Since erosion computes minimum intensity values over every neighborhood of (x,y) , the resulting image is darker and the size of bright features is in general reduced.

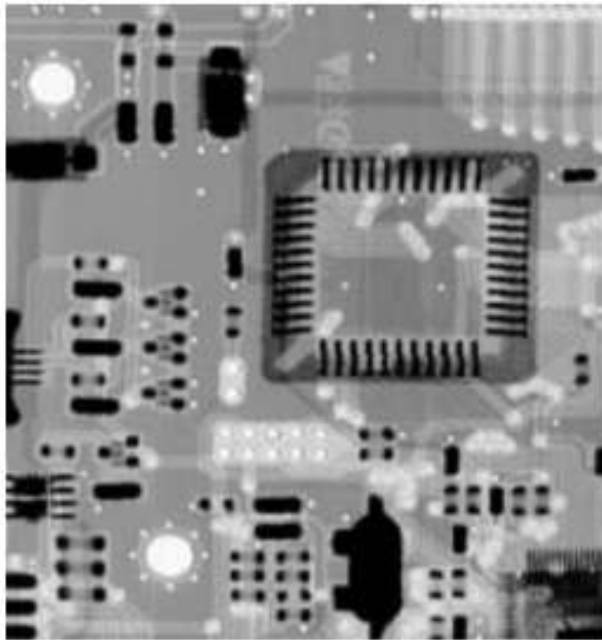
Effect of gray-scale dilation



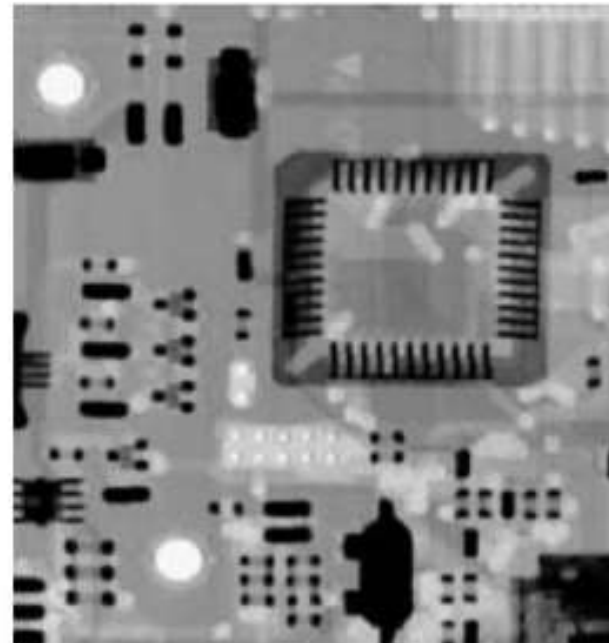
Dilation give opposite results with respect to erosion. The resulting image is brighter, bright features are thickened and the intensities of the dark features is reduced

Effect of gray-scale opening

Original

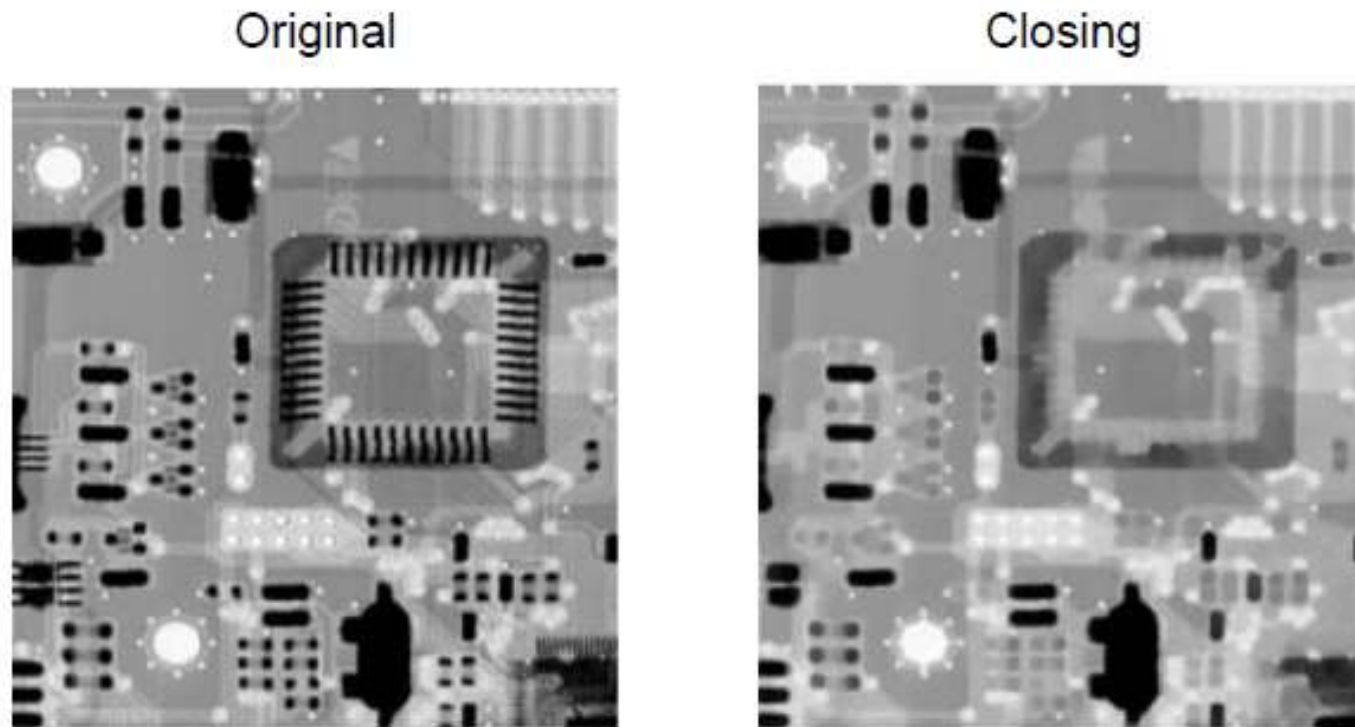


Opening



Opening attenuate bright features and has negligible effect on the dark features and the background of the image

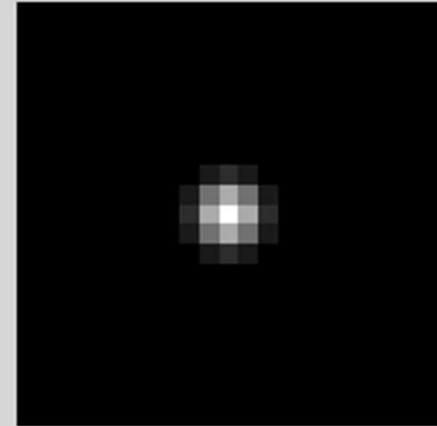
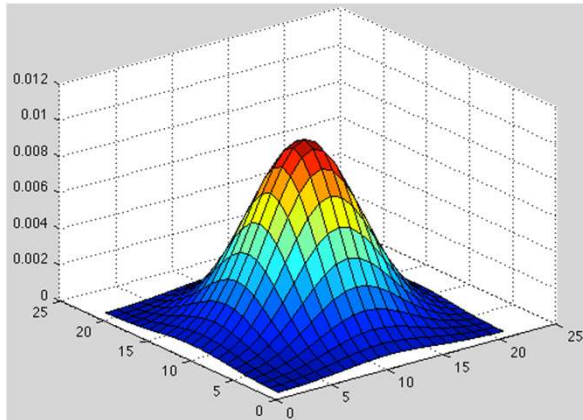
Effect of gray-scale closing



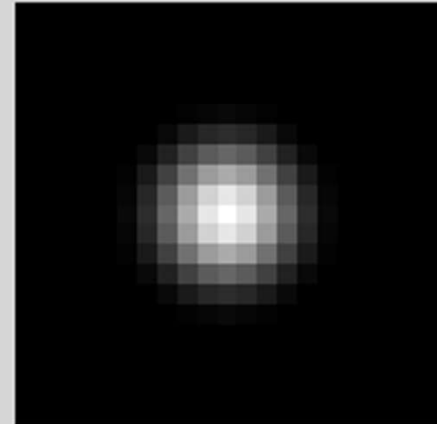
Closing attenuate dark features and has negligible effect on the bright features and the background of the image

Gaussian filter: change scales

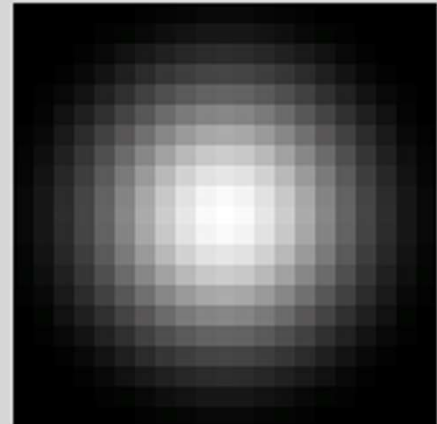
$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$



$\sigma=1$



$\sigma=2$



$\sigma=4$

Low and High Pass filtering

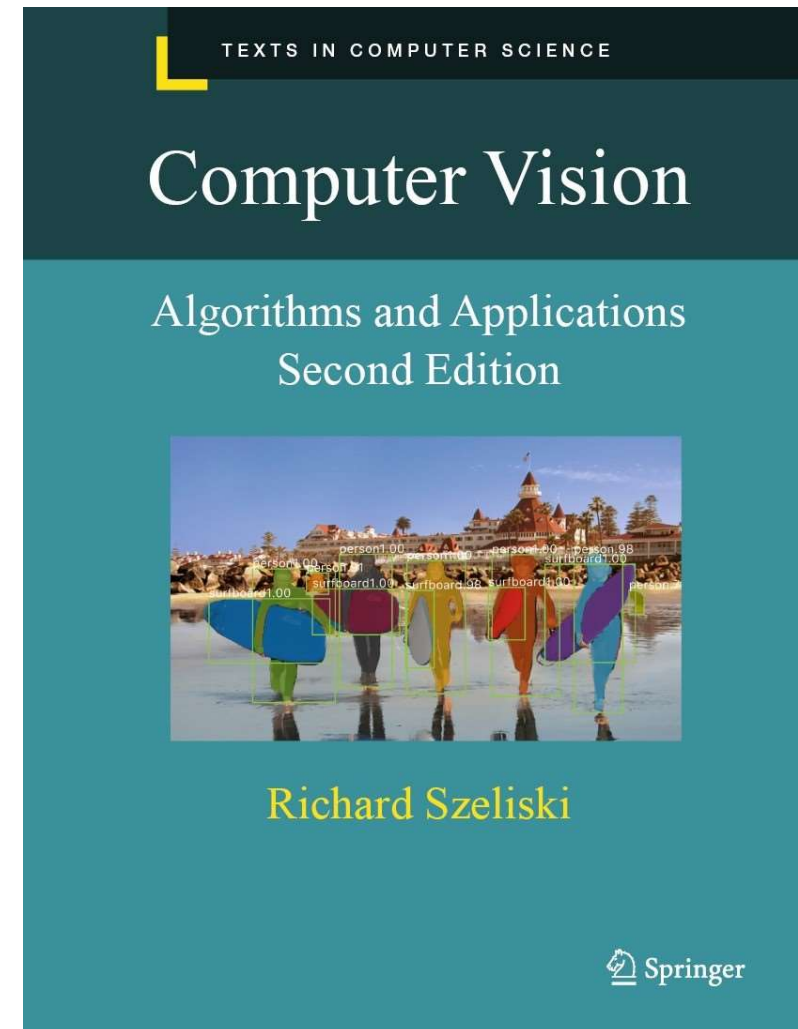


Outline

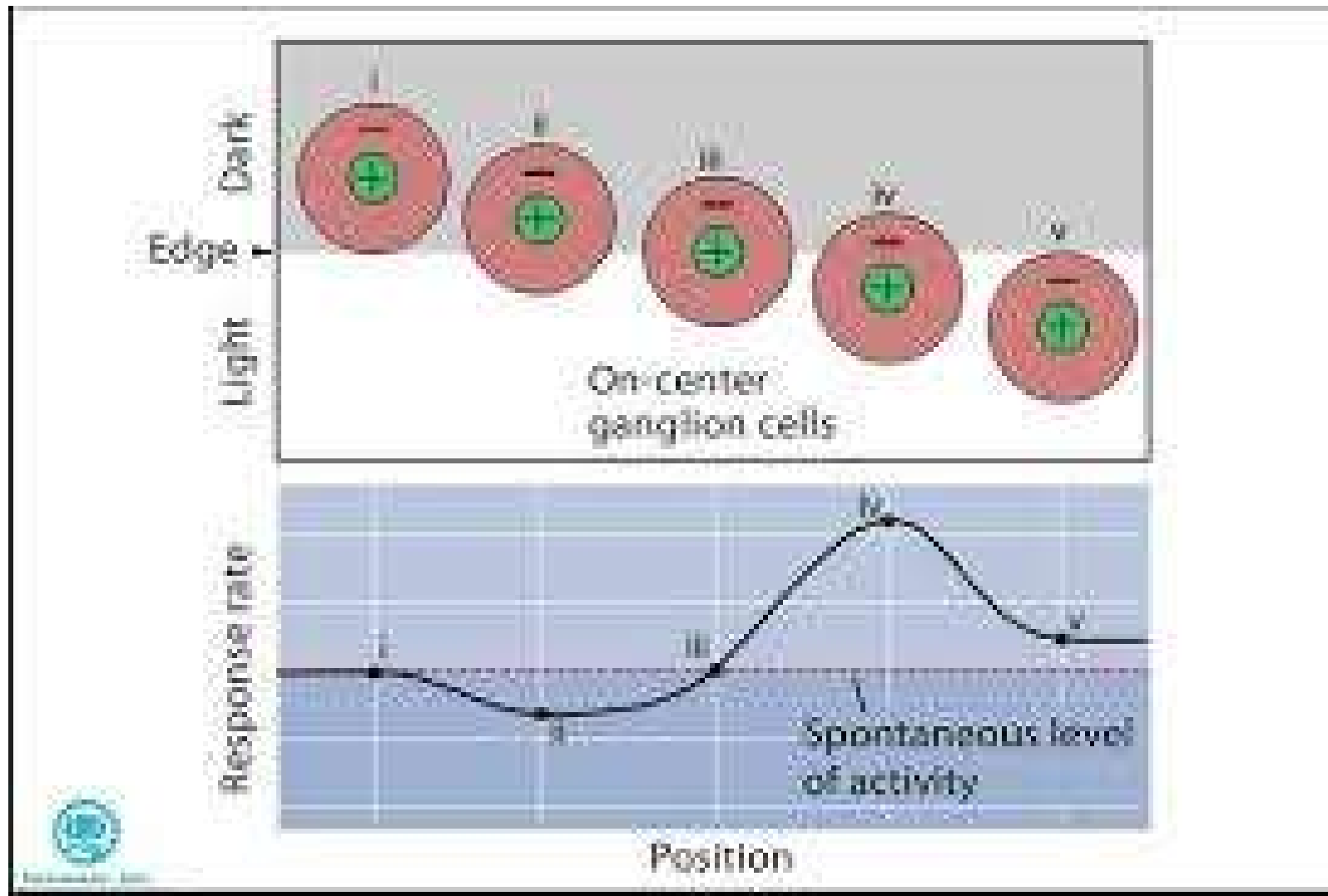
- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

Textbook

- 7.2
- 7.4
- 7.1



Responses: ON-Centre Ganglion Cells



Human is very shape biased

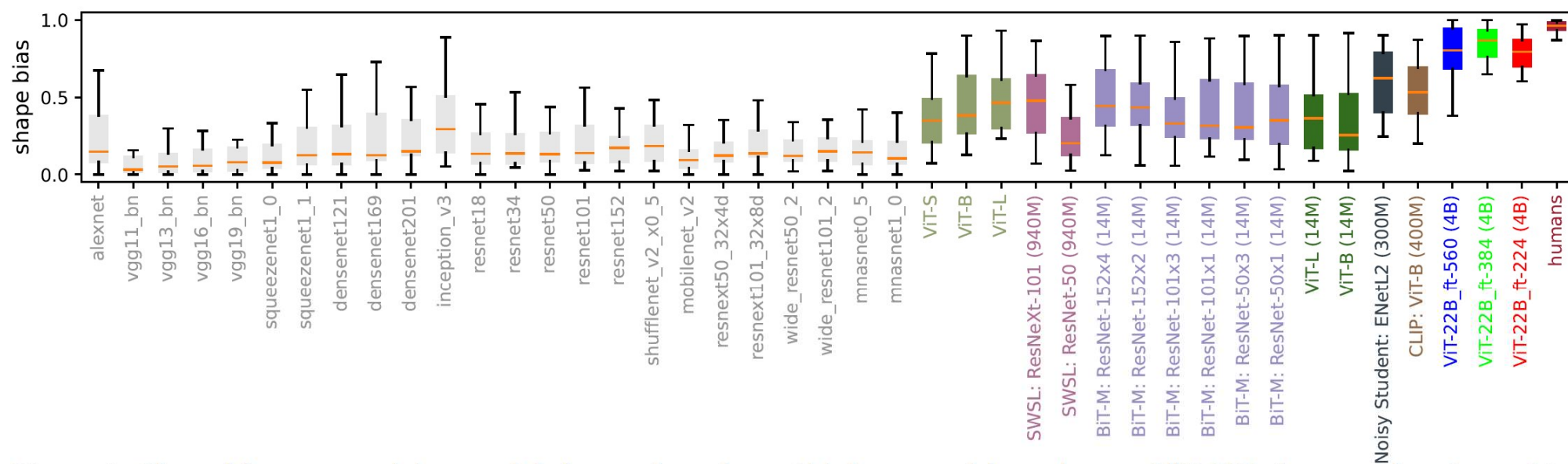
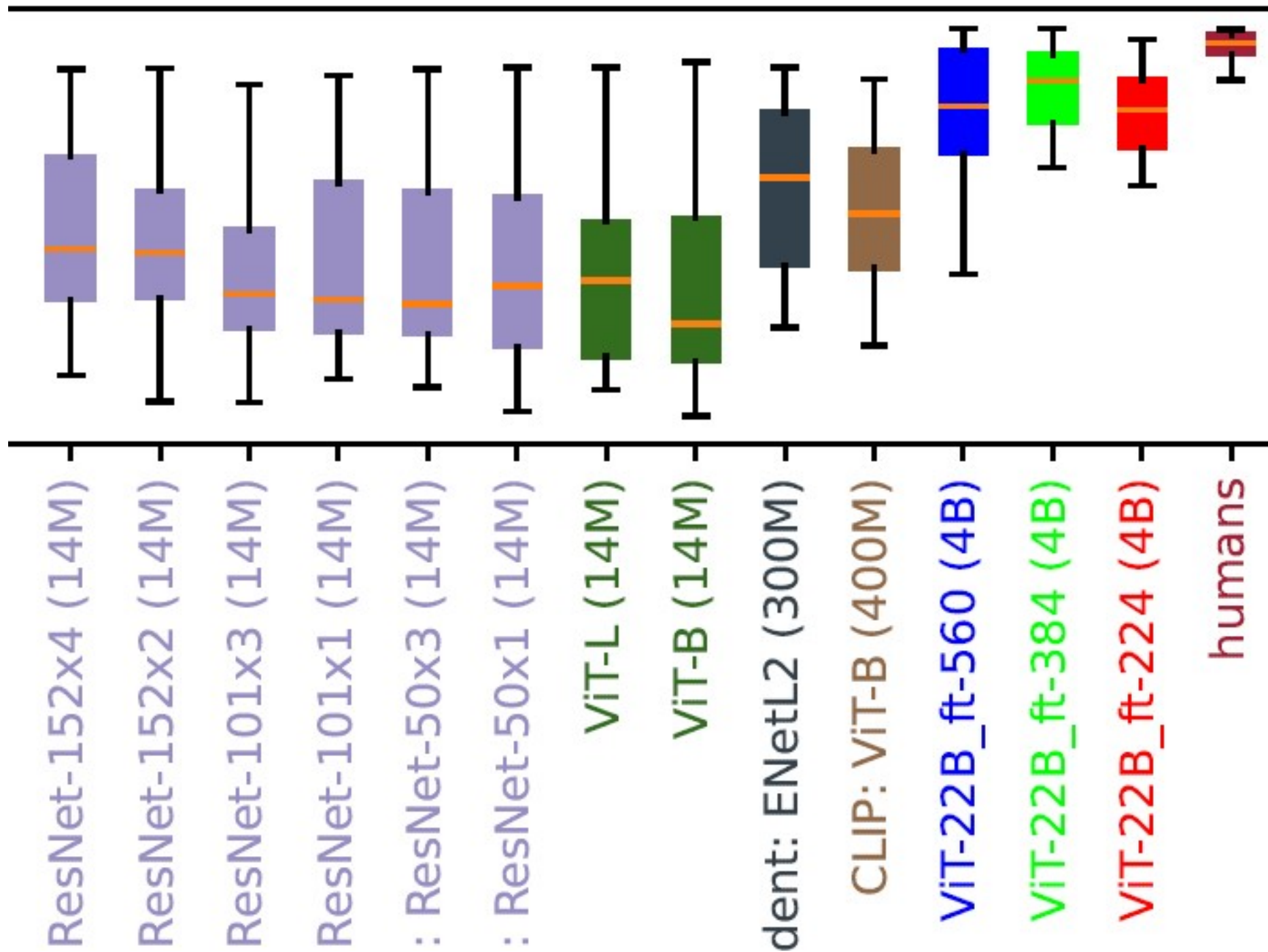
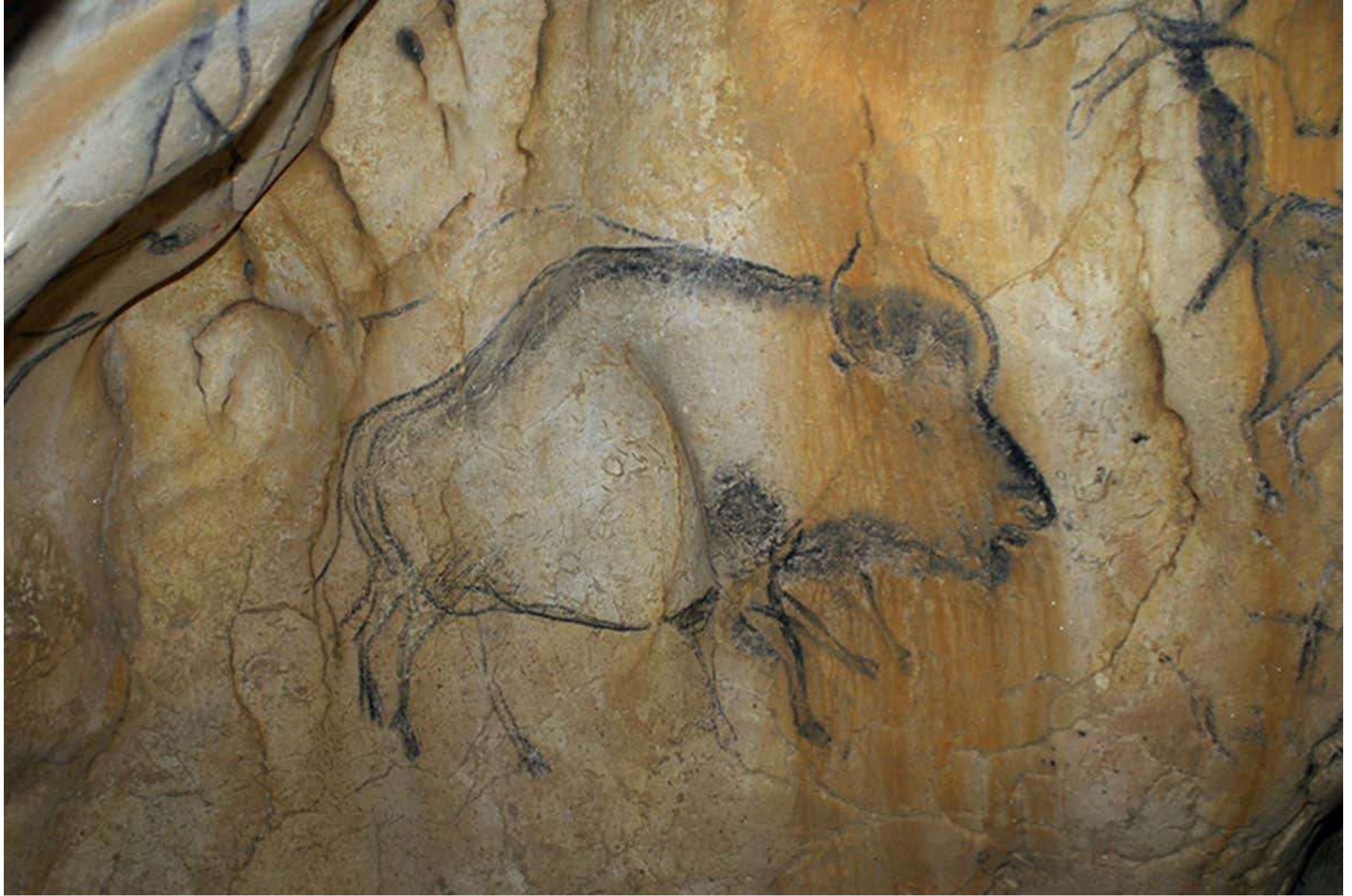


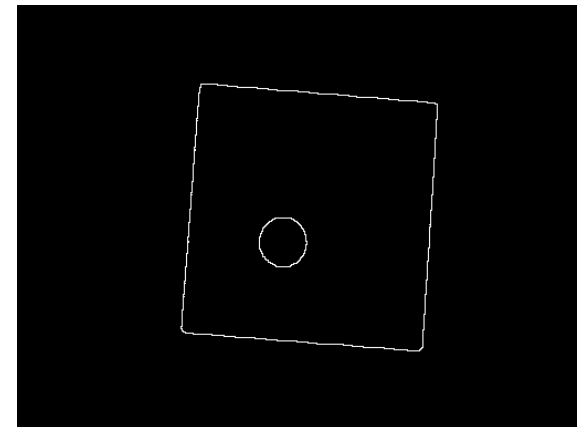
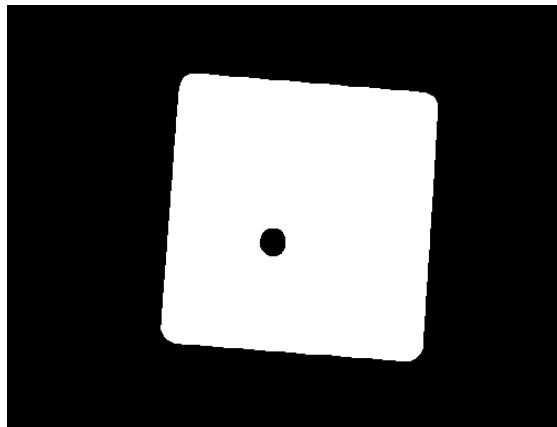
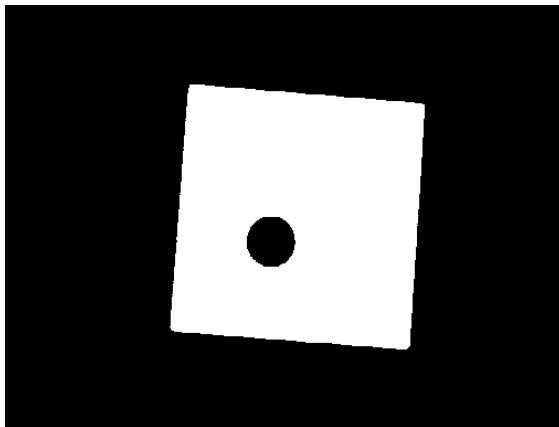
Figure 8: Shape bias: many vision models have a low shape / high texture bias, whereas ViT-22B fine-tuned on ImageNet (red, green, blue trained on 4B images as indicated by brackets after model names, unless trained on ImageNet only) have the highest shape bias recorded in a ML model to date, bringing them closer towards a human-like shape bias.



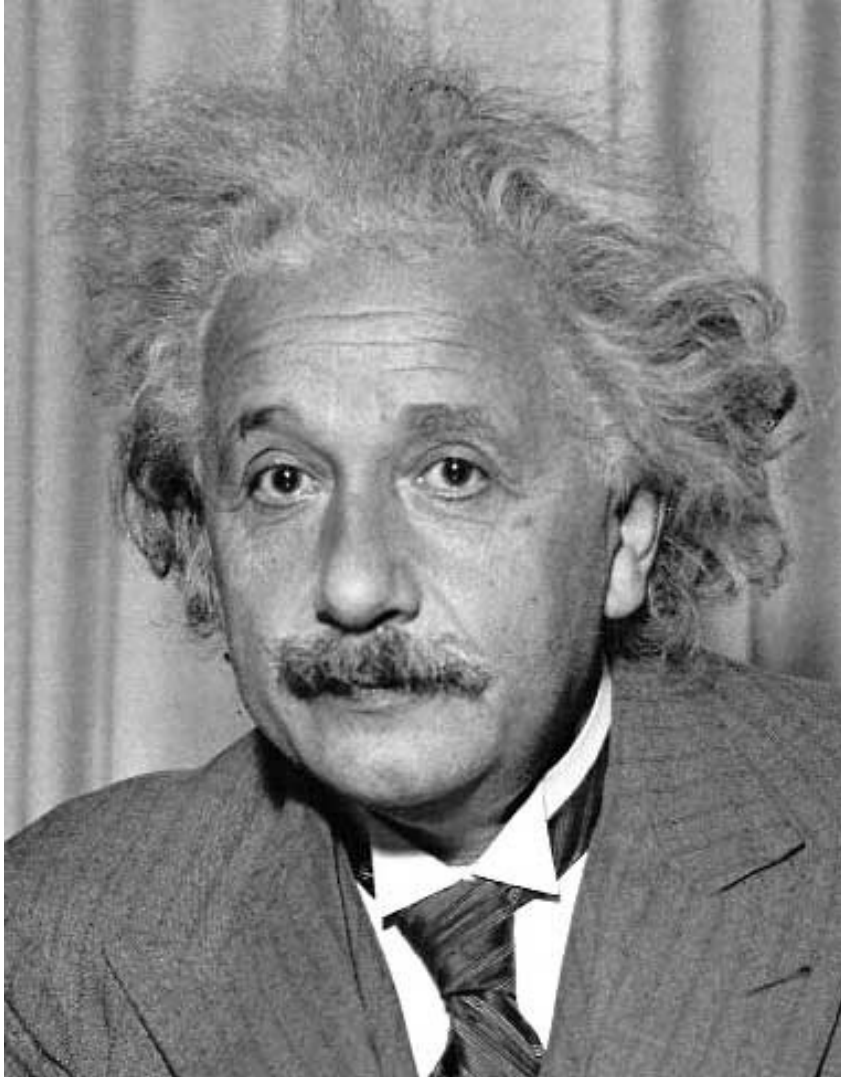


Edge detection example

- Edge Detection
 1. Dilate input image
 2. Subtract input image from dilated image
 3. Edges remain!

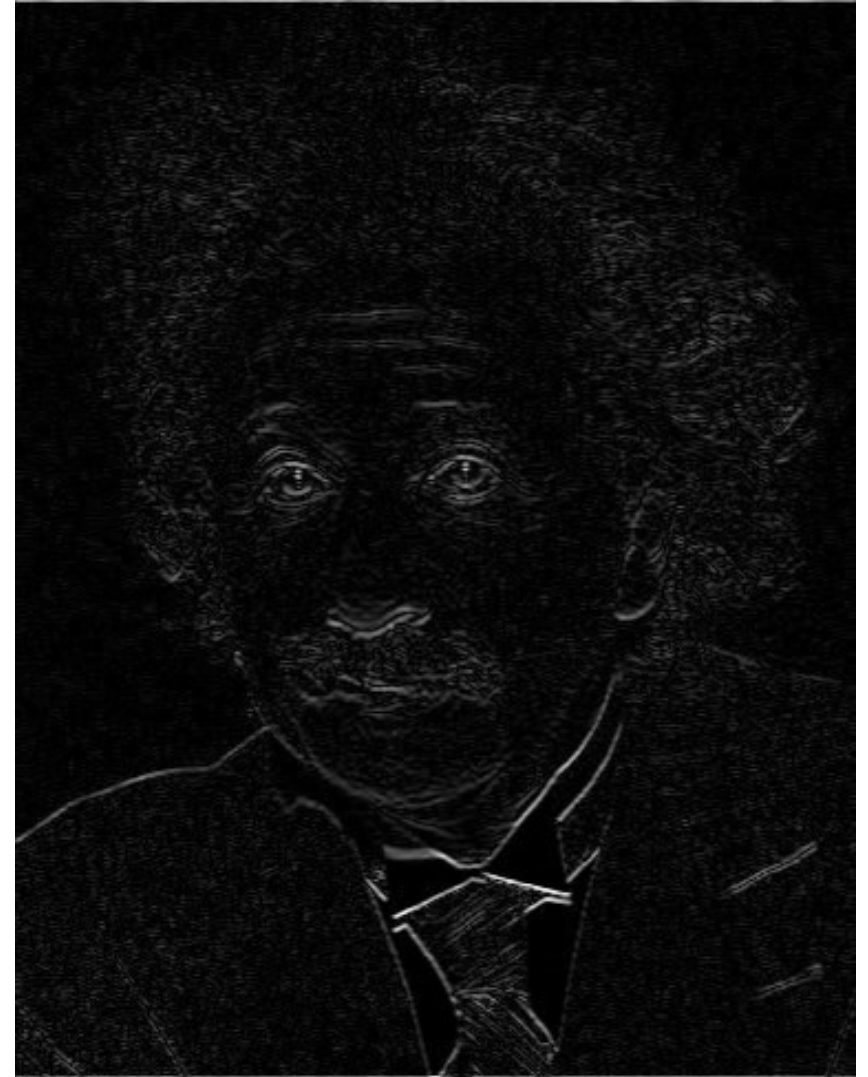


Derivative filters



1	2	1
0	0	0
-1	-2	-1

Sobel



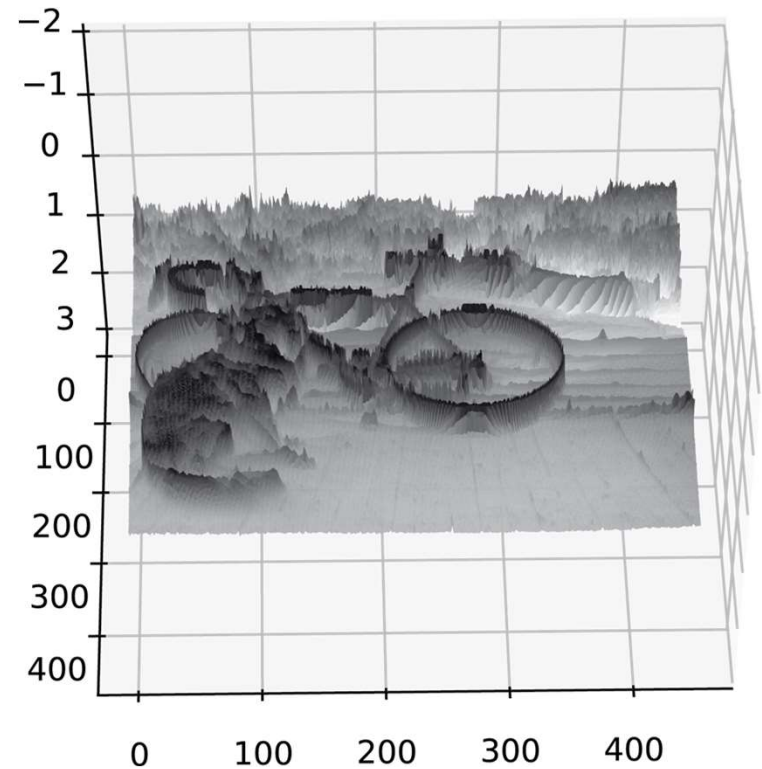
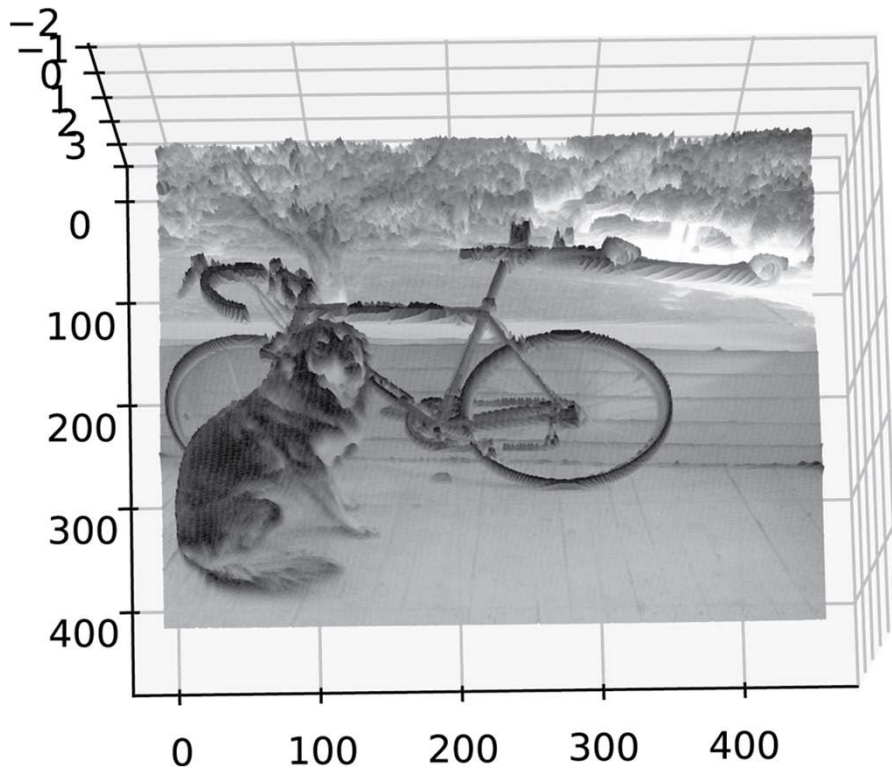
Horizontal Edge
(absolute value)

Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

What's an edge?

- Image is a function.
- Think of the gray tones as HEIGHTS.
- Edges are rapid changes in this function



Methods

- Using first order derivative: *Look for extrema*

- 1st directional derivative.
- 1st derivative of Gaussian

$$\left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right)$$
$$\left(\frac{\partial G}{\partial x}, \frac{\partial G}{\partial y} \right)$$

- Using second order derivative: *Look for zero-crossings*

- 2nd directional derivative (Laplacian)
- 2nd derivative of Gaussian:
 - Laplace of Gaussian (LoG)
 - Difference of Gaussians (DoG)

1st and 2nd Derivative Operations

- For a 2D function, the first derivative and its kernel are

$$\frac{\partial f(x,y)}{\partial x} \cong f(x,y) - f(x-1,y)$$

-1	1
----	---

- The second derivative and its kernel become

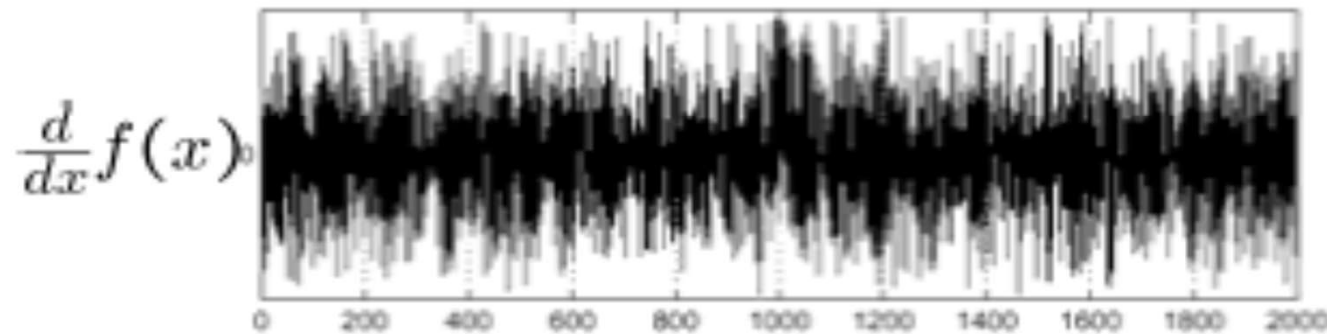
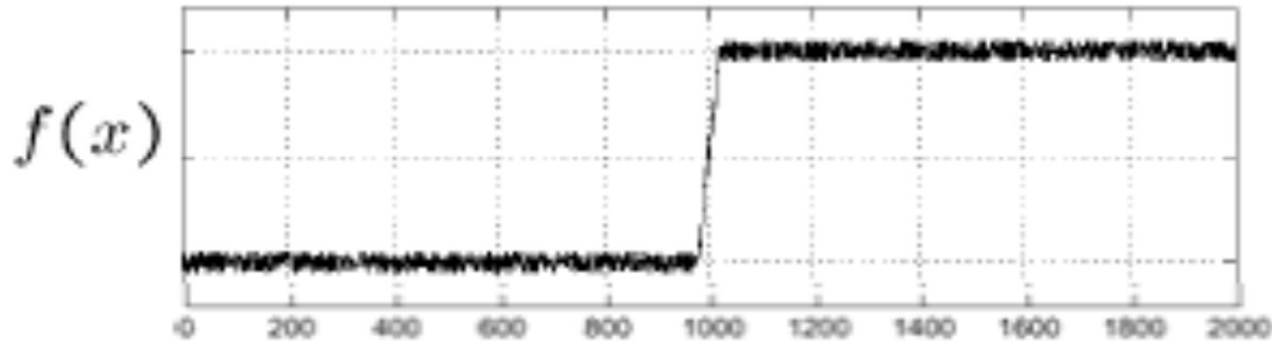
$$\begin{aligned}\frac{\partial^2 f(x,y)}{\partial x^2} &\cong [f(x+1,y) - f(x,y)] - [f(x,y) - f(x-1,y)] \\ &= f(x+1,y) - 2f(x,y) + f(x-1,y)\end{aligned}$$

1	-2	1
---	----	---

Problem with noisy image

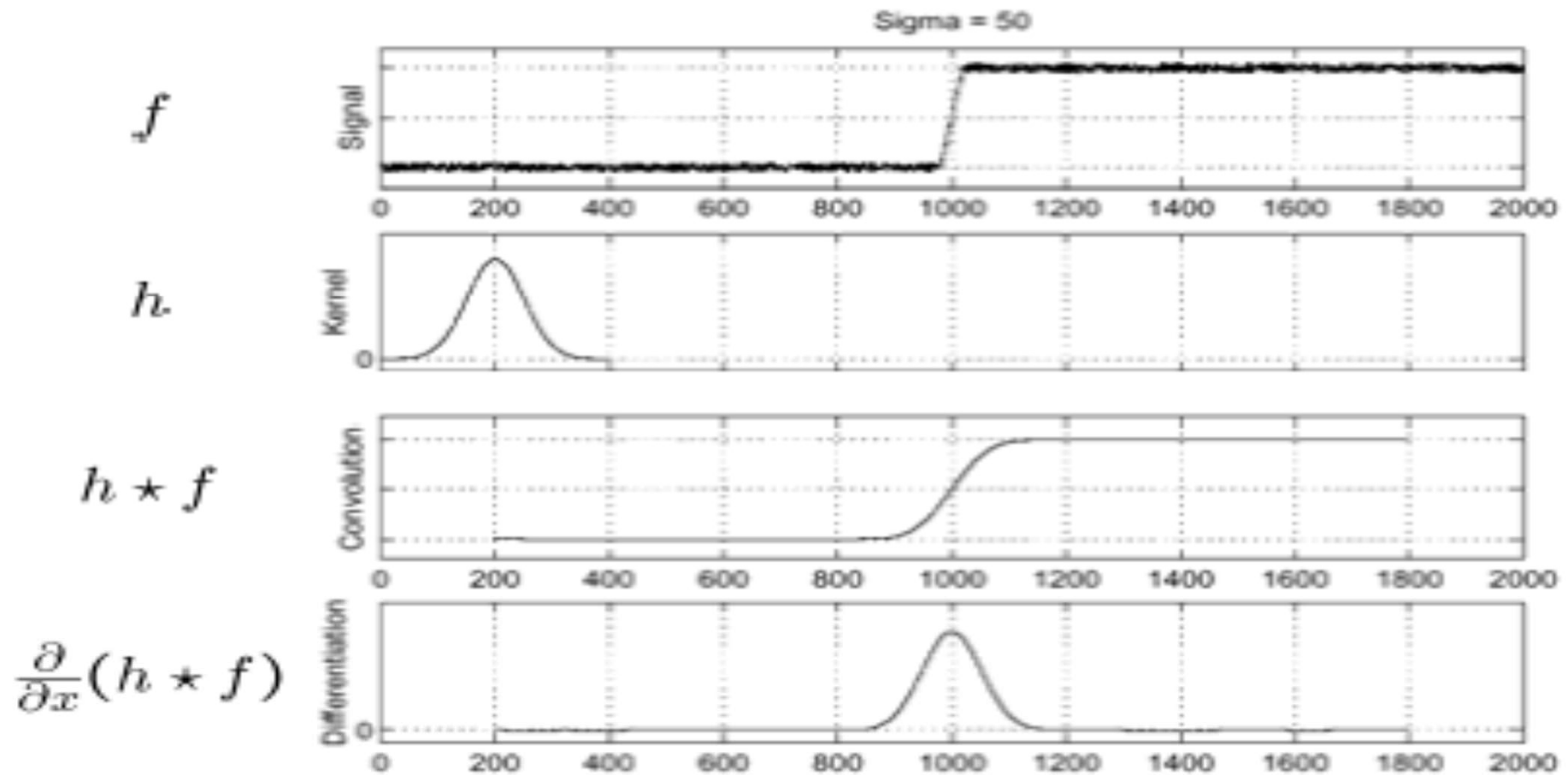
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal



Where is the edge?

Solution: smooth the image first



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

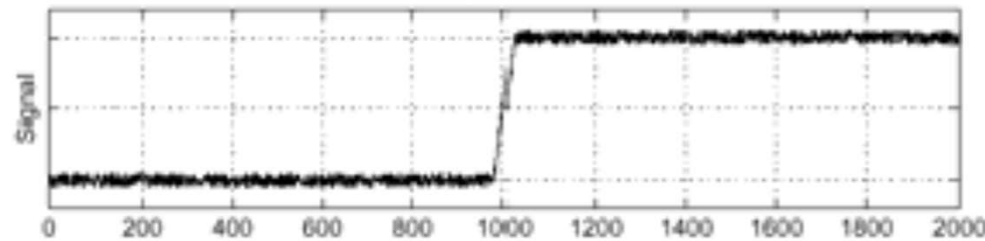
Derivative of Gaussian filter

$$\frac{\partial}{\partial x}(h \star f) = \left(\frac{\partial}{\partial x}h\right) \star f$$

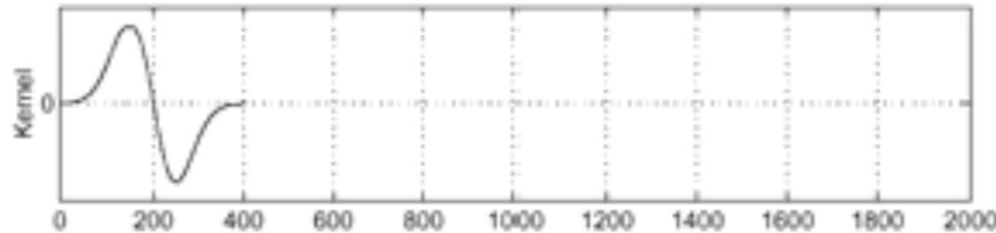
Differentiation property of convolution.

Sigma = 50

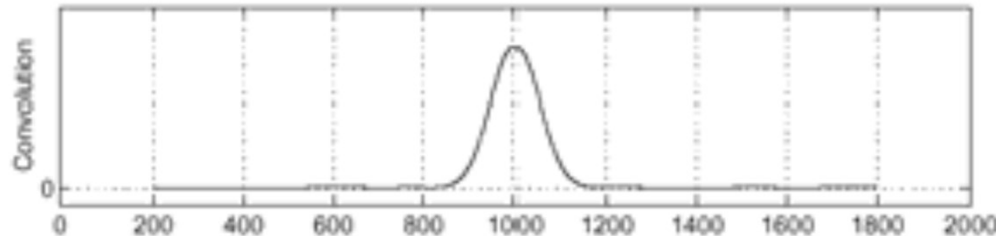
f



$\frac{\partial}{\partial x}h$



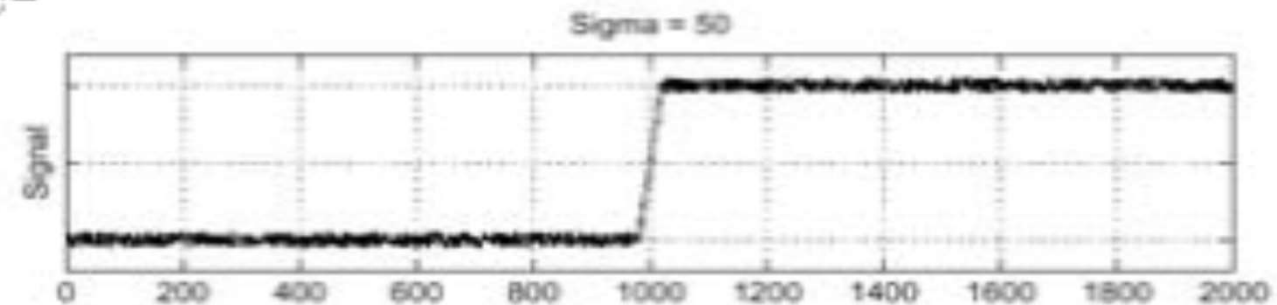
$\left(\frac{\partial}{\partial x}h\right) \star f$



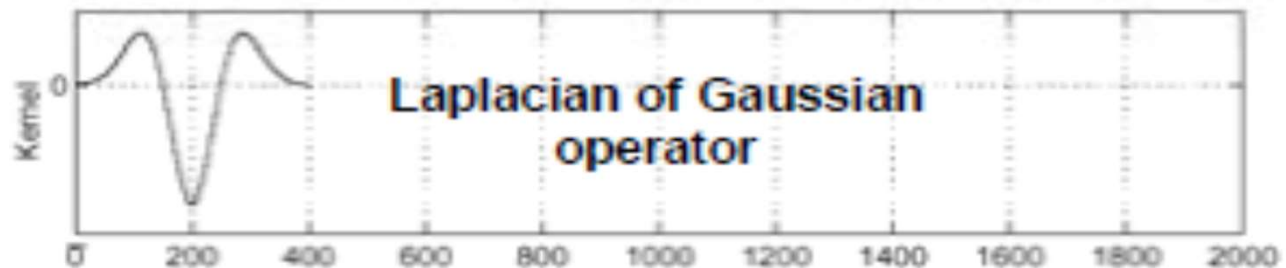
Edge as the zero-crossing of the second order derivative using Laplacian of Gaussian

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

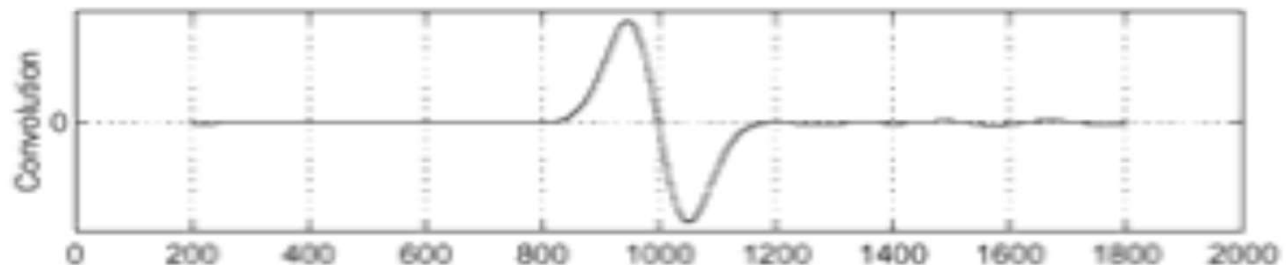
f



$\frac{\partial^2}{\partial x^2}h$



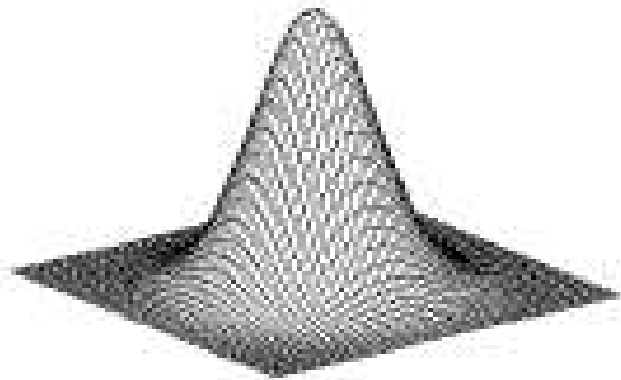
$(\frac{\partial^2}{\partial x^2}h) \star f$



Where is the edge?

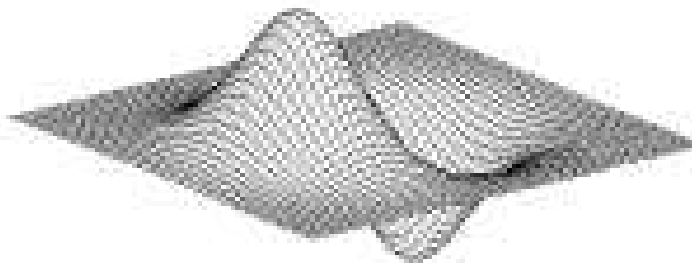
Zero-crossings of bottom graph

2D edge detection filters



Gaussian

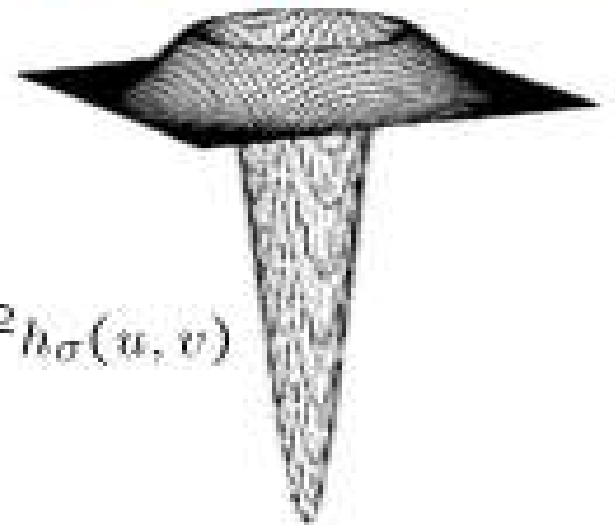
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian



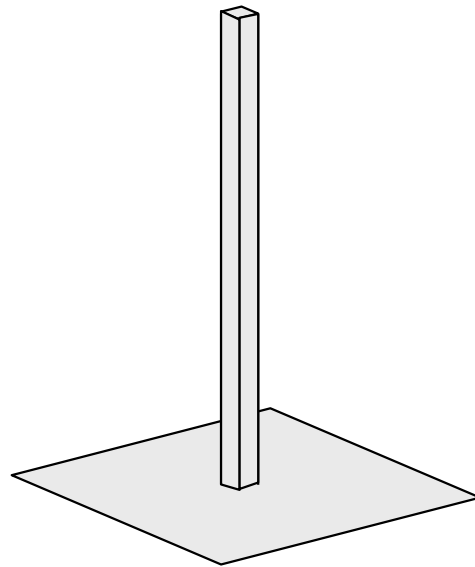
$$\nabla^2 h_{\sigma}(u, v)$$

- ∇^2 is the Laplacian operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

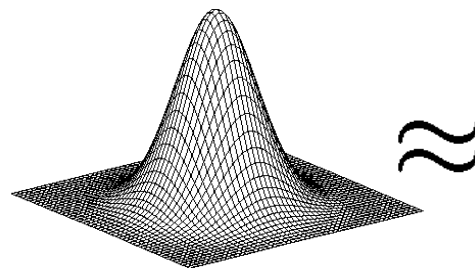
Difference of Gaussians (DoG)

- Laplacian of Gaussian (LoG) can be approximately computed as Difference of Gaussians (DoG).



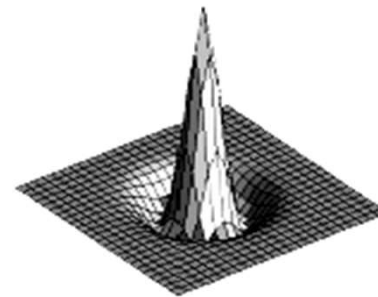
A narrow Gaussian
(or an impulse function)

—



A wide Gaussian

$\approx$

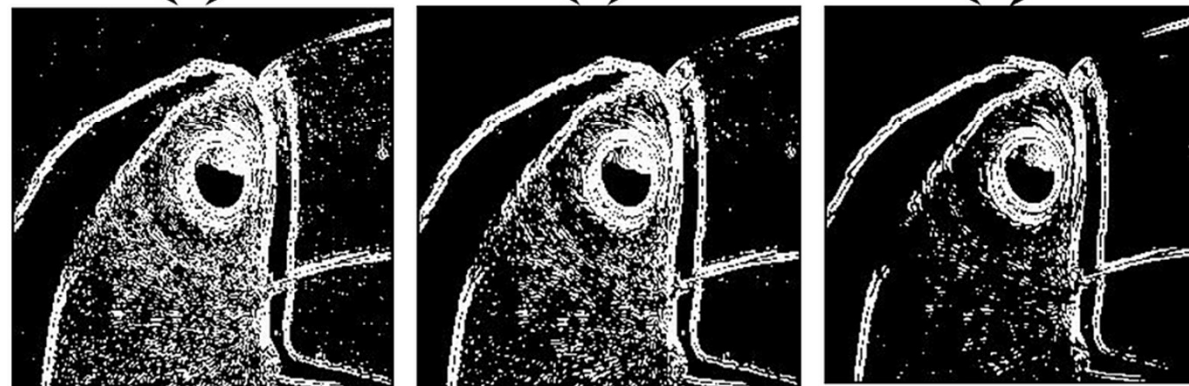
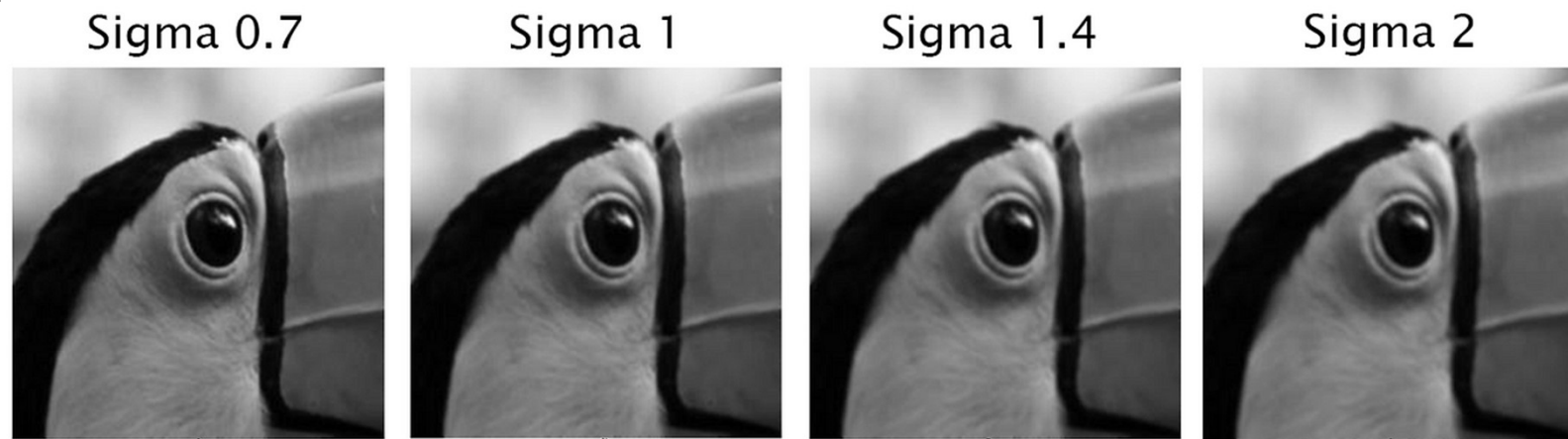


Laplacian of Gaussian

Change of Sigma Values in DoG

Small
Sigma

Large
Sigma



Difference of Gaussian

Example: LoG filtering

Note that LoG output has negative values which need offset (e.g. add 128) and scaling

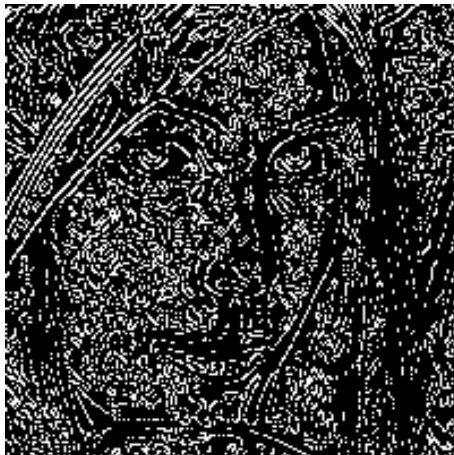
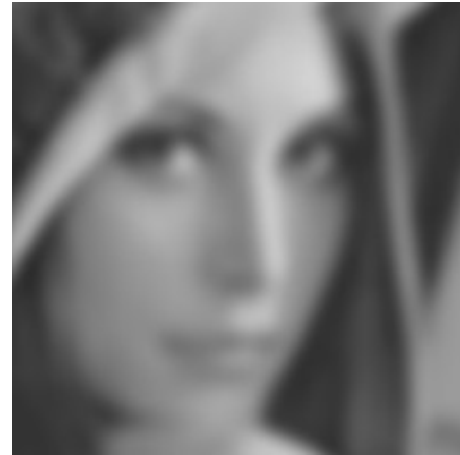


Change of Sigma Values in LoG

Small
Sigma



Large
Sigma



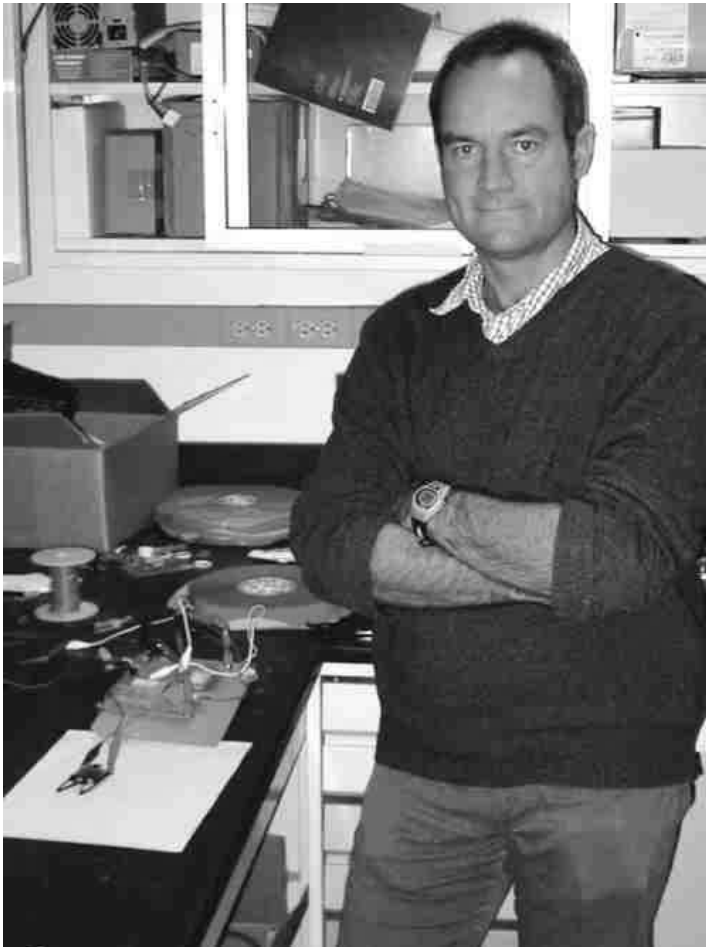
Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

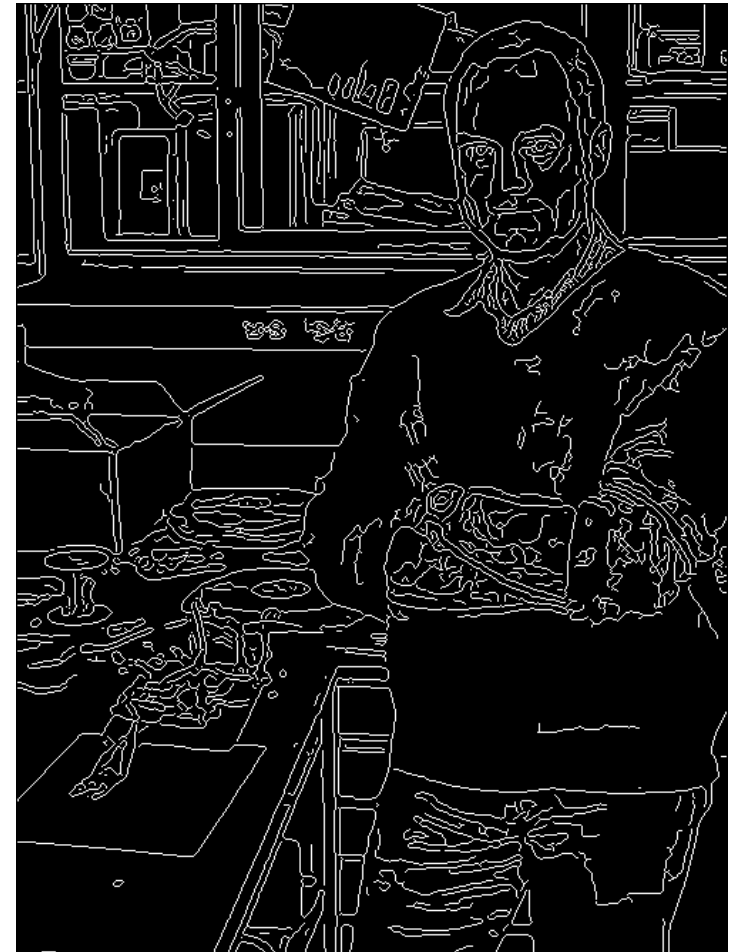
Canny Edge Detector is a statistically optimal edge detector

- Make use of Derivative of Gaussian
- Thinning edges (called non-maximum suppression)
 - Thin multi-pixel wide “ridges” down to single pixel
- Optimal thresholding
 - Accept all edges over low threshold if they are connected to edge over high threshold
- **OpenCV:** `cv.canny(I, low_threshold, high_threshold)`

Canny Edge Detector



[John Canny](#)



Canny Edge Detector is one of the most successful edge detection method.

Ref: John F. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Analysis and Machine Intelligence, Vol. 8, No. 16, pp. 679-698, Nov. 1986.

9/16/2024

Example: Canny edge detector



Blurred

Original image



Gradient Magnitude

Norm of the gradient

The Canny edge detector



Non-max Suppression

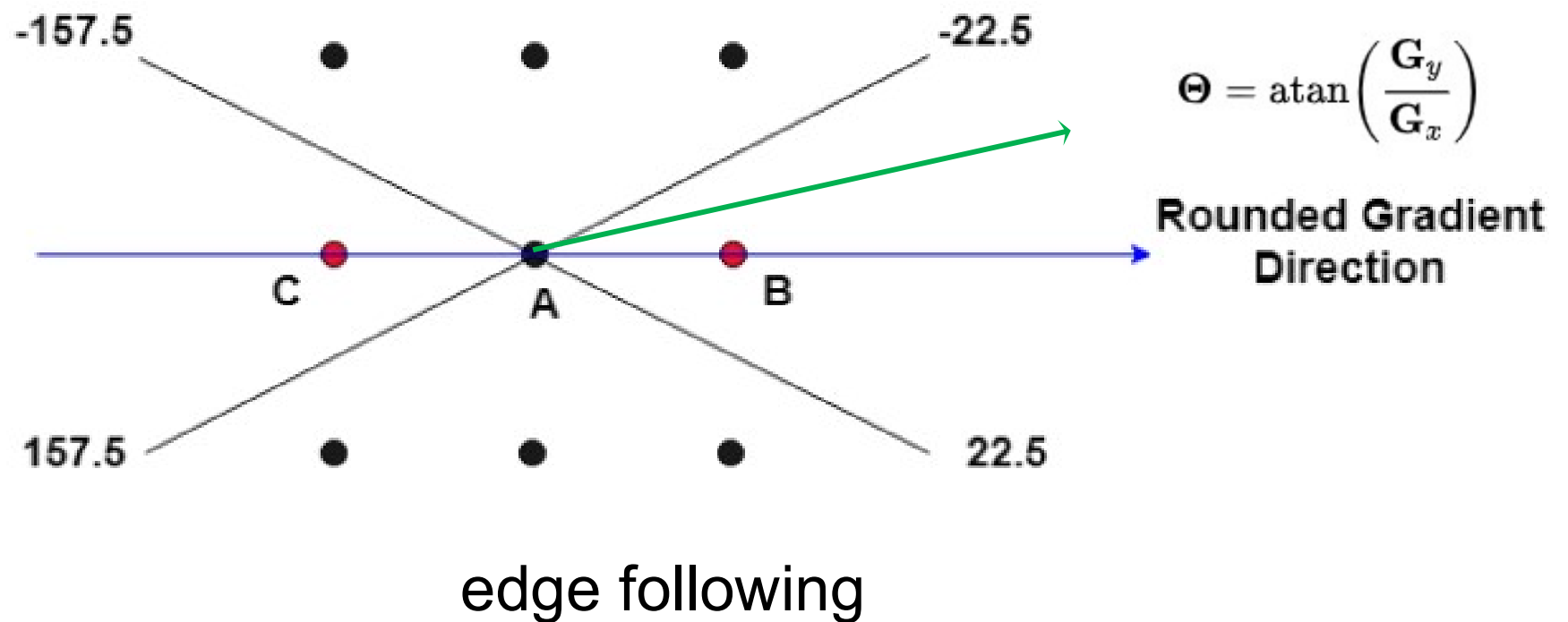
edge following



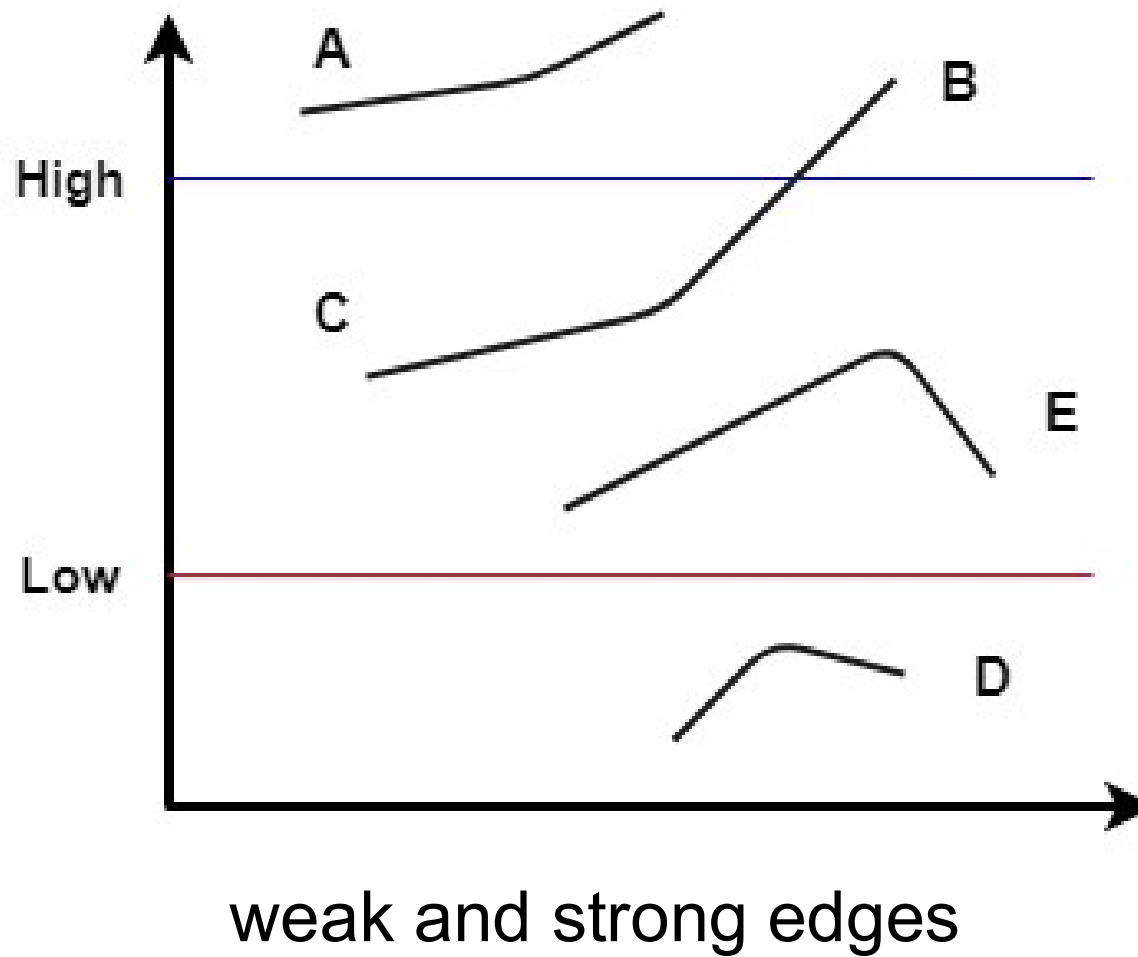
Final Output

thresholding
weak & strong edges

Non-max suppression



Thresholding



The Canny edge detector



Non-max Suppression

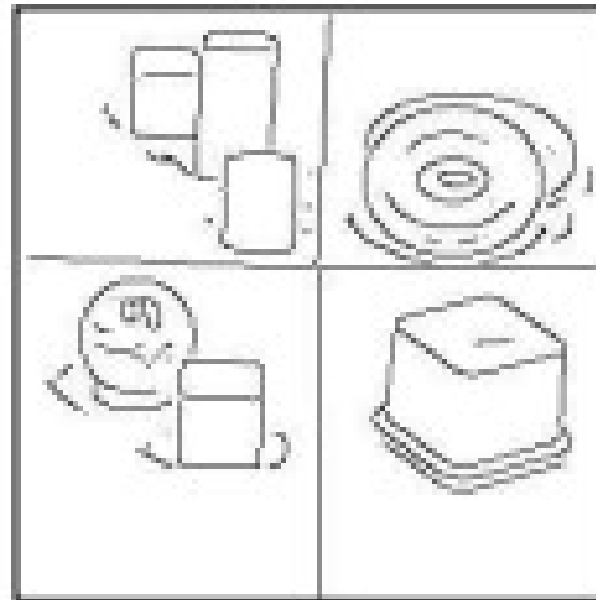
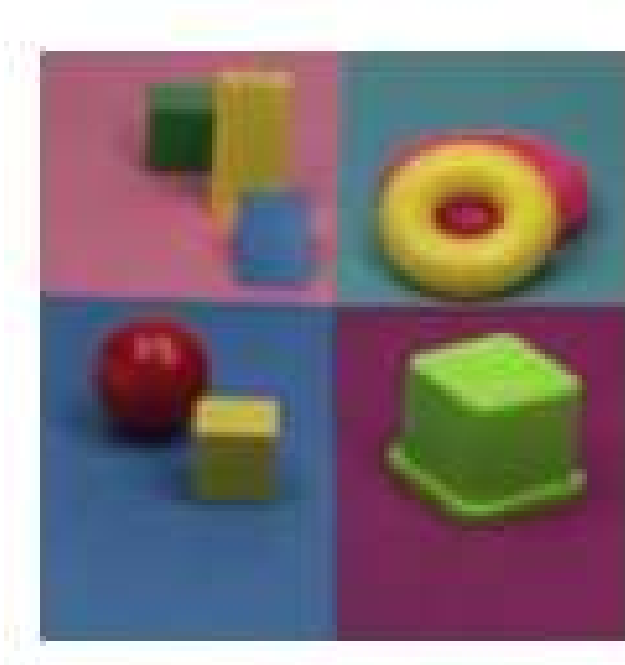
edge following



Final Output

thresholding
weak & strong edges

The Canny edge detector



Can also applied on color images

Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

A Binary Edge Map is not Enough

- Fragmented edges
- Many broken lines
- Noise in edge orientations
- Membership not clear: which points on which line ?



Any Features Here ?



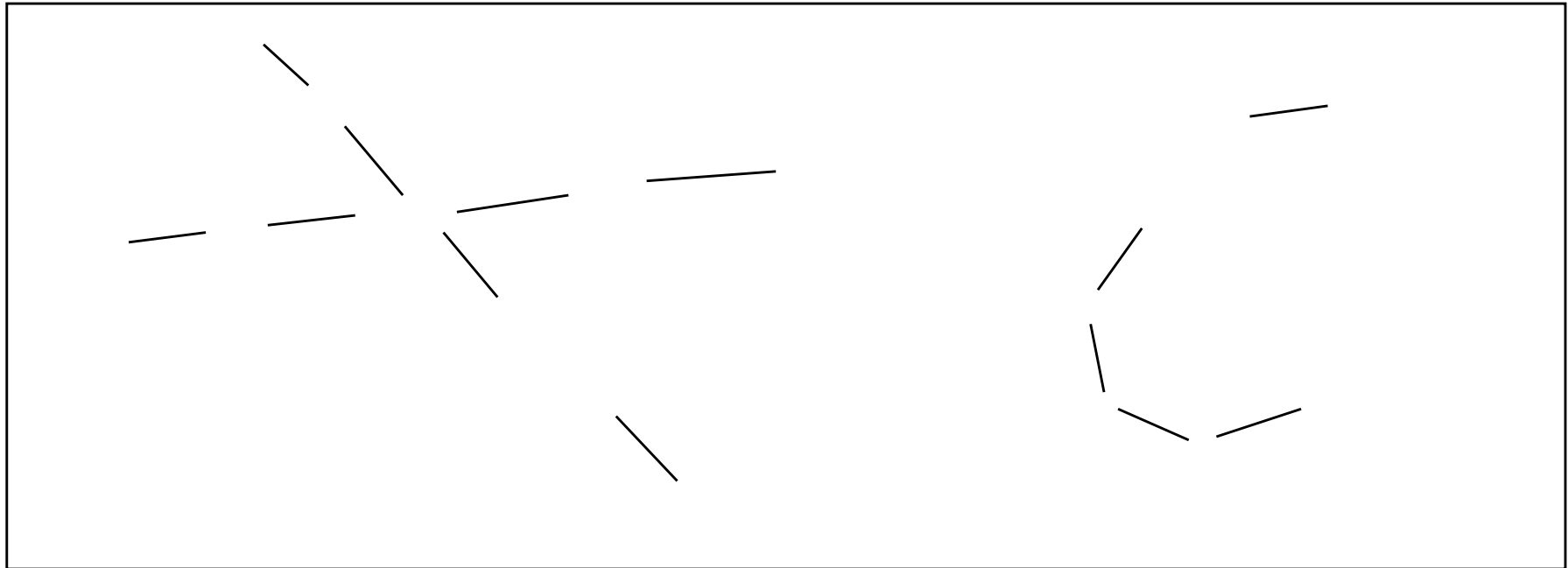
Local versus global

Vanishing lines



Goal of Line Fitting

- From fragmented edges to straight lines or curves

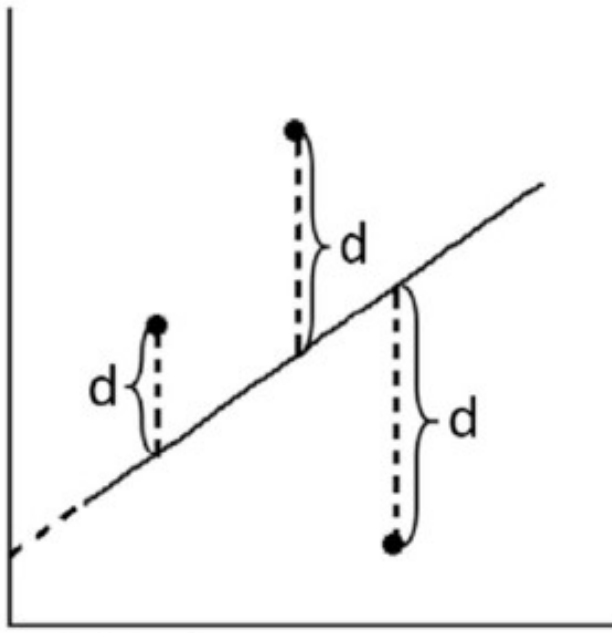


Outline

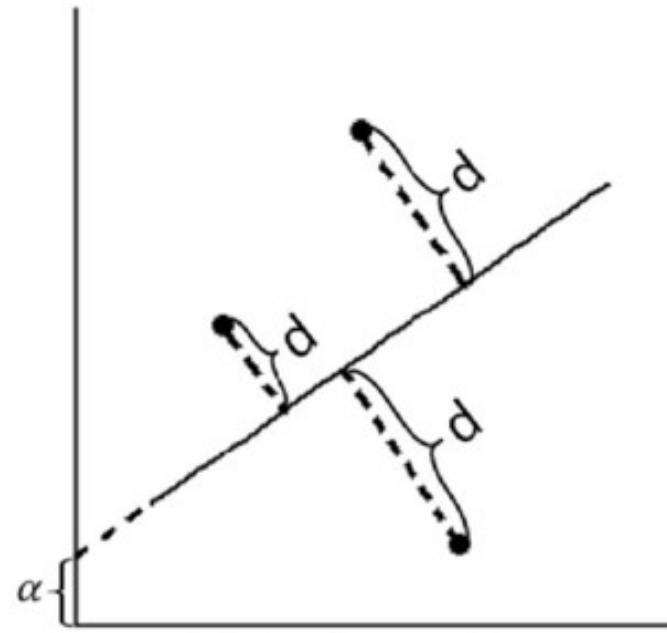
- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

Basic line fitting: Least Squares

- Given a set of points *belong to* a straight line, find the equation of the line.
- Find the line parameters that yield minimal error in a least-square sense.



Least Squares

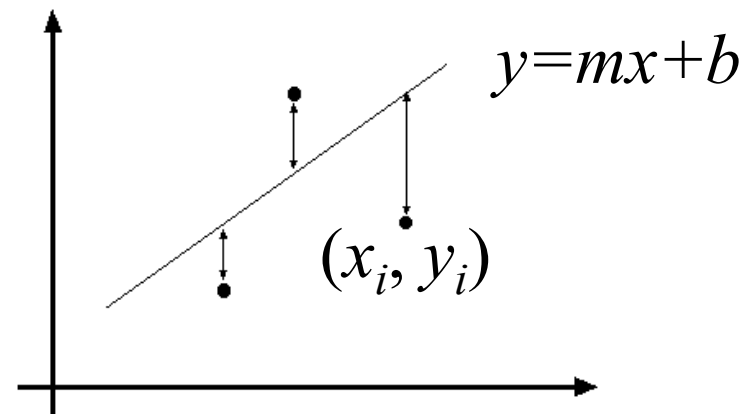


Total Least Squares

Basic line fitting: Least Squares

- Given data: $(x_1, y_1), \dots, (x_n, y_n)$, and
- Line model: $y_i = mx_i + b$
- Find $p=(m, b)$ to minimize

$$e = \sum_{i=1}^n (y_i - mx_i - b)^2$$



- Solve:

$$e = \sum_{i=1}^n \left(y_i - \begin{bmatrix} x_i & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} \right)^2 = \left\| \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} - \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} \right\|^2 = \|\mathbf{y} - \mathbf{A}p\|^2$$
$$= \mathbf{y}^T \mathbf{y} - 2(\mathbf{A}p)^T \mathbf{y} + (\mathbf{A}p)^T (\mathbf{A}p)$$

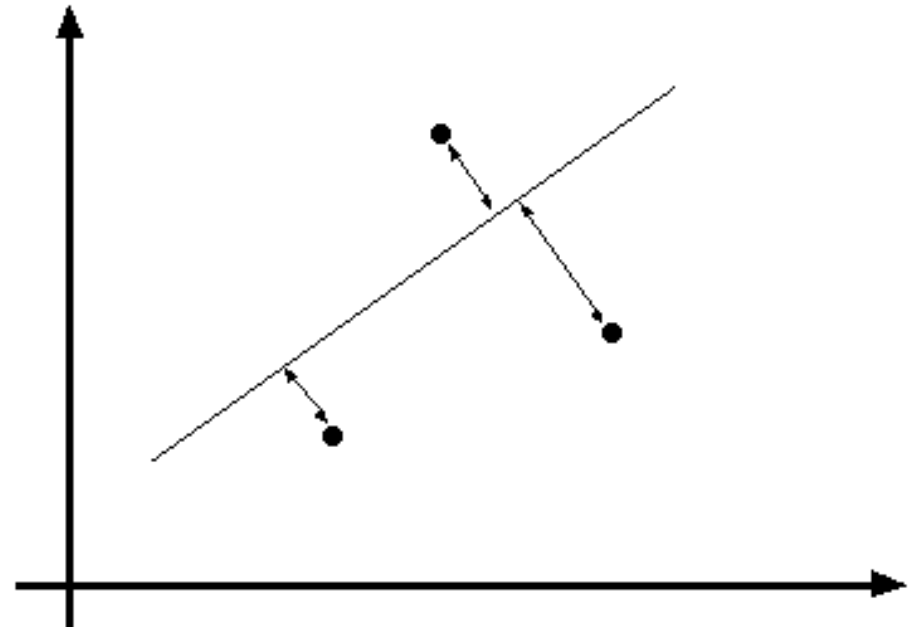
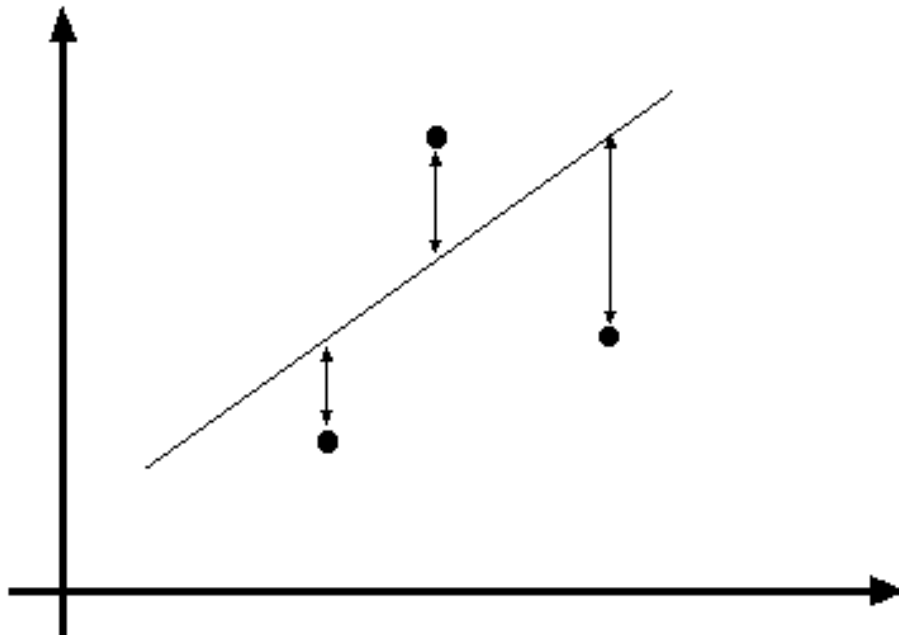
$$\frac{de}{dp} = 2\mathbf{A}^T \mathbf{A}p - 2\mathbf{A}^T \mathbf{y} = 0$$

$$\therefore p = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

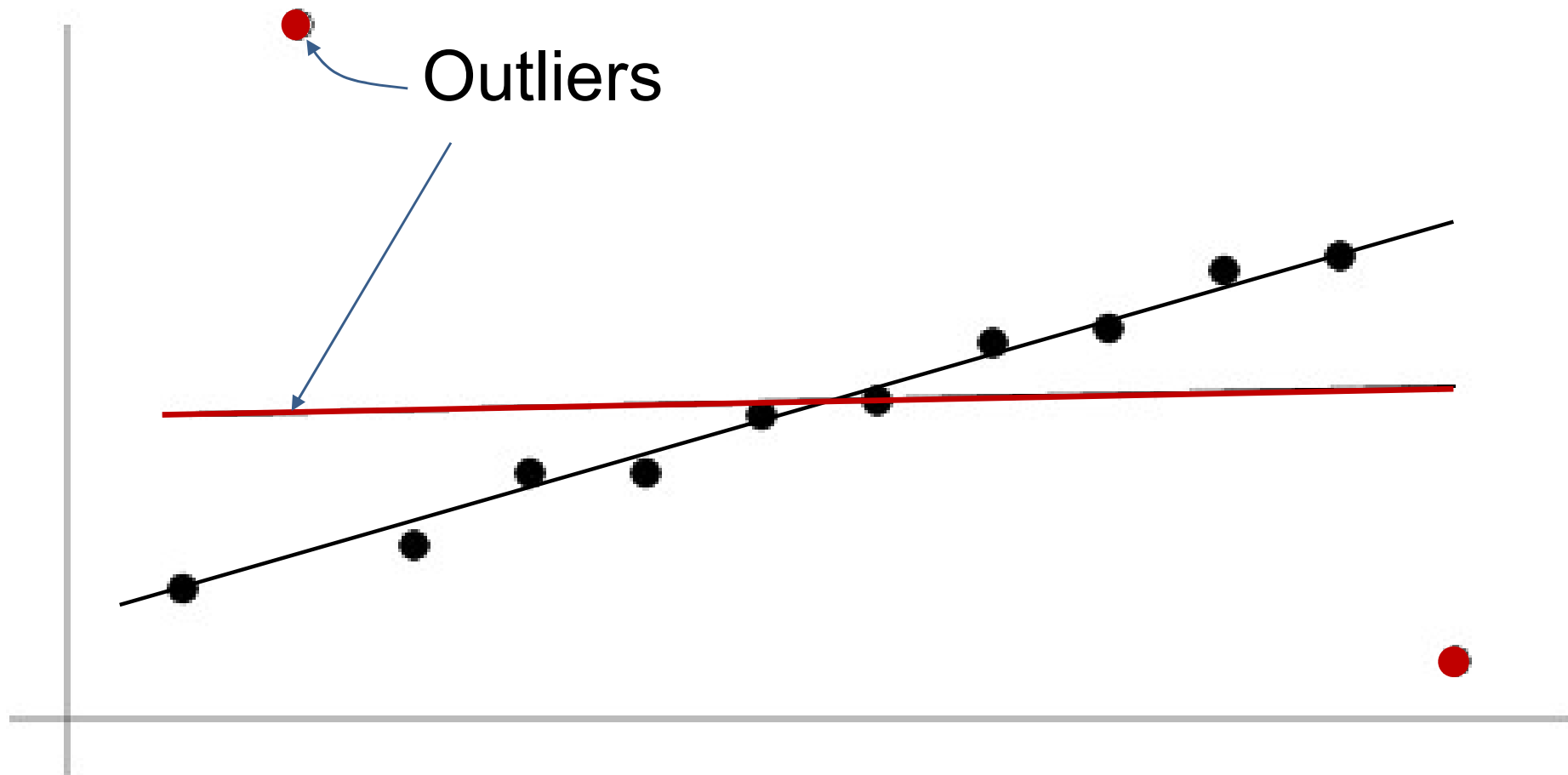
$$\text{Matlab: } p = A \setminus y;$$

Basic line fitting: Issues

- Fitting performance is affected by the slope
- Fails completely for vertical lines
- Modified least squares using a perpendicular distance



Basic line fitting: Not Robust to Outliers



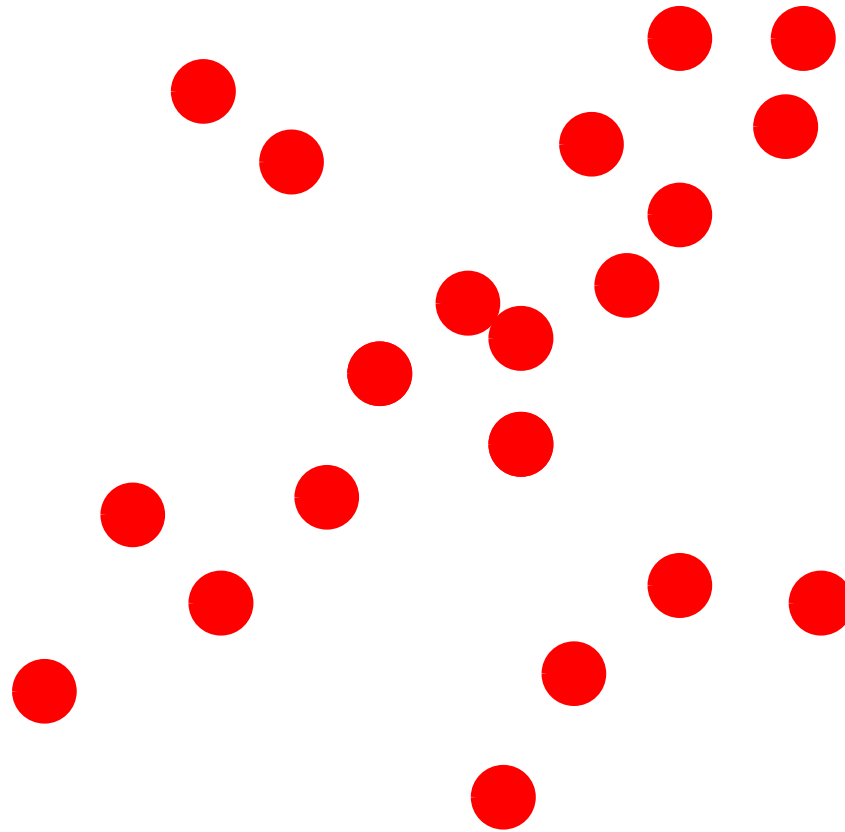
Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

RANSAC

(**RAN**dom **SA**mples **C**onsensus) :

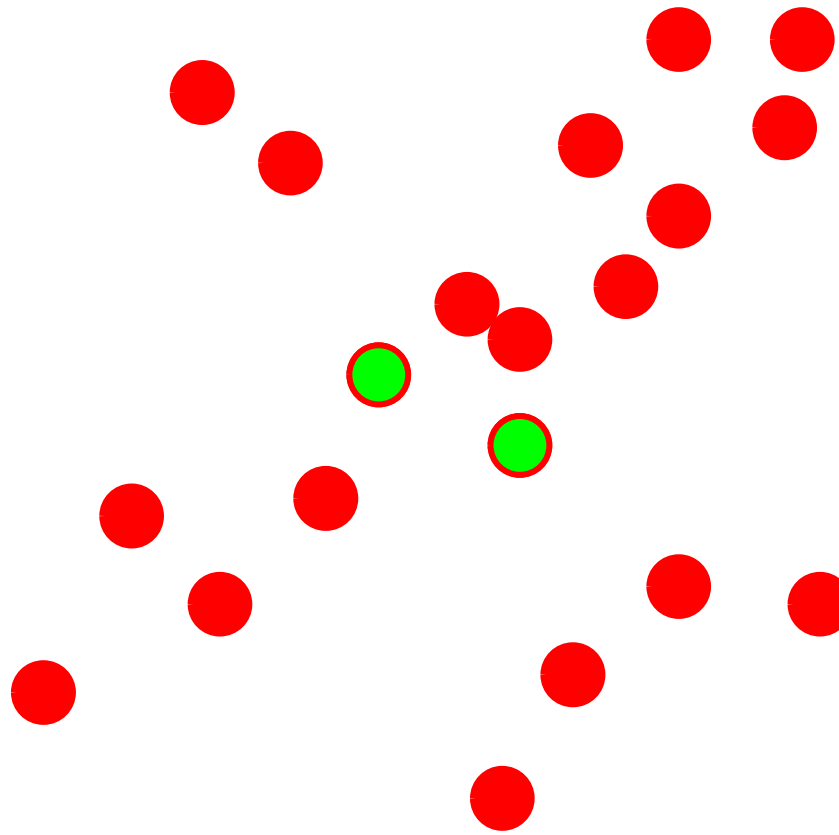
Fischler & Bolles in '81.



- Basic idea:
- Randomly select two samples (minimal number for a line) and check how many other samples fall close to the line (within a distance threshold). Choose the line which has the most number (thus consensus)
- Similar concept to Median (excluding extreme values or outliers)

RANSAC

Line fitting example



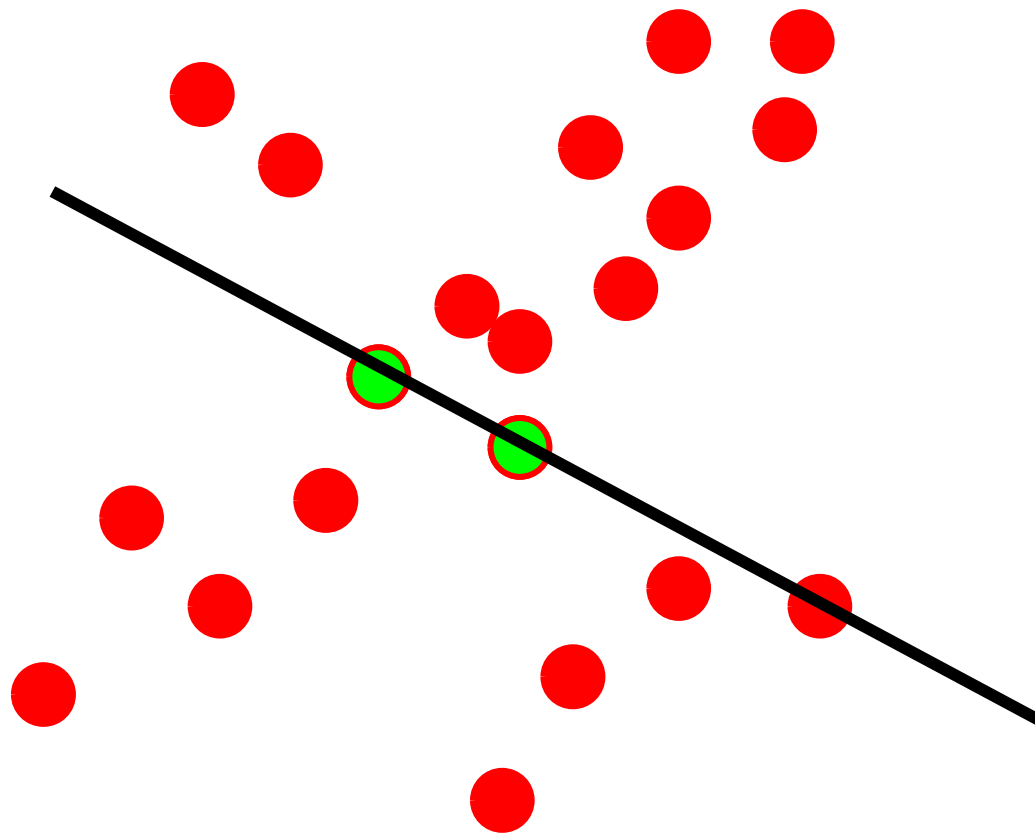
Algorithm:

1. **Randomly sample** the minimum number of points required to fit the model ($n=2$)
2. **Solve** for model parameters using the minimum set of samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC

Line fitting example



Algorithm:

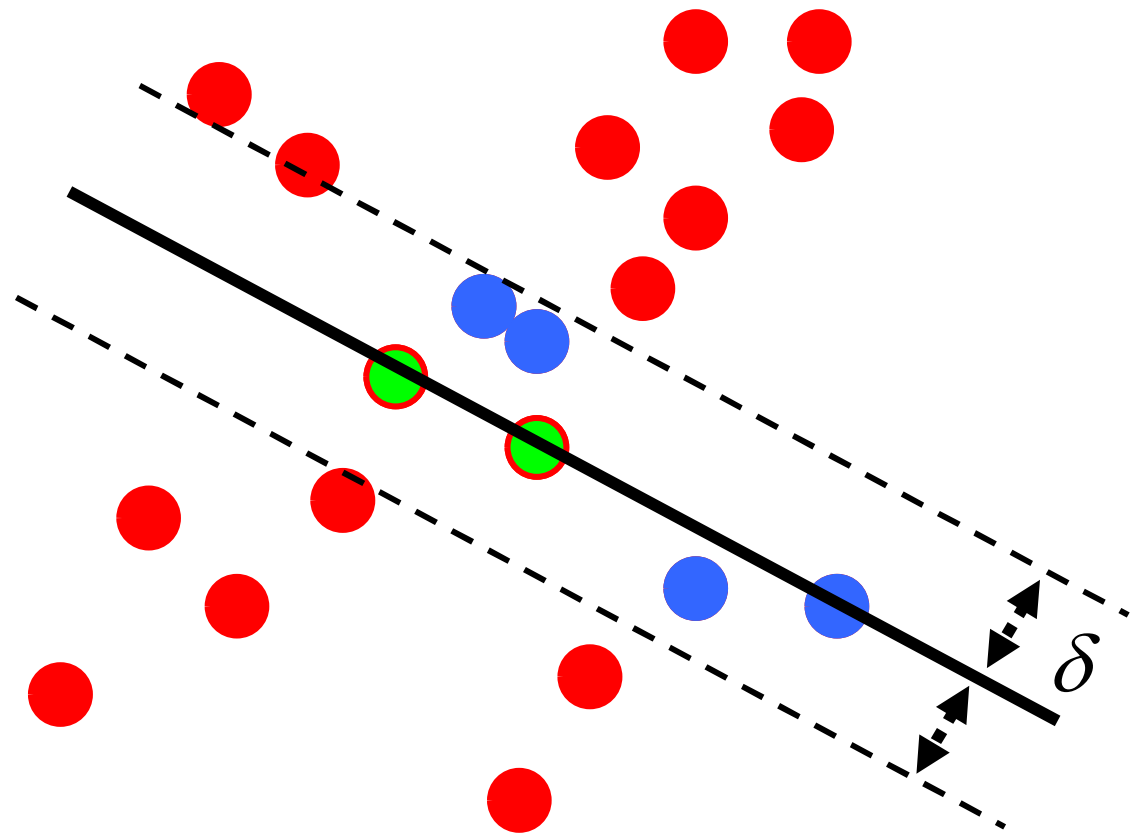
1. **Sample** (randomly) the number of points required to fit the model ($\#=2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC

Line fitting example

$$N_I = 6$$

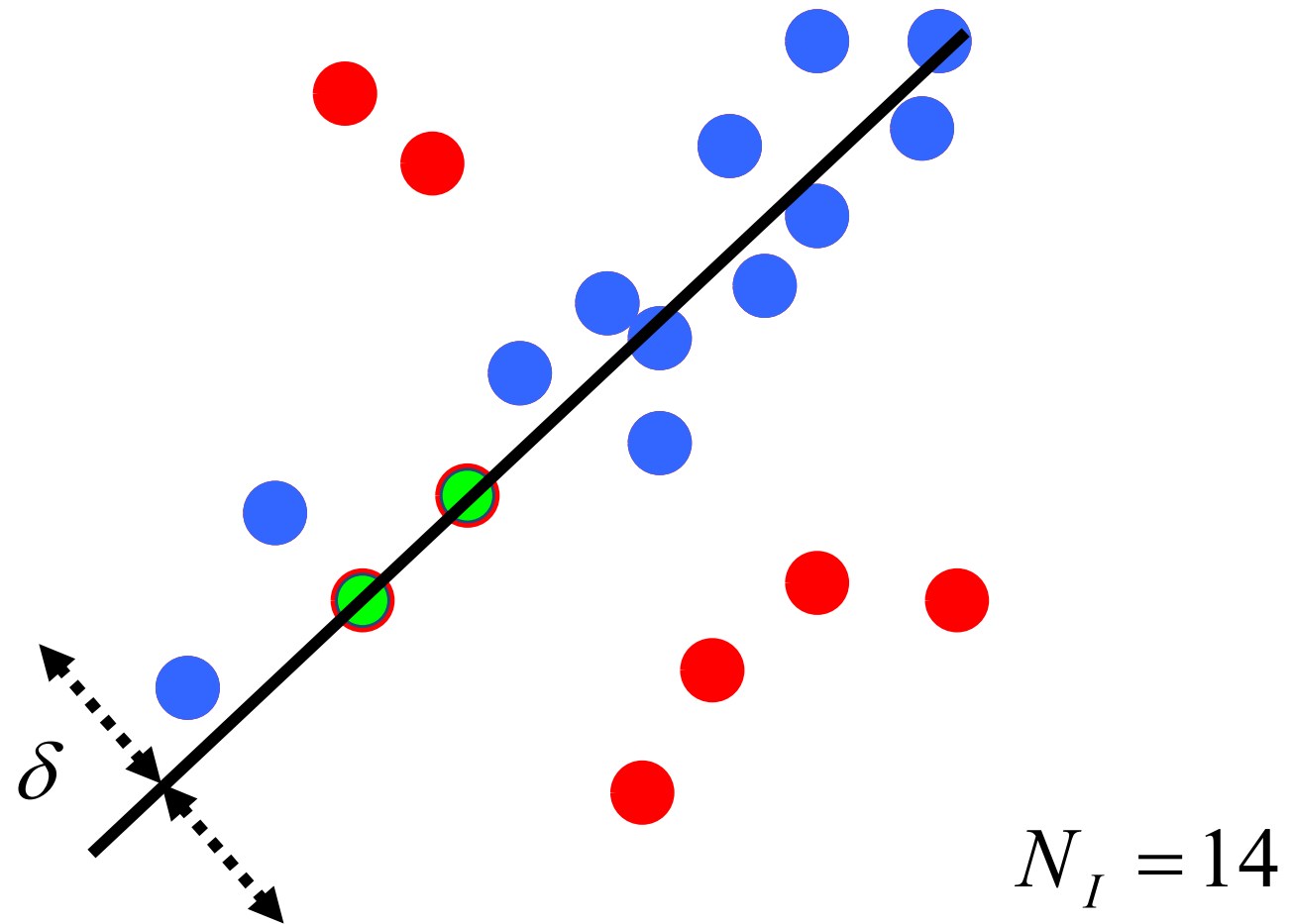


Algorithm:

1. **Sample** randomly the number of points required to fit the model ($\#=2$)
2. **Solve** for model parameters using the minimum set of samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC



Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using the minimum set of samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC: Pros and Cons

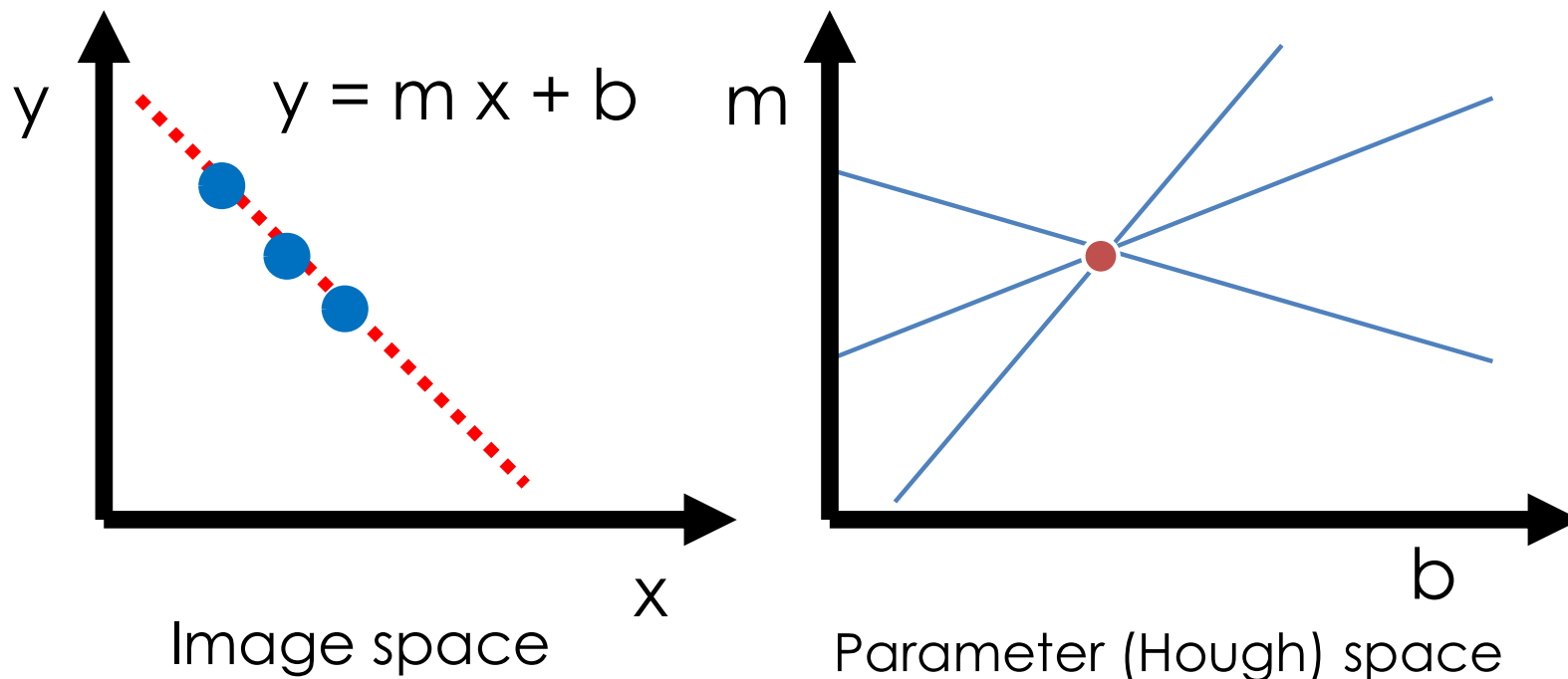
- Pros:
 - Robust to outliers
 - Applicable for large number of parameters
- Cons:
 - Computational time grows quickly with fraction of outliers and number of parameters
 - **Not good for getting multiple line fits**

Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC
 - Hough transform
- Corners
 - Harris corner
 - SIFT
 - Applications

Fitting Multiple Lines using Hough Transform

- Given a binary edge image, find the lines (or curves) that explain the data points best in the parameter space.
- This parameter space is called a Hough space

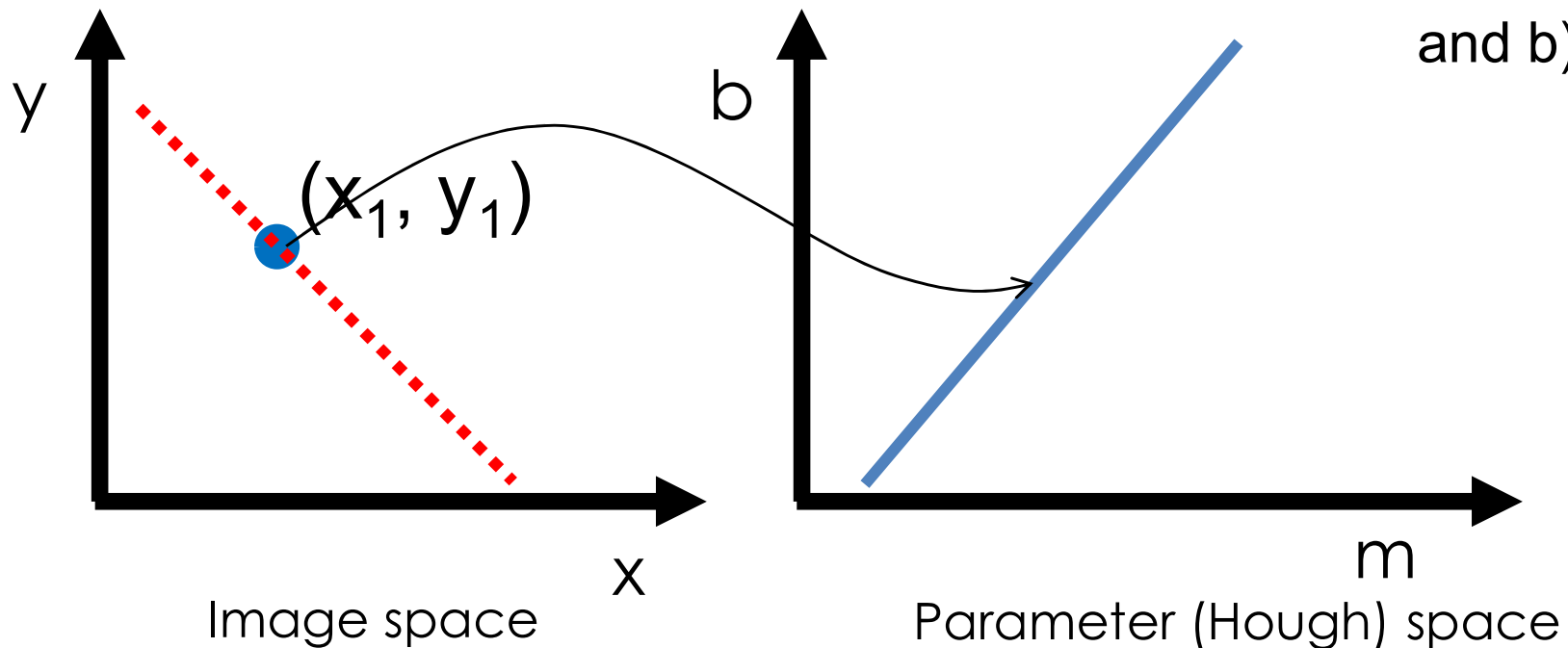


P.V.C. Hough, *Machine Analysis of Bubble Chamber Pictures*, Proc. Int. Conf. High Energy Accelerators and Instrumentation, 1959. (Patented).

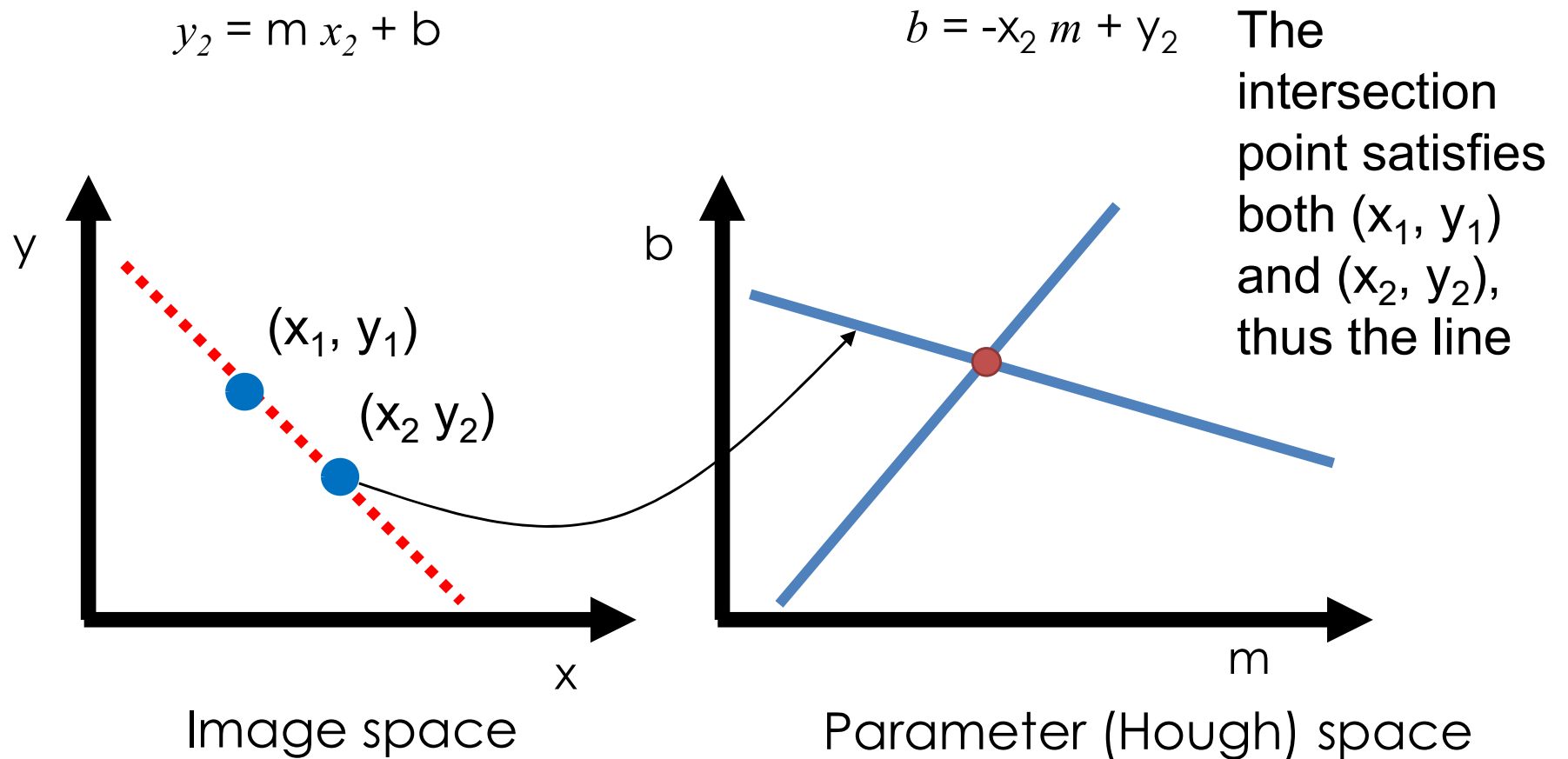
A point (x_1, y_1) is mapped to a line in the Hough space

$$y_1 = m x_1 + b$$

$$b = -x_1 m + y_1 \quad (\text{rewrite the equation in terms of } m \text{ and } b)$$



A point (x_2, y_2) is mapped to
a line in the Hough space

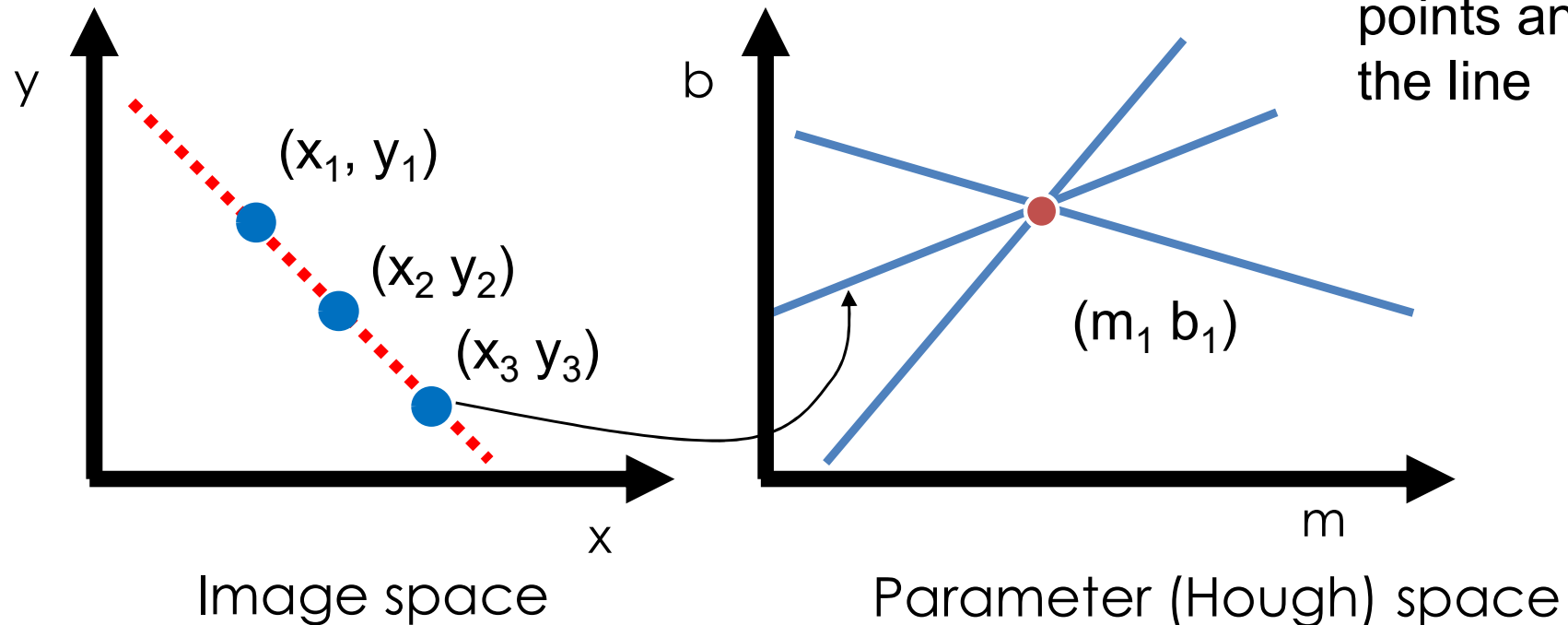


A point (x_3, y_3) is mapped to
a line in the Hough space

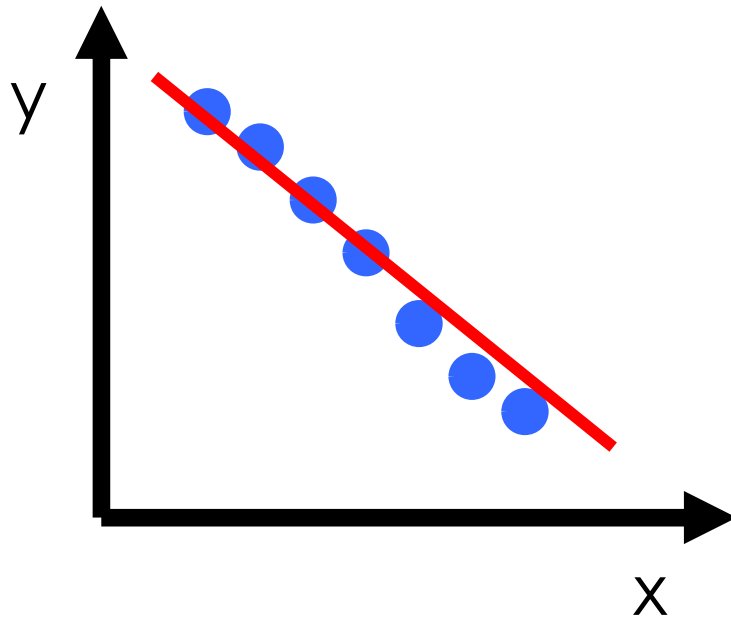
$$y_3 = m x_3 + b$$

$$b = -x_3 m + y_3$$

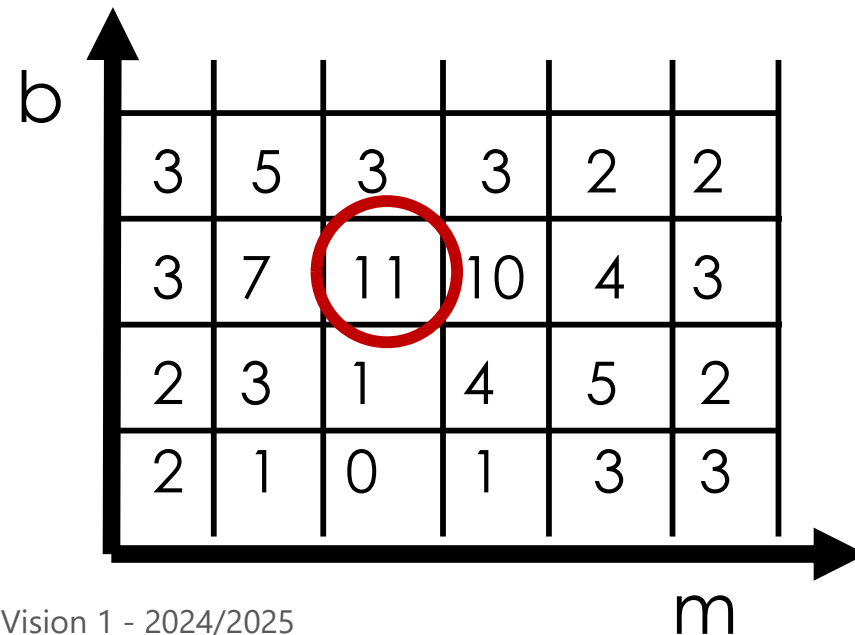
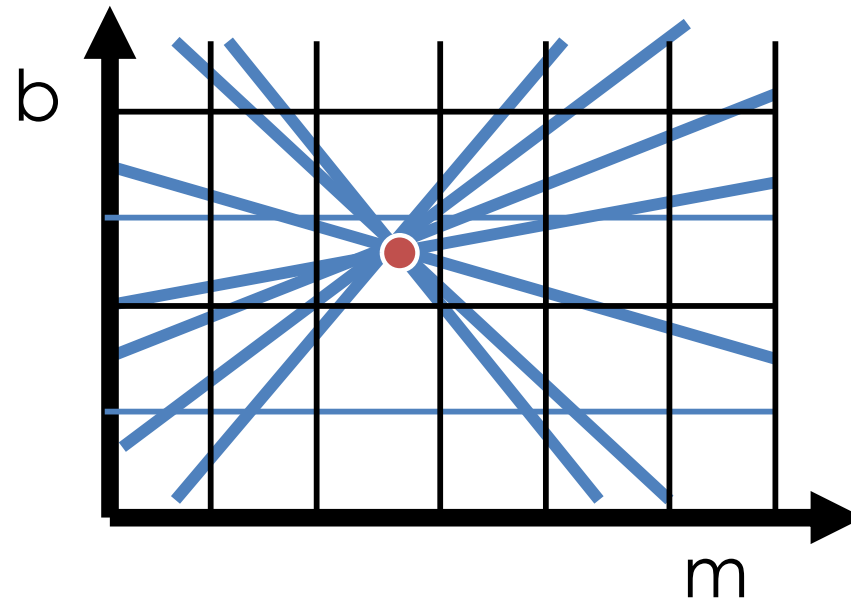
The
intersection
point (m_1, b_1)
satisfies the 3
points and thus
the line



Hough Transform: Accumulator



- For each pixel, draw a line in the discretised Hough space, assigning a value of one.
- Process all image pixels and accumulate the values
- Find the local maxima (can be multiple points due to many lines in image) – called a voting process



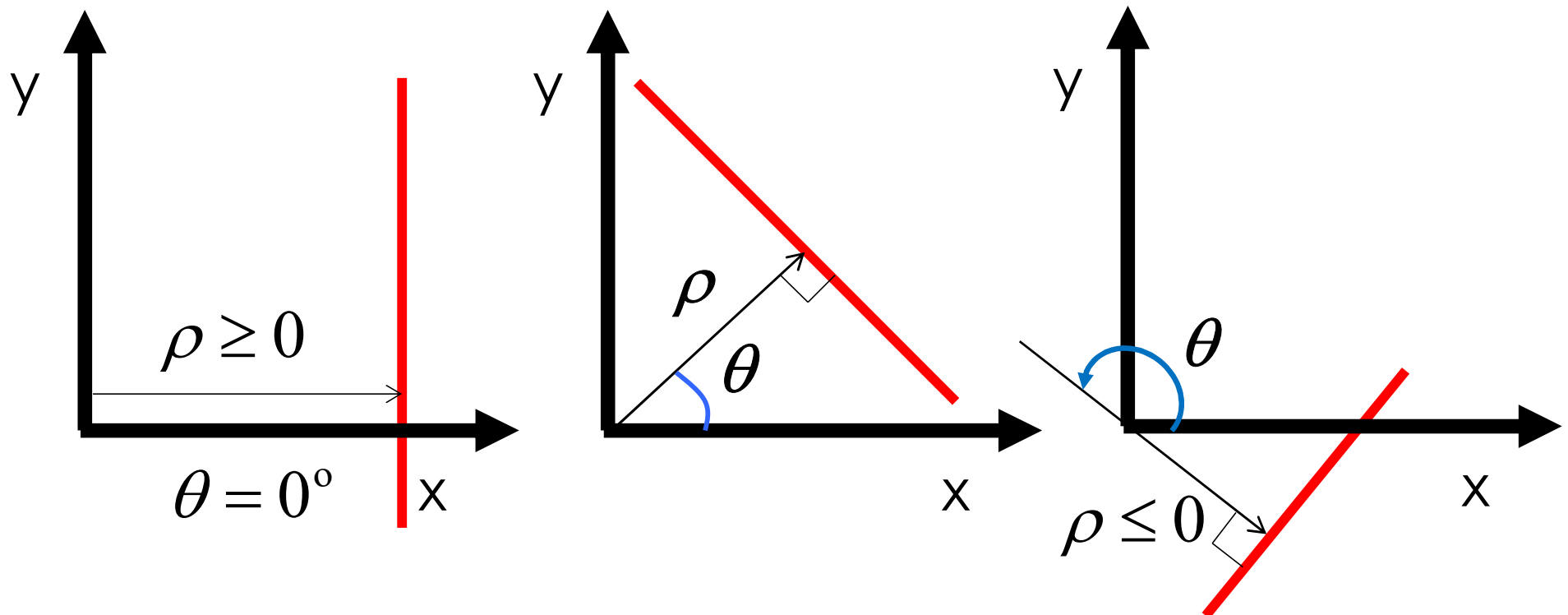
Hough Transform

- Can detect multiple lines in an image (from multiple local maxima)
- Can be easily extended for circles and ellipses
- Computationally efficient
- Problem: (m, b) are unbounded. For example, the slope parameter m can have an infinite value.

Hough Transform: Using Polar Form

Use a polar representation for the parameter space

$$x \cos \theta + y \sin \theta = \rho \quad (0 \leq \theta \leq 180^\circ)$$

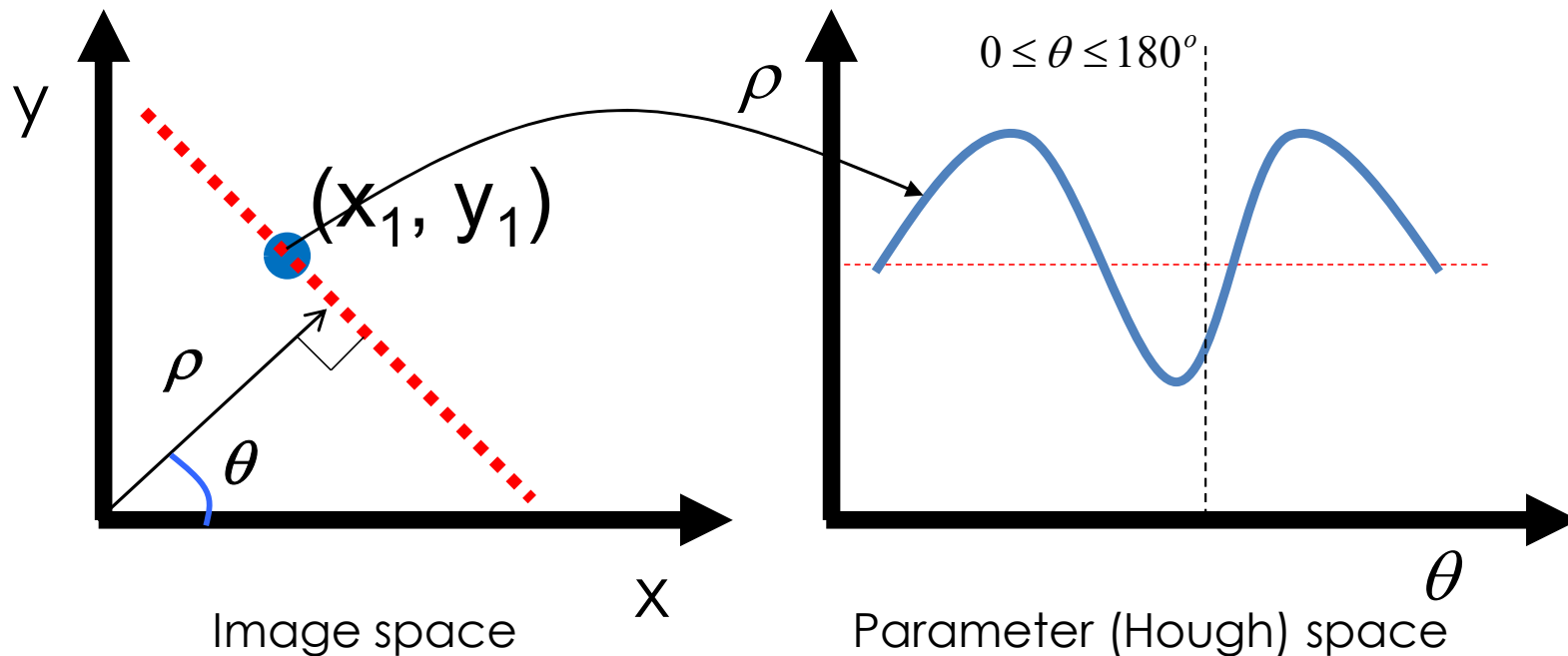


Note: ρ can have a positive or negative value

A point (x_1, y_1) is mapped to a sinusoid in the polar parameter space

$$x_1 \cos \theta + y_1 \sin \theta = \rho$$

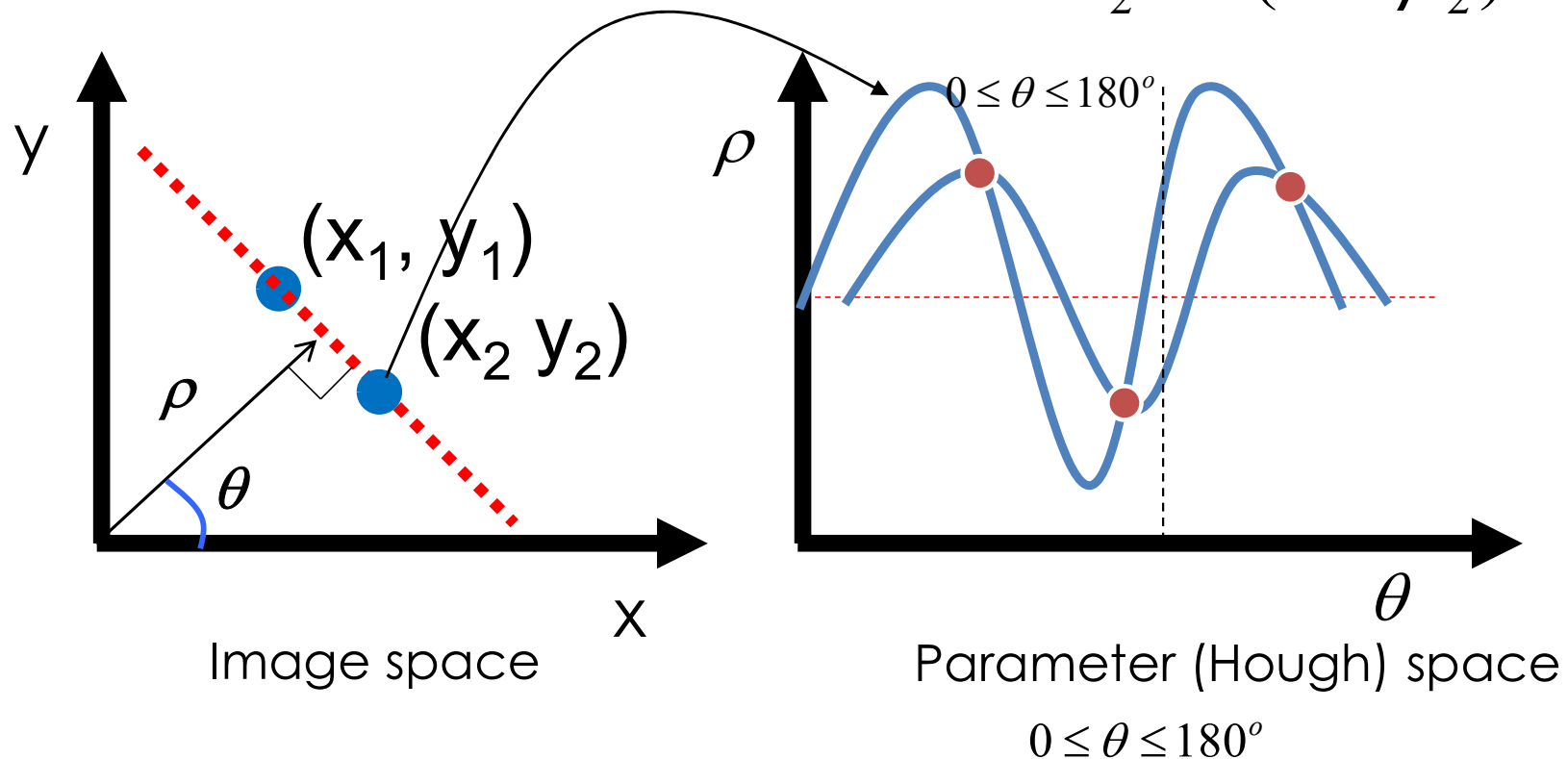
$$\begin{aligned} \rho &= x_1 \cos \theta + y_1 \sin \theta \\ &= \alpha_1 \sin(\theta + \beta_1) \end{aligned}$$



A point (x_2, y_2) is mapped to a sinusoid in the polar parameter space

$$x_2 \cos \theta + y_2 \sin \theta = \rho$$

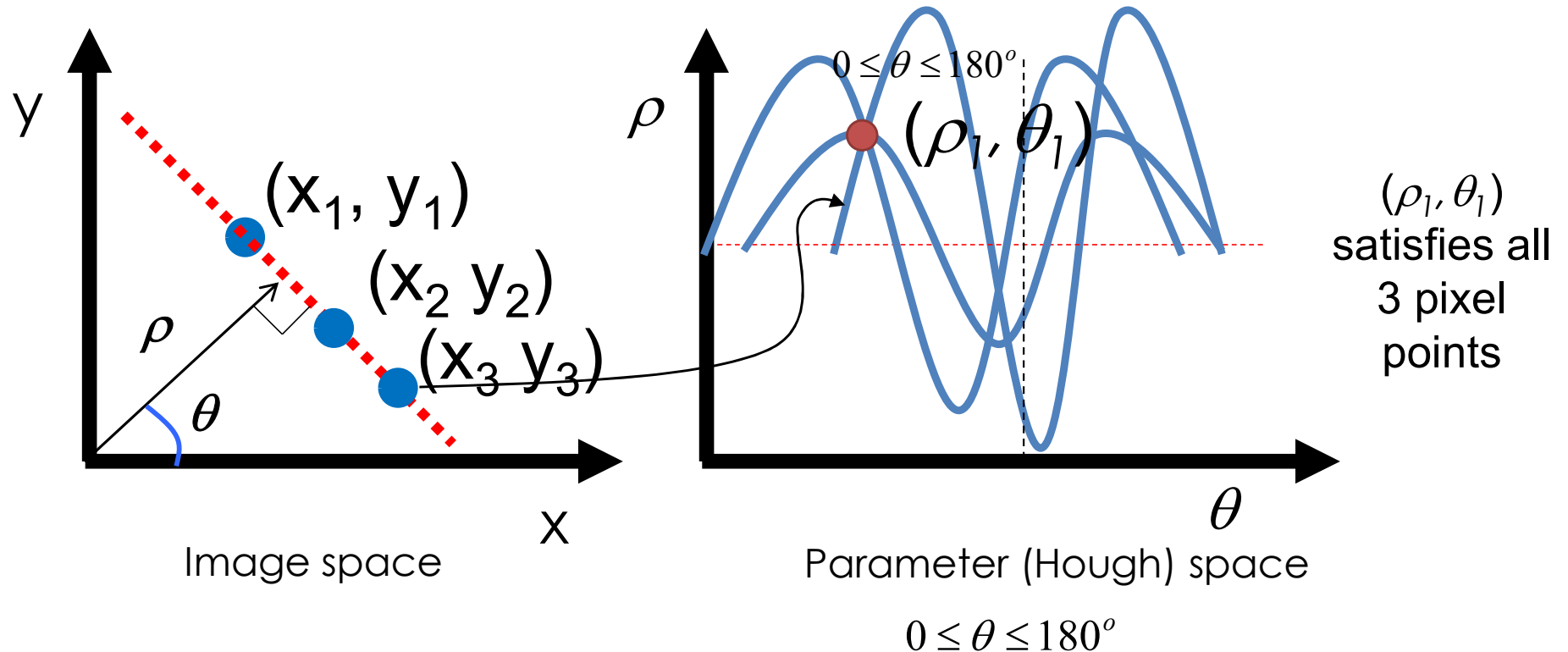
$$\begin{aligned} \rho &= x_2 \cos \theta + y_2 \sin \theta \\ &= \alpha_2 \sin(\theta + \beta_2) \end{aligned}$$



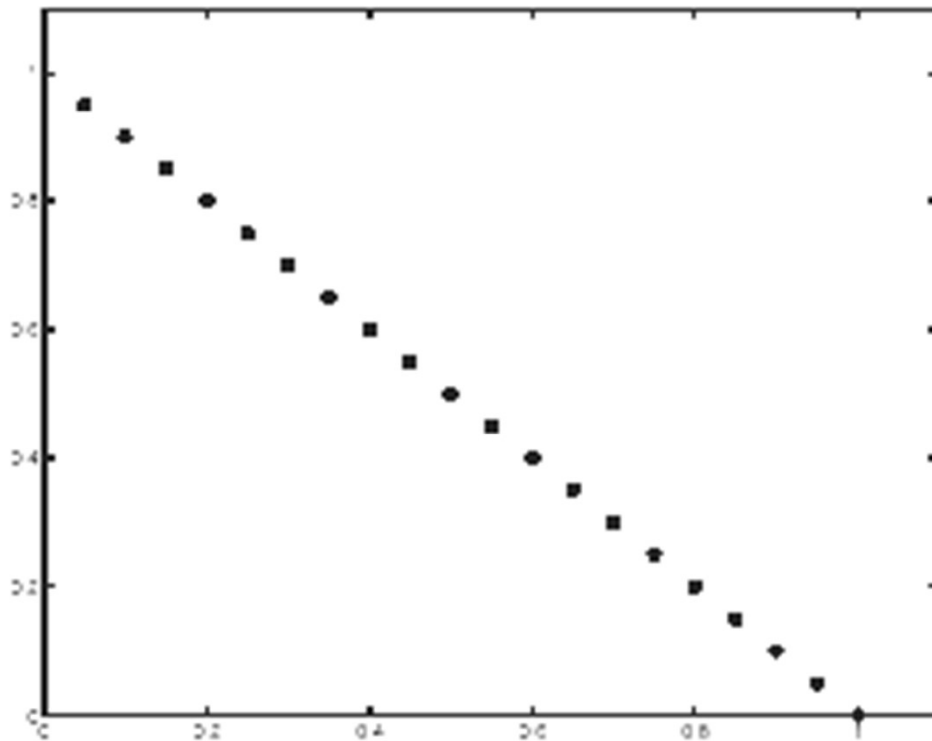
A point (x_3, y_3) is mapped to a sinusoid in the polar parameter space

$$x_3 \cos \theta + y_3 \sin \theta = \rho$$

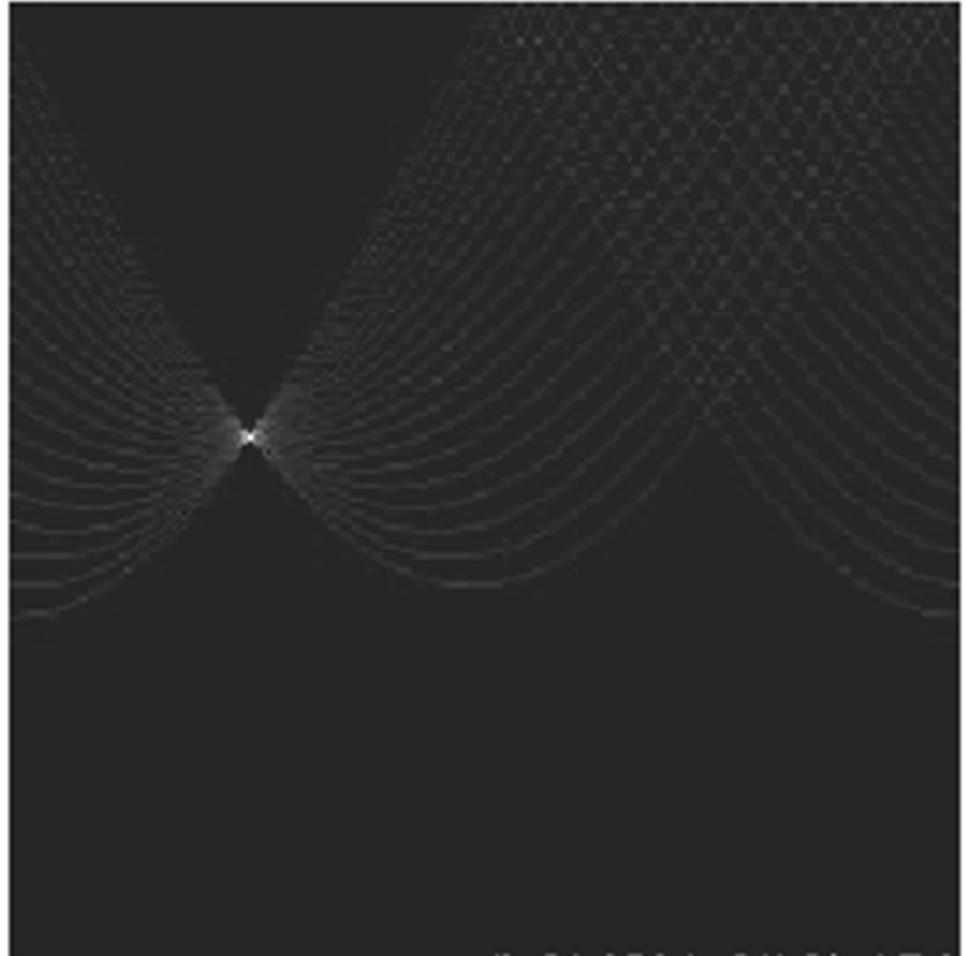
$$\begin{aligned} \rho &= x_3 \cos \theta + y_3 \sin \theta \\ &= \alpha_3 \sin(\theta + \beta_3) \end{aligned}$$



Hough Transform - Example



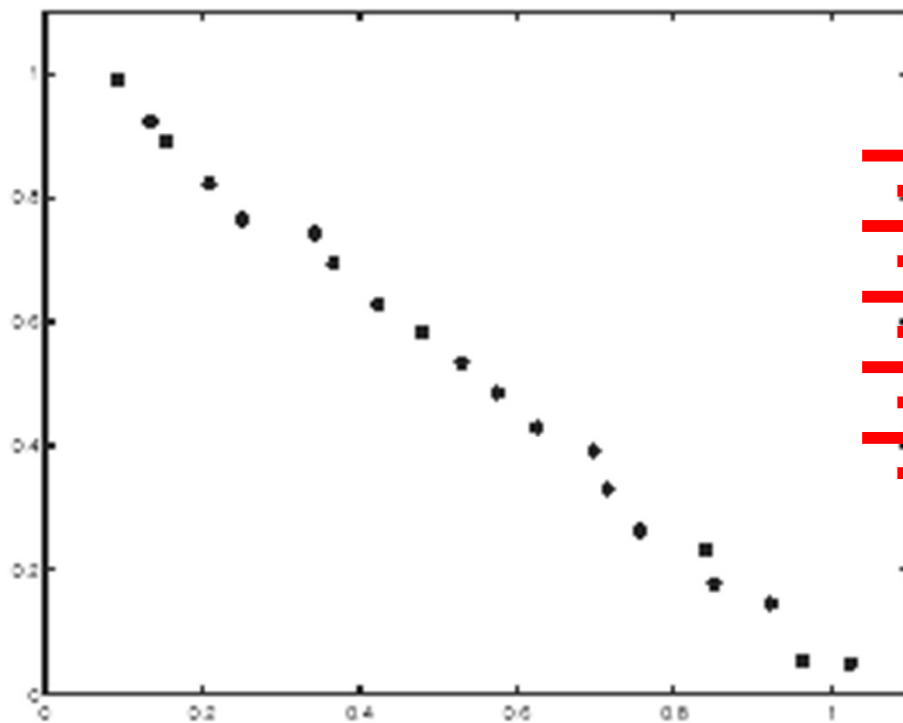
features



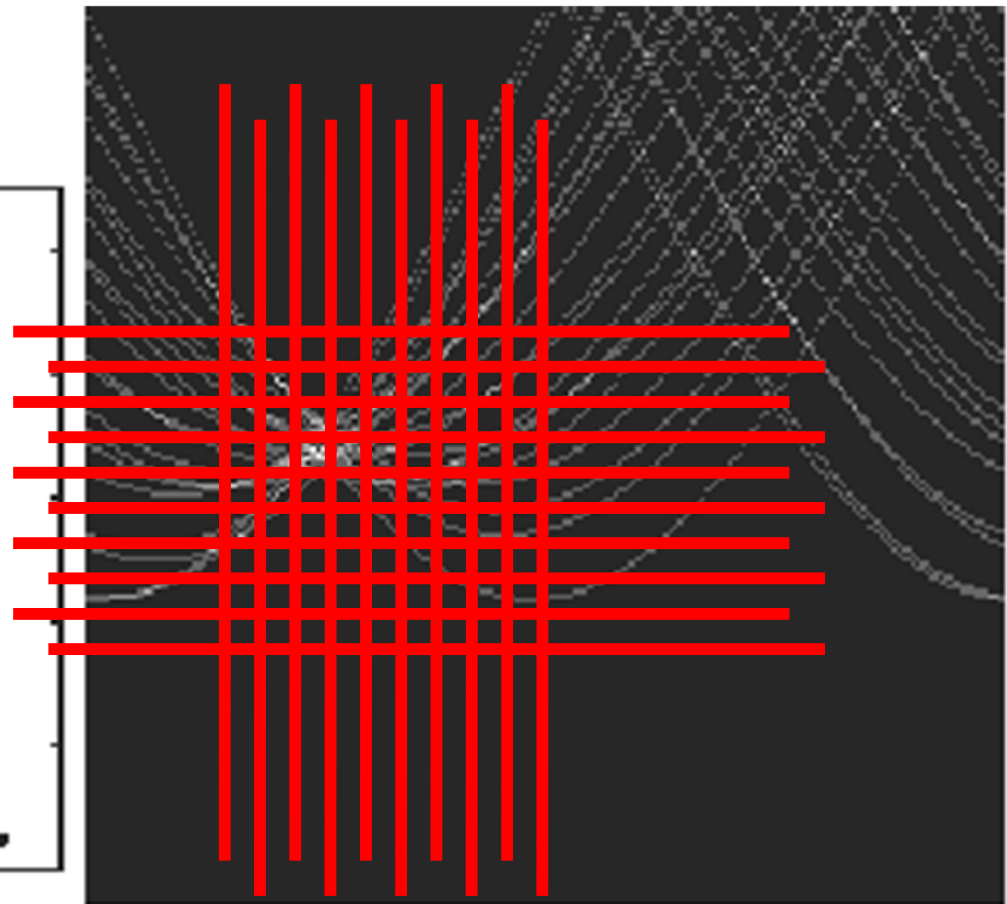
votes

Hough Transform - Example

Noisy data



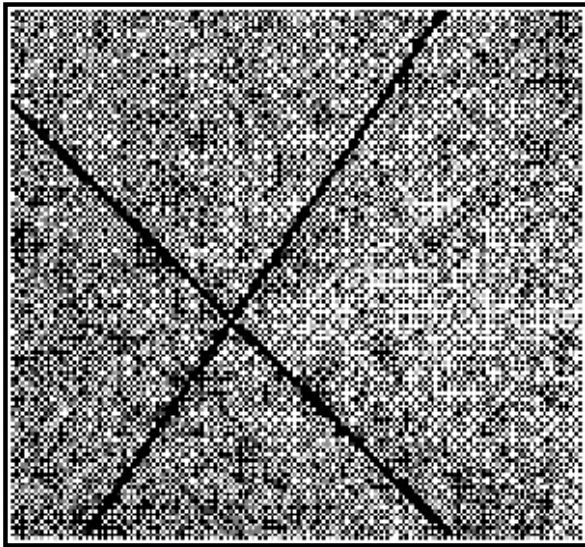
features



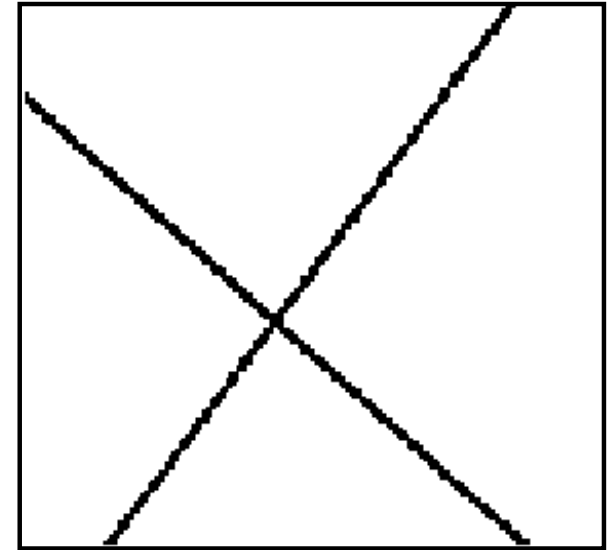
votes

Issue: the grid size still needs to be adjusted...

Hough Transform - Example



Image

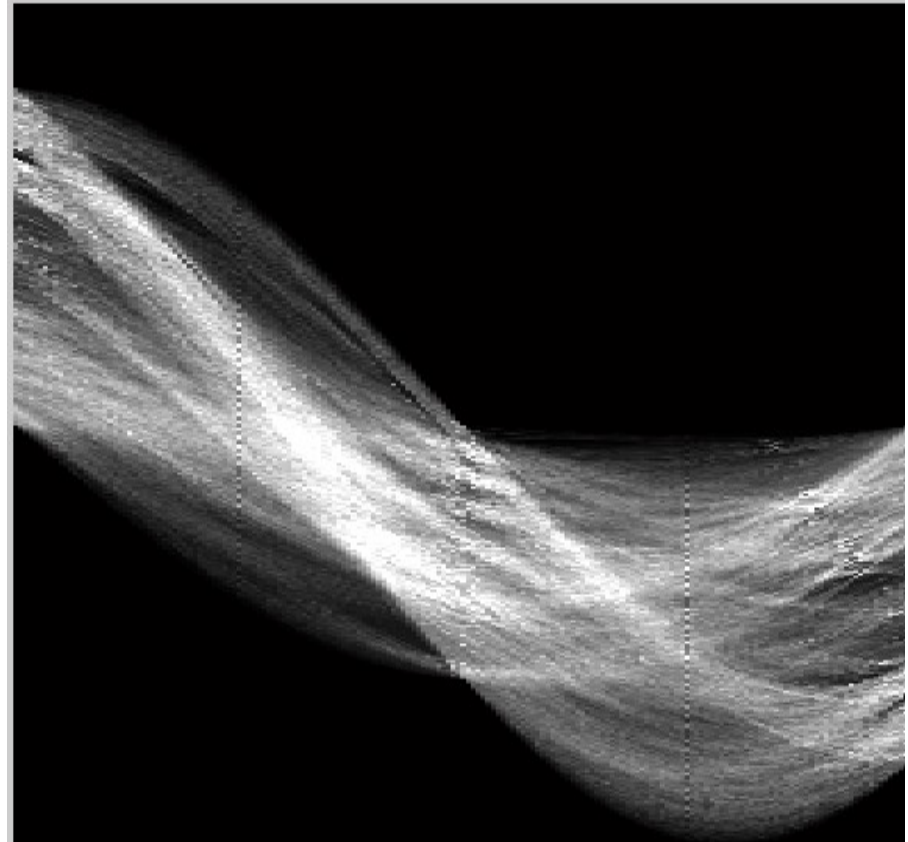


Edge detection Hough Transform

1) Image → Canny Edges

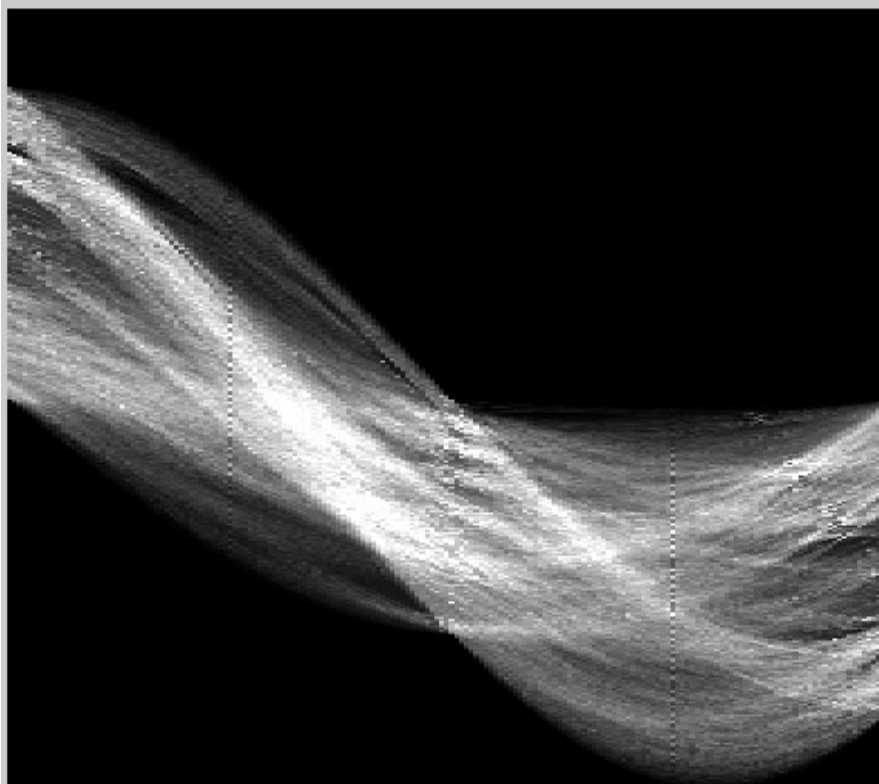


2) Canny Edges $\rightarrow$ Hough votes

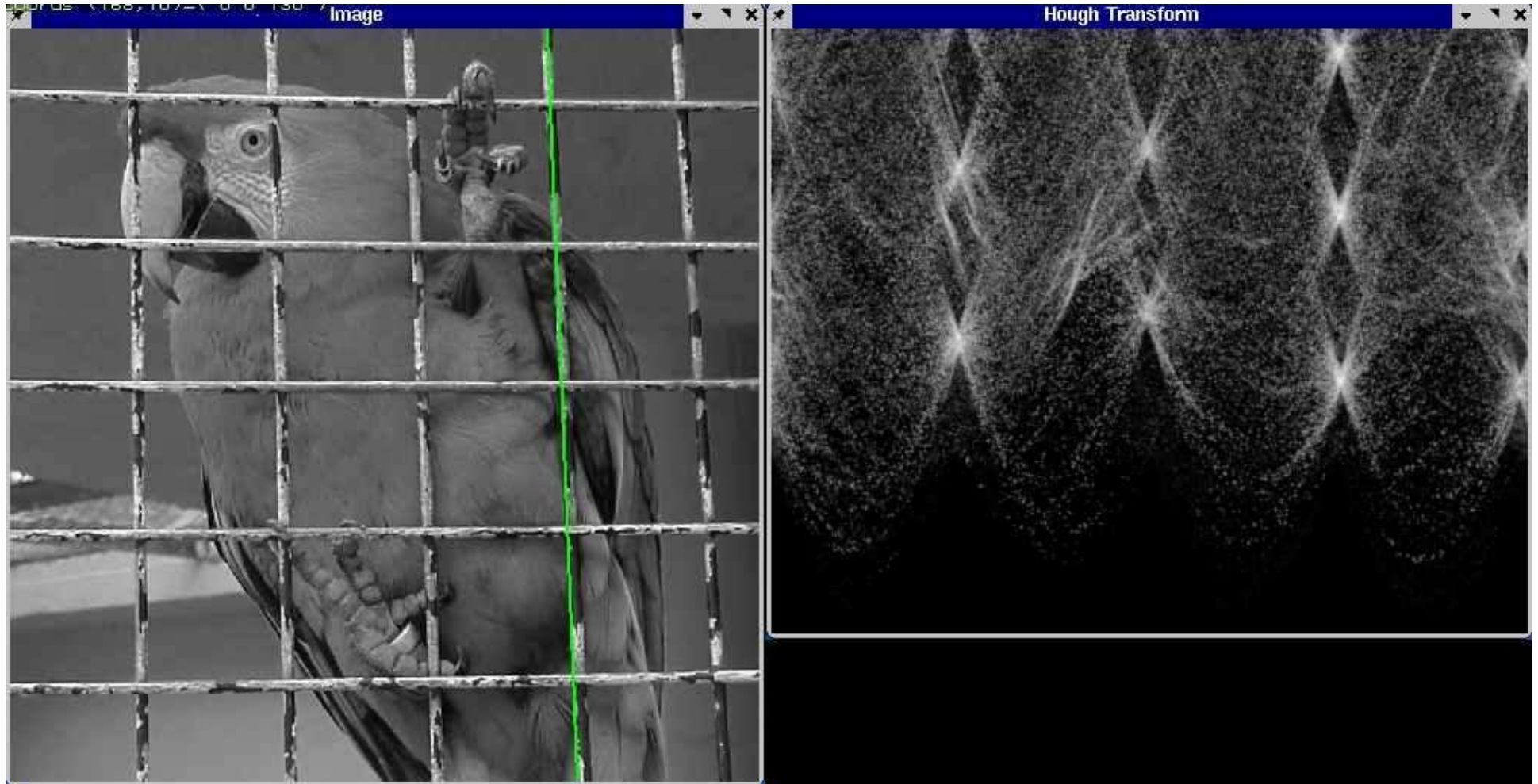


3) Hough votes $\rightarrow$ Lines

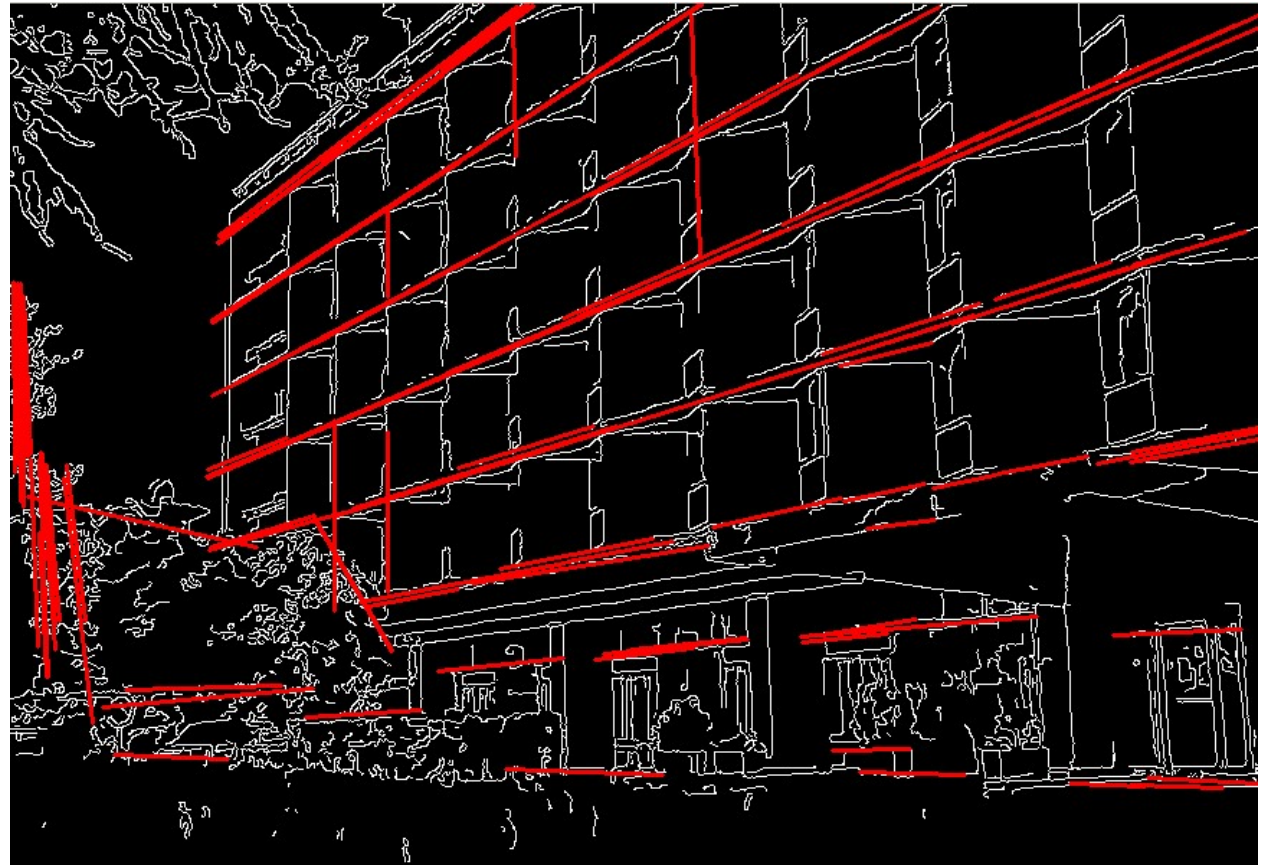
Find peaks and post-process



Hough Transform - Example



Hough Transform - Example



OpenCV: probabilistic Hough transform

Hough transform pros and cons

Pros

- All points are processed independently, so can cope with occlusion
- Some robustness to noise: noise points unlikely to contribute consistently to any single bin
- Can detect multiple instances of a model in a single pass

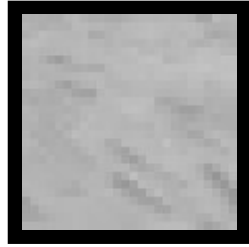
Cons

- Complexity of search time increases exponentially with the number of model parameters
- Non-target shapes can produce spurious peaks in parameter space
- Quantization: hard to pick a good grid size

Outline

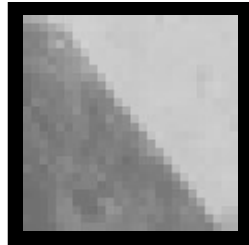
- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC
 - Hough transform
- Corners
 - Harris corner
 - SIFT
 - Applications

Interest Points



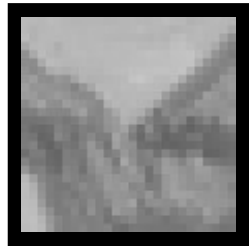
0D structure: **single points**

➡ not useful for matching



1D structure: **lines**

➡ edge, can be localised in 1D,
subject to the aperture problem



2D structure: **corners**

➡ corner, or **interest point**, can be
localised in 2D, good for matching

Interest Points have **2D** structure.

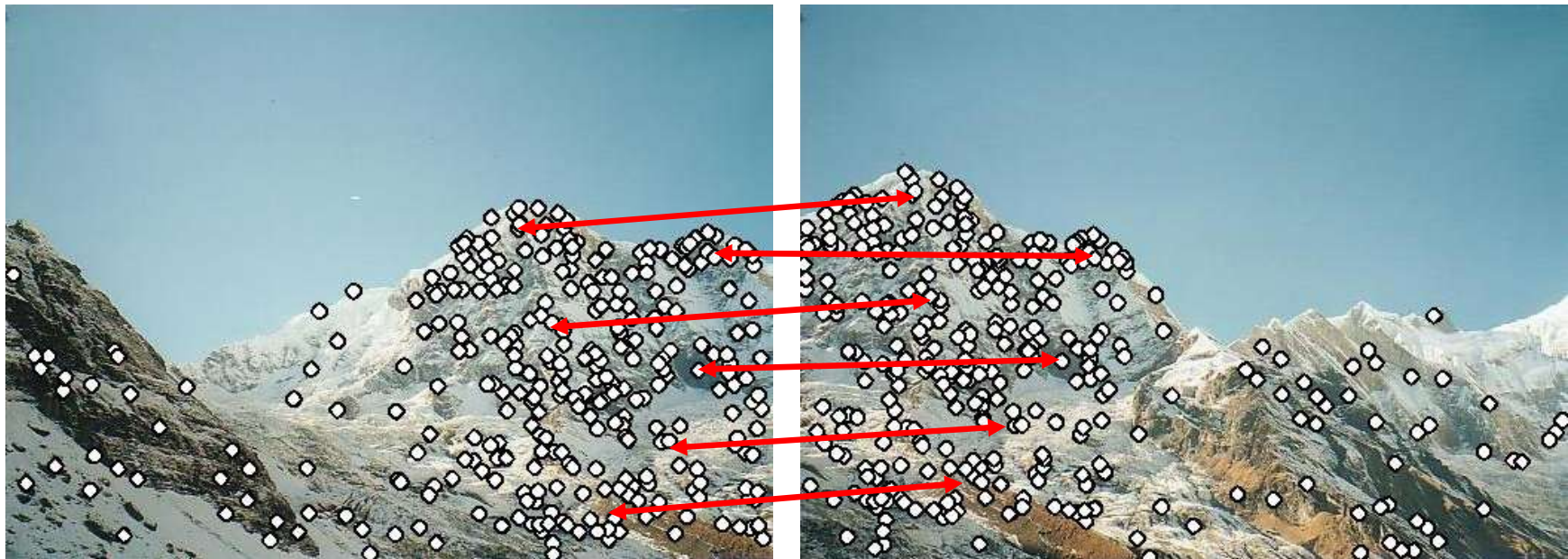
Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Why extract corner features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract features

Step 2: match features

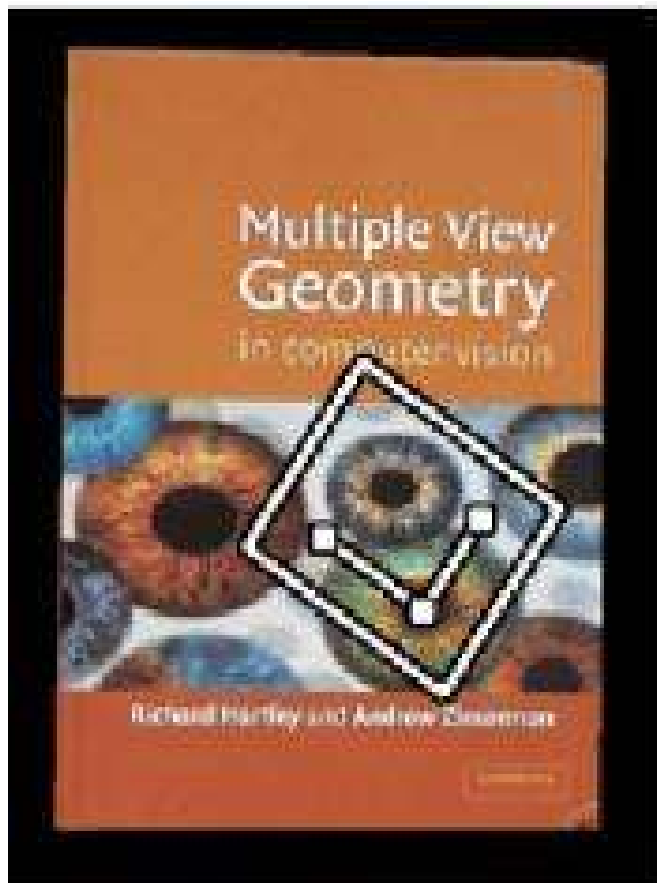
Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract features
Step 2: match features
Step 3: align images

Feature Matching



Feature Matching



Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC
 - Hough transform
- Corners
 - Harris corner
 - SIFT
 - Applications

Harris Detector: Mathematics

Change of intensity for the shift $[u, v]$:

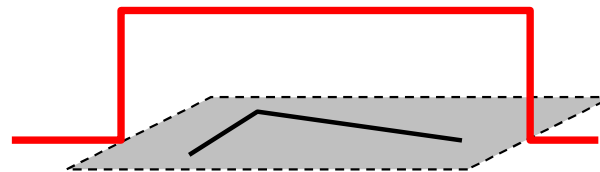
$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

Window
function

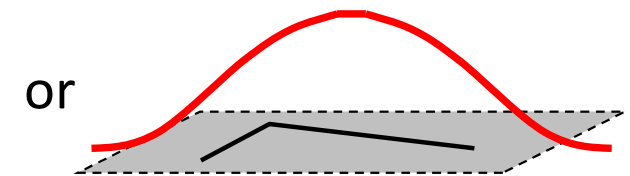
Shifted
intensity

Intensity

Window function $w(x, y) =$



1 in window, 0 outside



Gaussian

Measuring the “Properties” of $E()$

$$E(u, v) = \sum [I(x - u, y - v) - I(x, y)]^2 =$$

$$= \sum_{x,y} [I(x, y) - I_x u - I_y v + \varepsilon - I(x, y)]^2 \approx$$

$$\approx \sum_{x,y} [-I_x u - I_y v]^2 =$$

$$= \sum_{x,y} I_x^2 u^2 + I_x u I_y v + I_y v I_x u + I_y^2 v^2 =$$

$$= \sum_{x,y} [u \quad v] \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E(u, v) \cong [u, v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix}$$

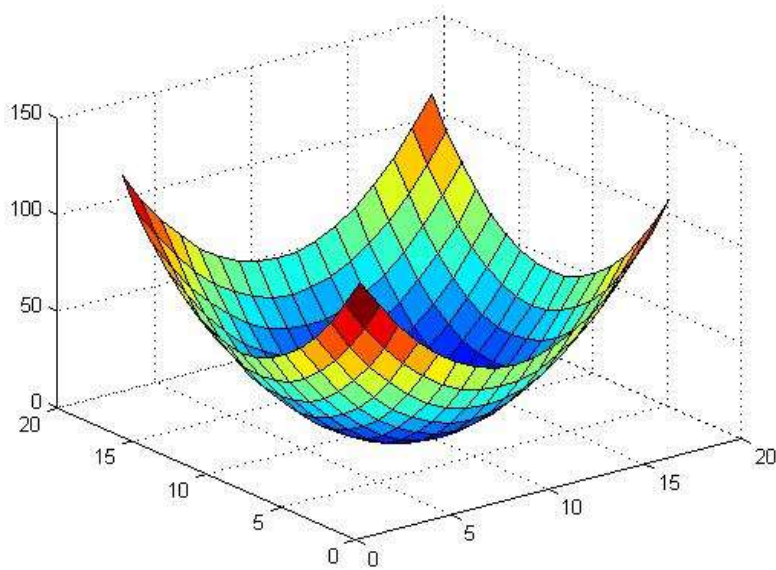
M depends on image properties

Quadratic Form and its Eigenvalue

$$E(u, v) \cong [u, v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = U^T \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} U =$$

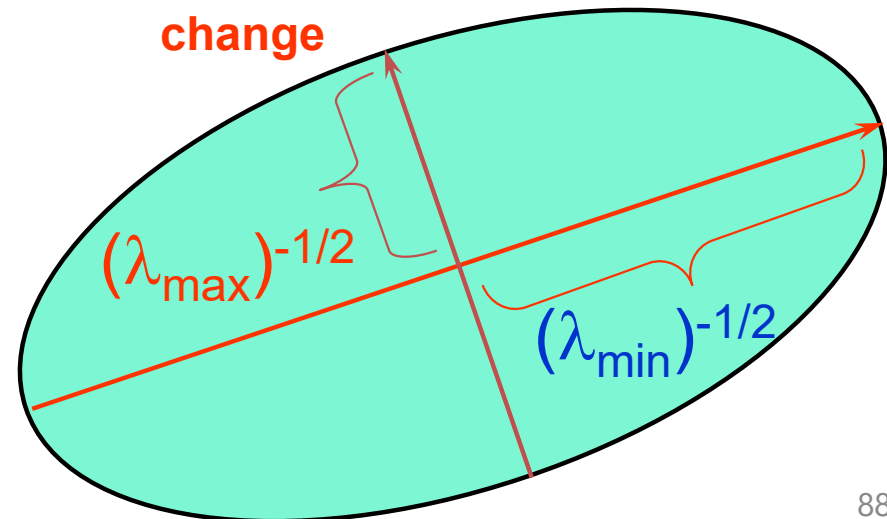
λ_1, λ_2 – eigenvalues of M



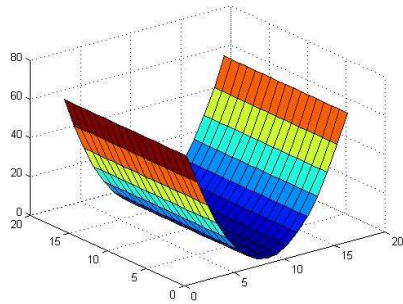
direction of the
slowest
change

Ellipse $E(u, v) = \text{const}$

direction of
the fastest
change



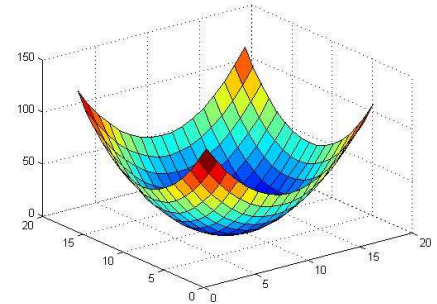
Classification of Image Points using Eigenvalues of M :



λ_2

“Edge”

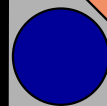
$\lambda_2 \gg \lambda_1$



● “Corner”

λ_1 and λ_2 are large,
 $\lambda_1 \sim \lambda_2$;
 E increases in all
directions

λ_1 and λ_2 are
small;
 E is almost
constant in all
directions



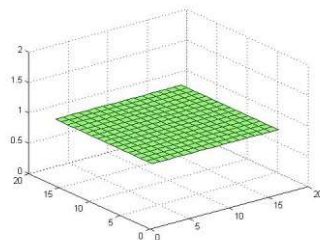
“Flat”
region



“Edge”

$\lambda_1 \gg \lambda_2$

λ_1
89



Harris Detector: Mathematics

Measure of corner response:

$$R = \det M - k (\text{trace } M)^2$$

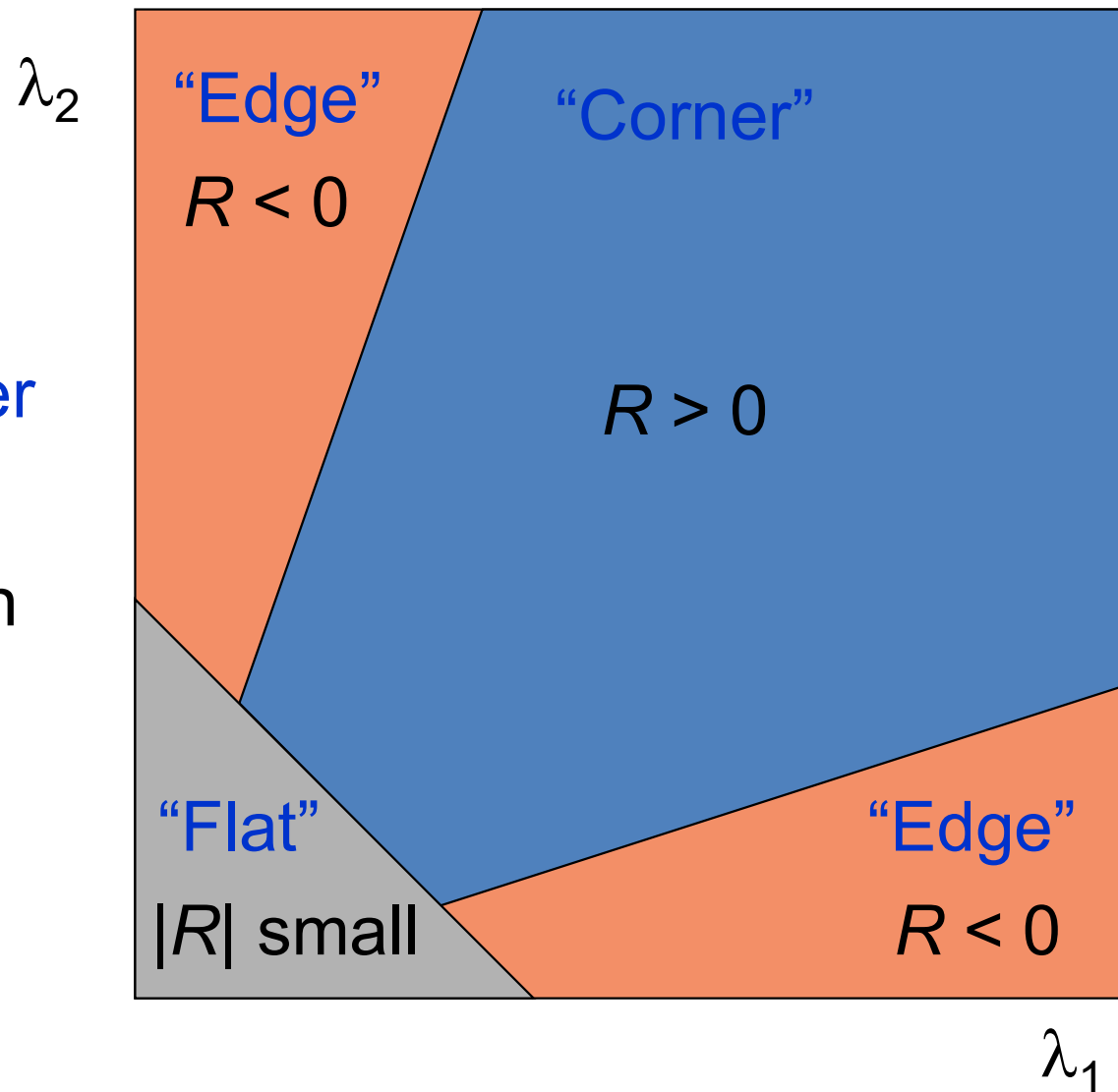
$$\det M = \lambda_1 \lambda_2$$

$$\text{trace } M = \lambda_1 + \lambda_2$$

(k – empirical constant, $k = 0.04-0.06$)

Harris Detector

- R depends only on eigenvalues of M
- R is large for a **corner**
- R is negative with large magnitude for an **edge**
- $|R|$ is small for a **flat** region



Summary of the Harris detector

1. Compute x and y derivatives of image

$$I_x = G_\sigma^x * I \quad I_y = G_\sigma^y * I$$

2. Compute products of derivatives at every pixel

$$I_{x2} = I_x \cdot I_x \quad I_{y2} = I_y \cdot I_y \quad I_{xy} = I_x \cdot I_y$$

3. Compute the sums of the products of derivatives at each pixel

$$S_{x2} = G_{\sigma^2} * I_{x2} \quad S_{y2} = G_{\sigma^2} * I_{y2} \quad S_{xy} = G_{\sigma^2} * I_{xy}$$

4. Define at each pixel (x, y) the matrix

$$H(x, y) = \begin{bmatrix} S_{x2}(x, y) & S_{xy}(x, y) \\ S_{xy}(x, y) & S_{y2}(x, y) \end{bmatrix}$$

5. Compute the response of the detector at each pixel

$$R = \text{Det}(H) - k(\text{Trace}(H))^2$$

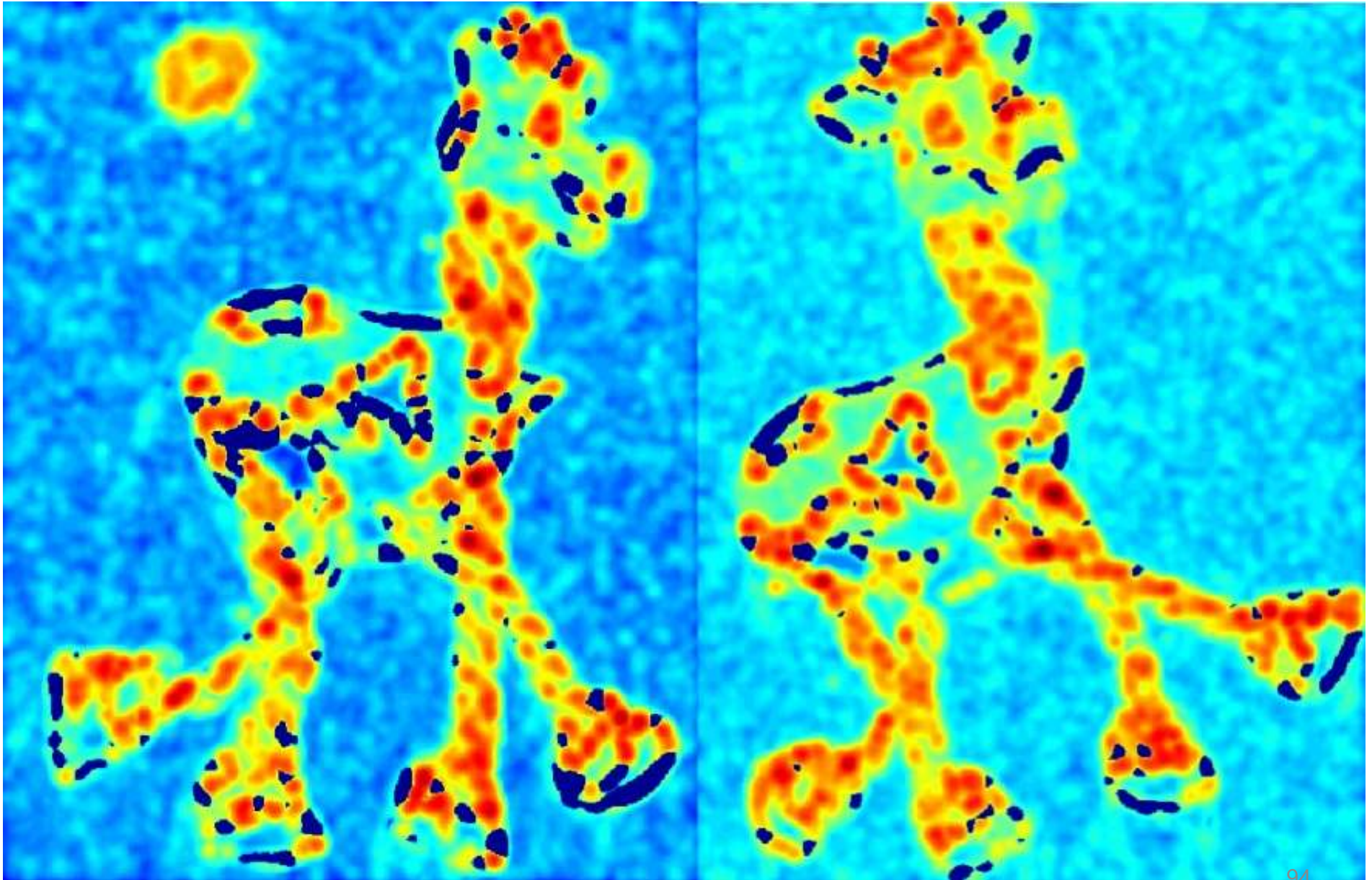
6. Threshold on value of R . Compute nonmax suppression.

Harris Detector: Workflow



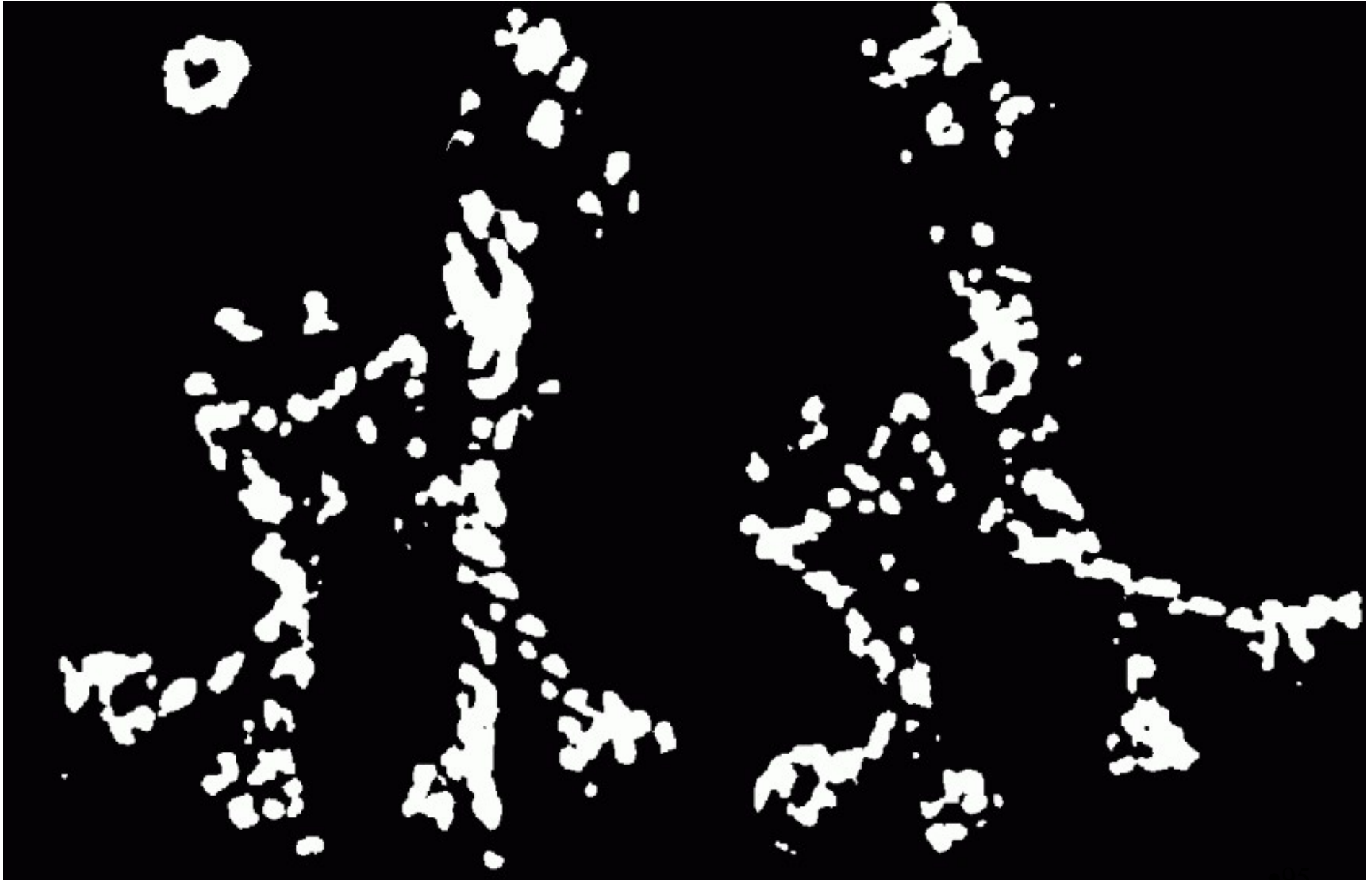
Harris Detector: Workflow

Compute corner response R



Harris Detector: Workflow

Find points with large corner response: $R > \text{threshold}$



Harris Detector: Workflow

Take only the points of local maxima of R



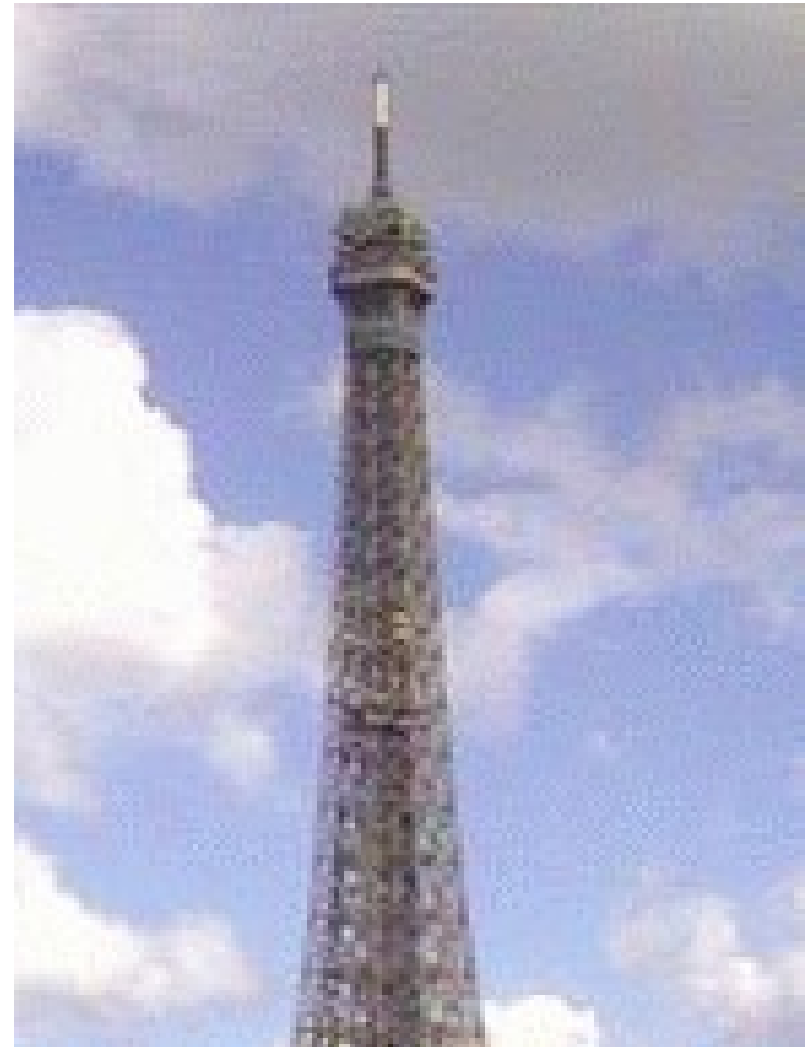
Harris Detector: Workflow



Outline

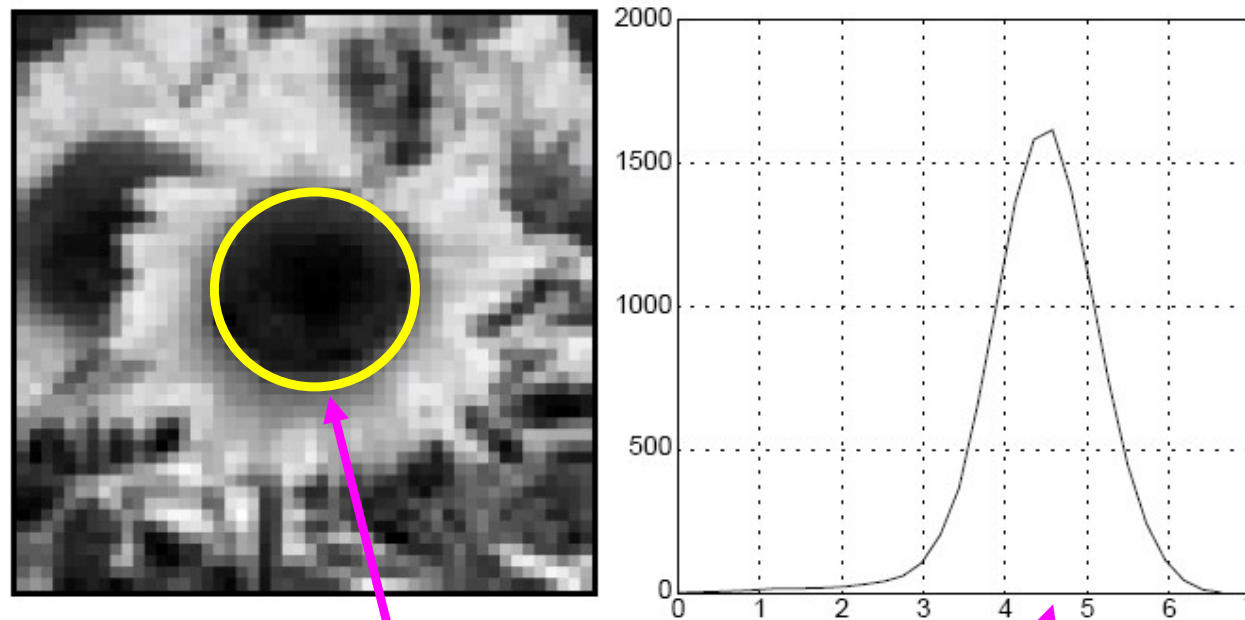
- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

Can we detect *the same* interest points regardless of *image scale changes*?



Blob Detection in 2D

- We define the *characteristic scale* as the scale that produces peak of Laplacian response



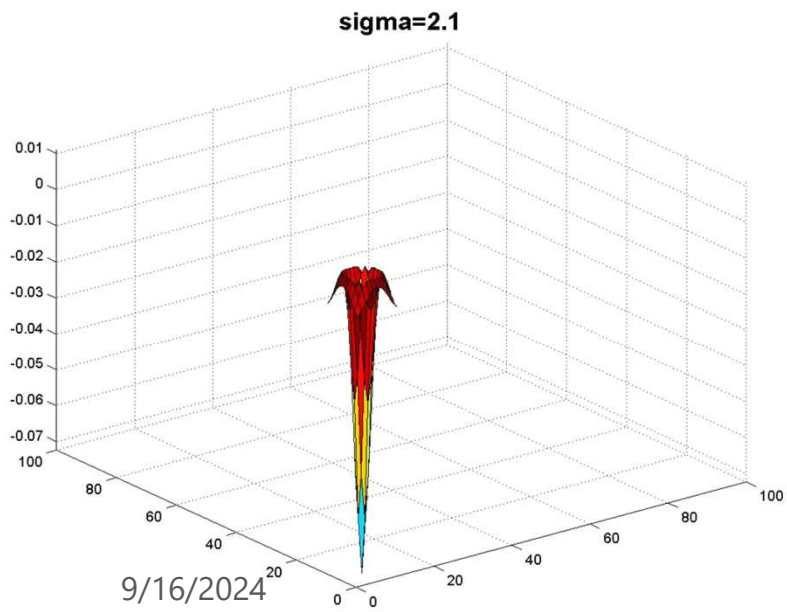
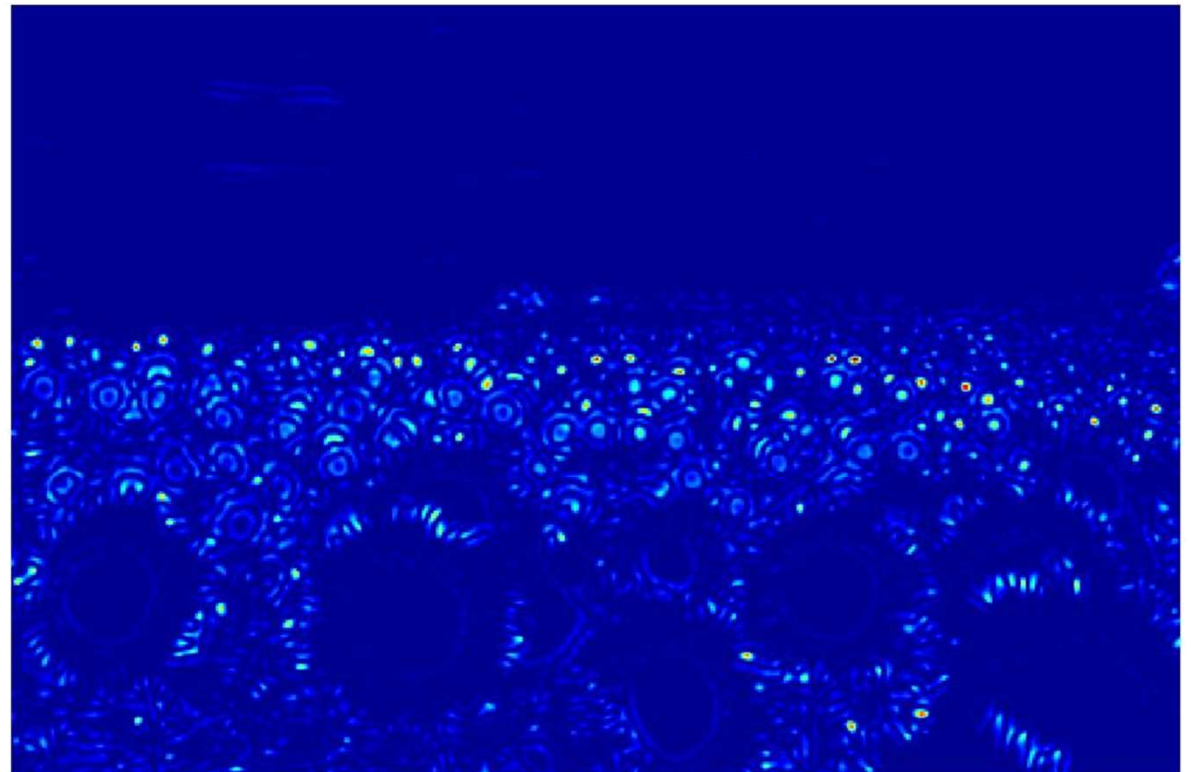
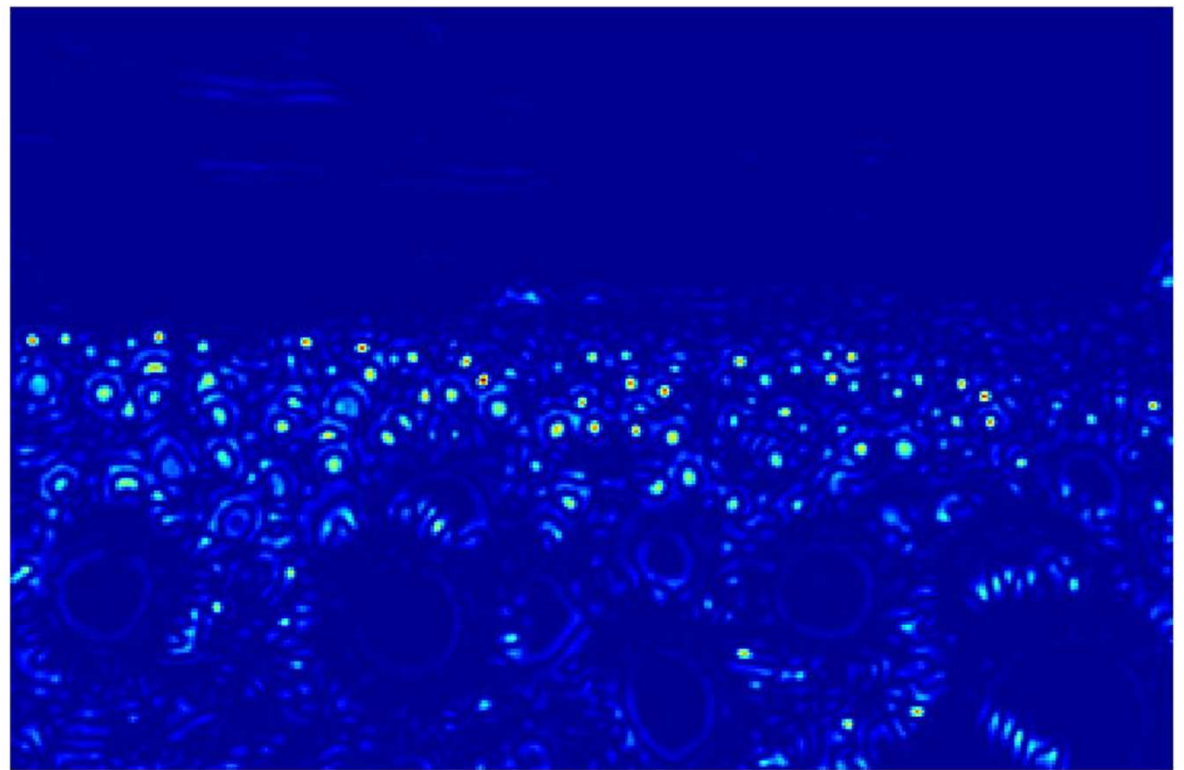
characteristic scale

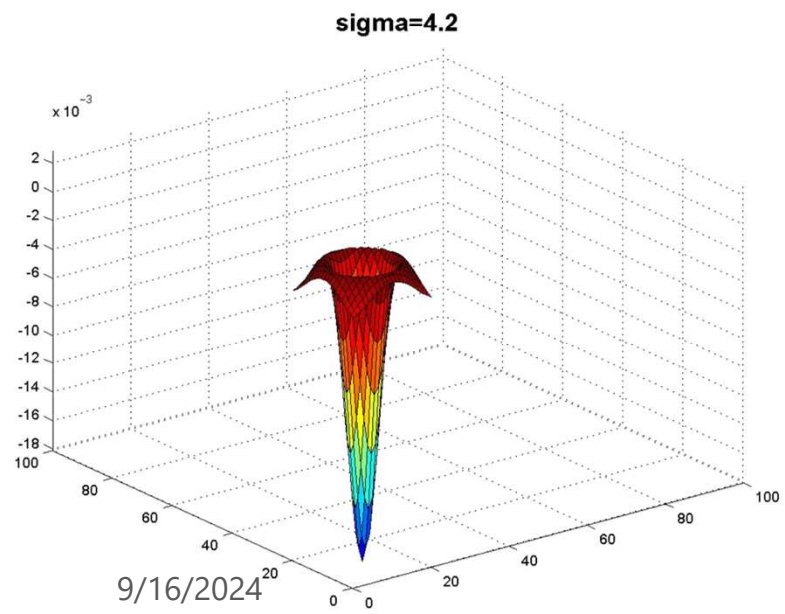
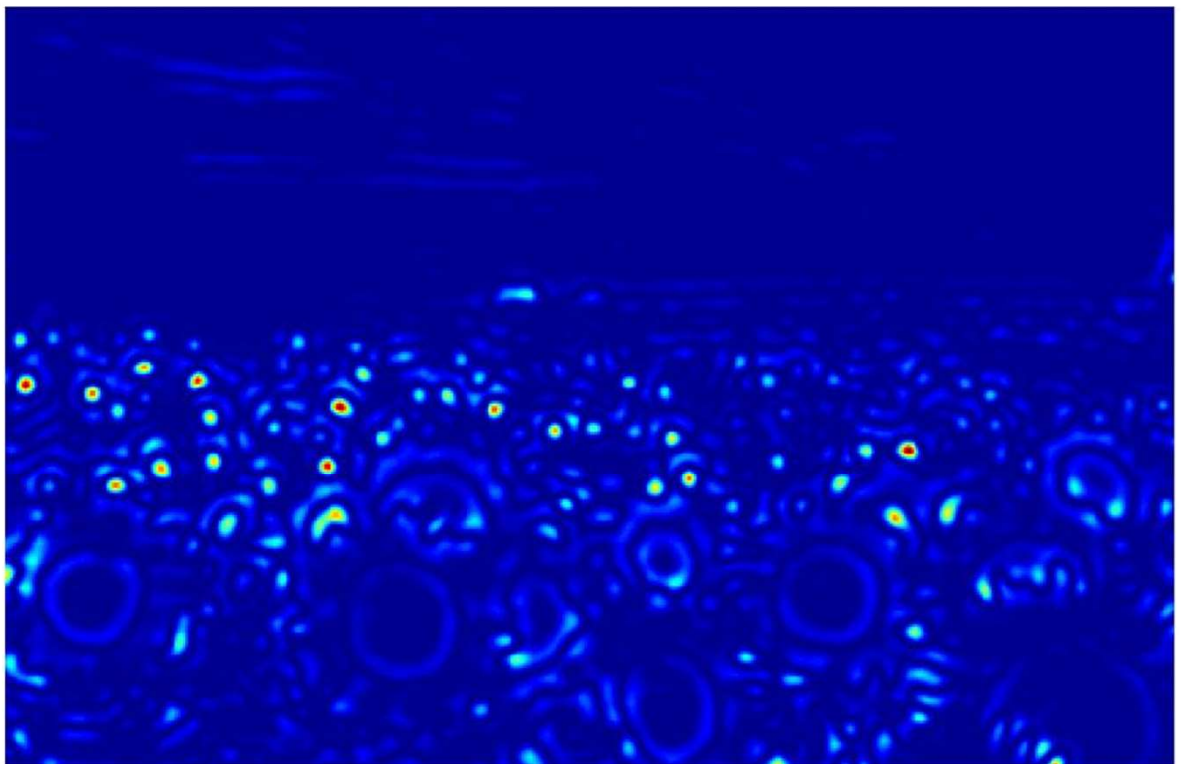
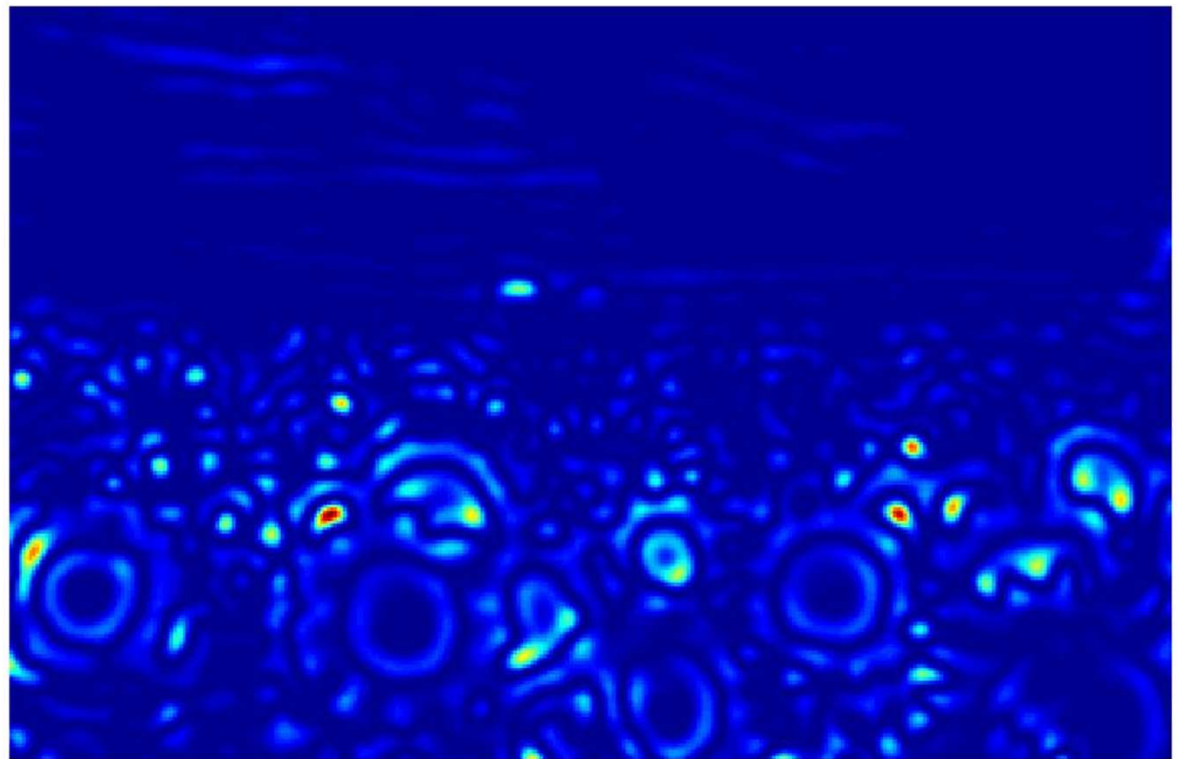
Example

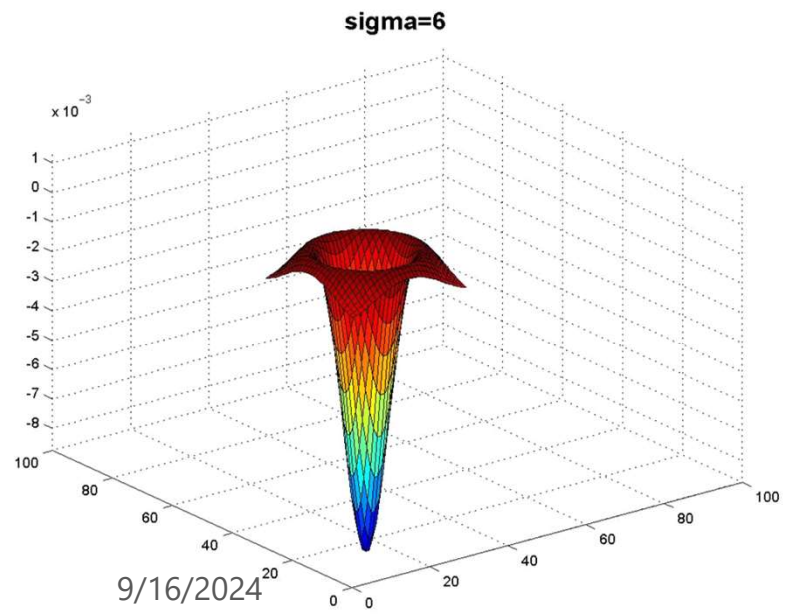
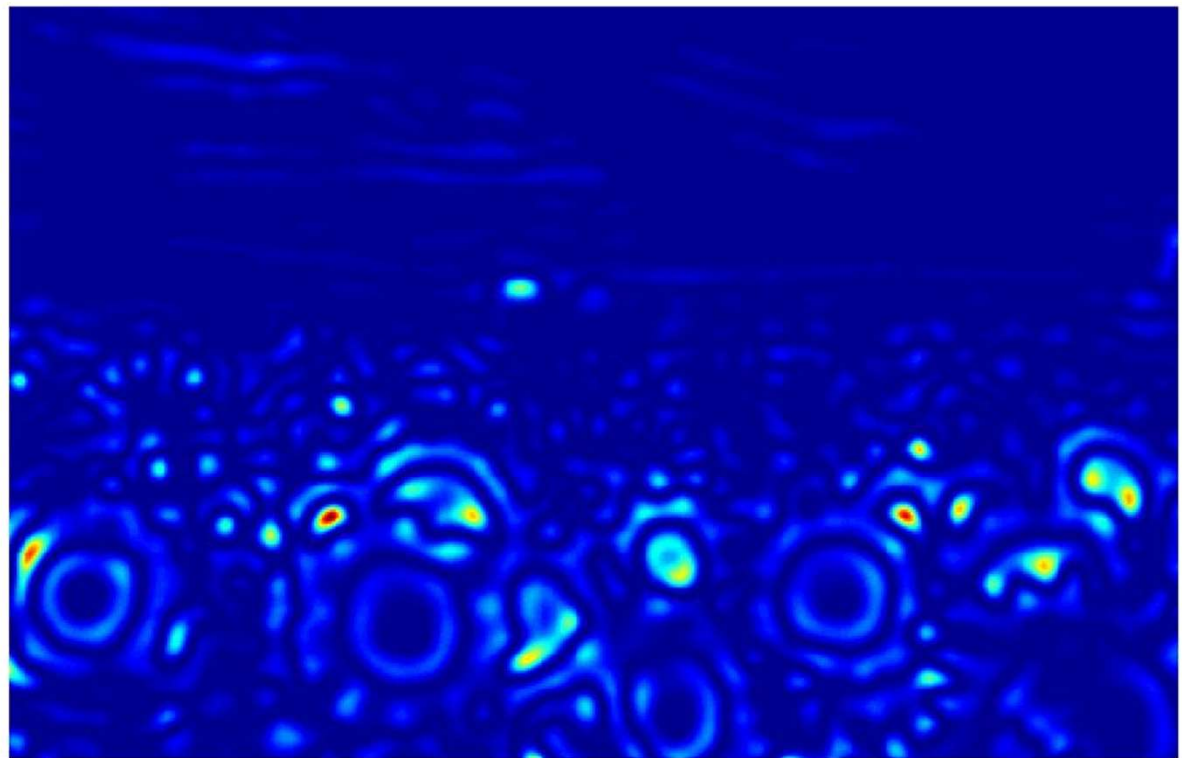
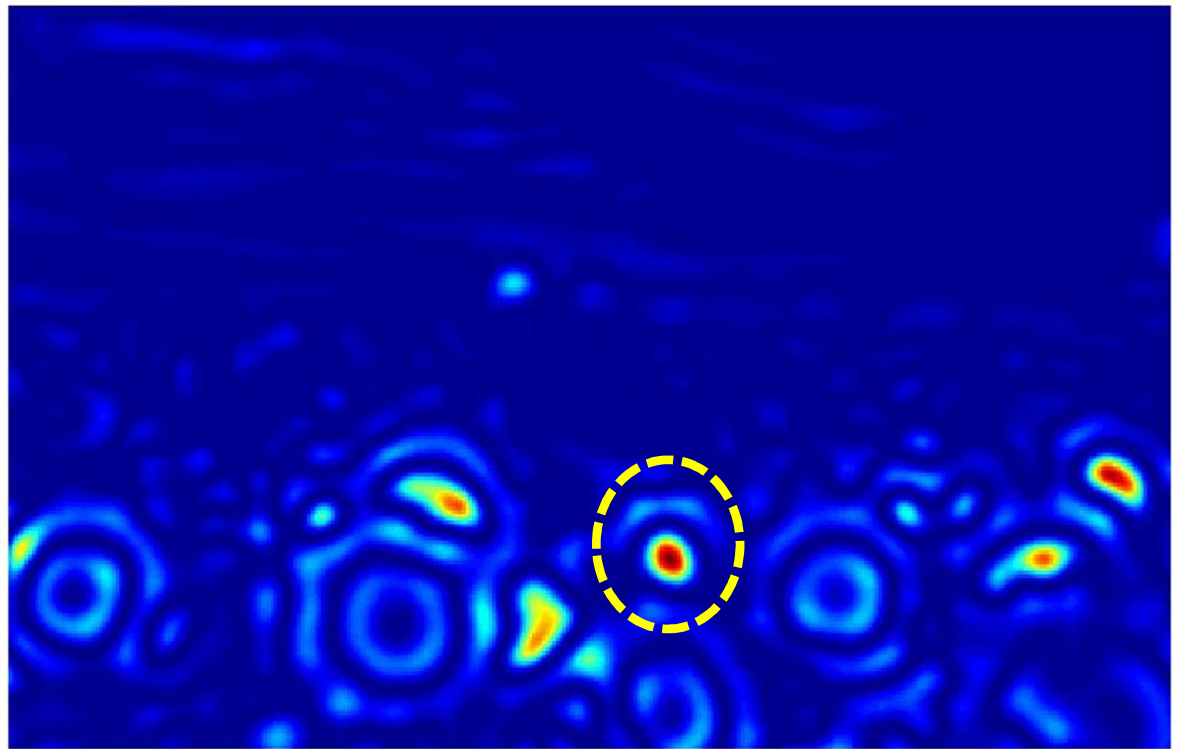
Original image
at $\frac{3}{4}$ the size

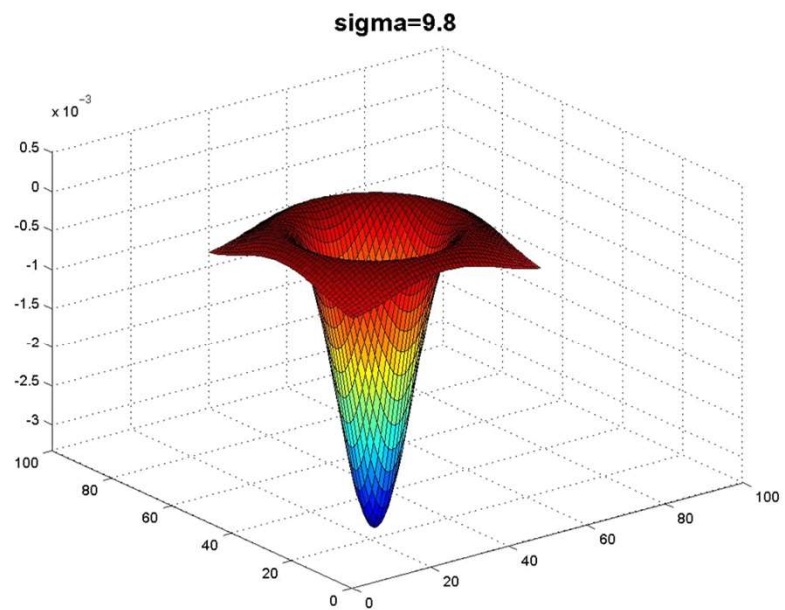
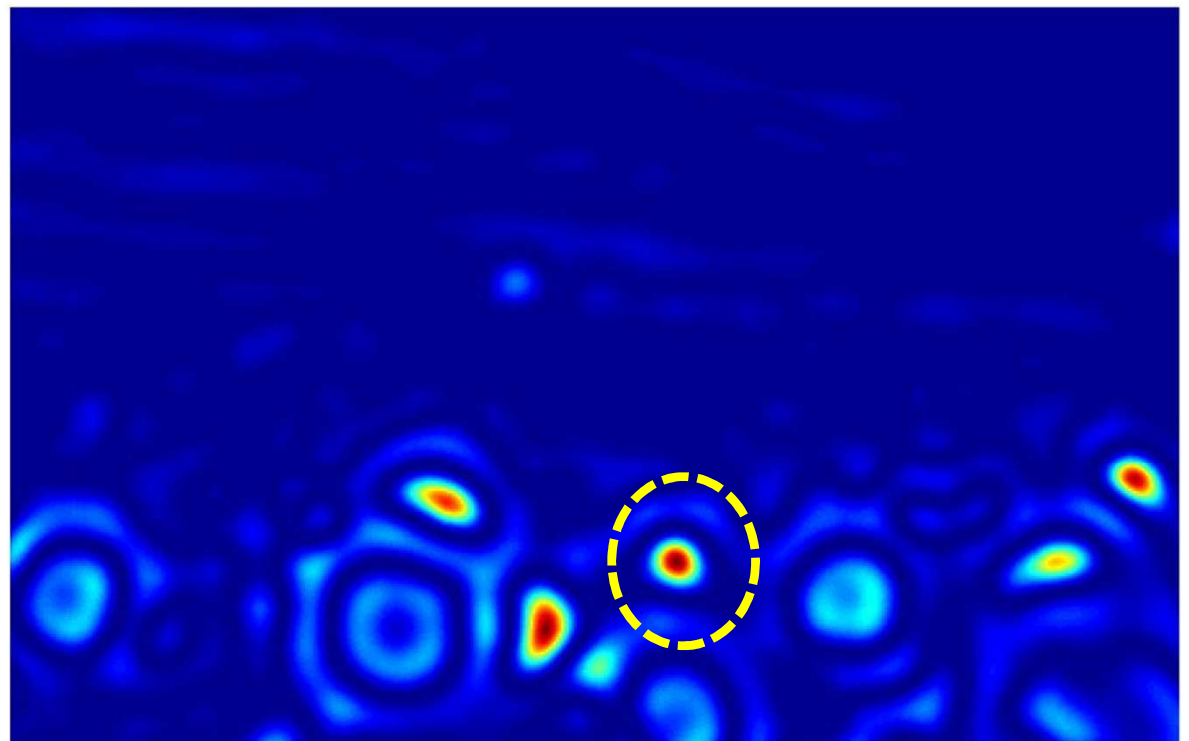
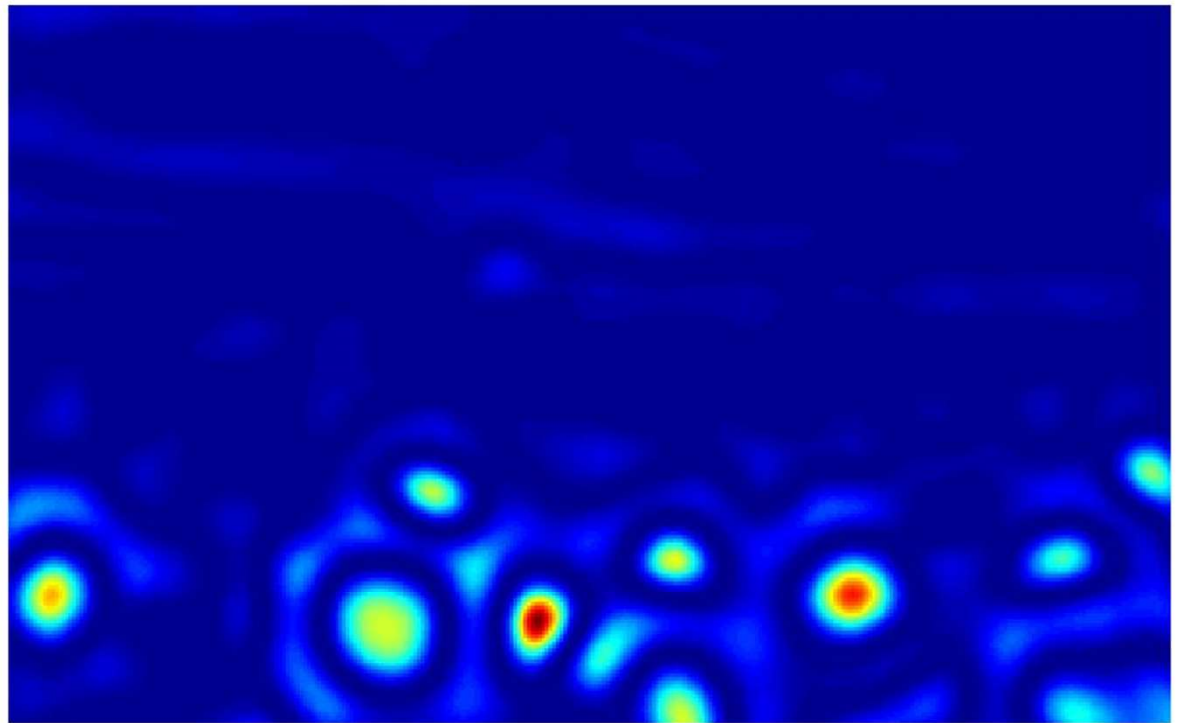


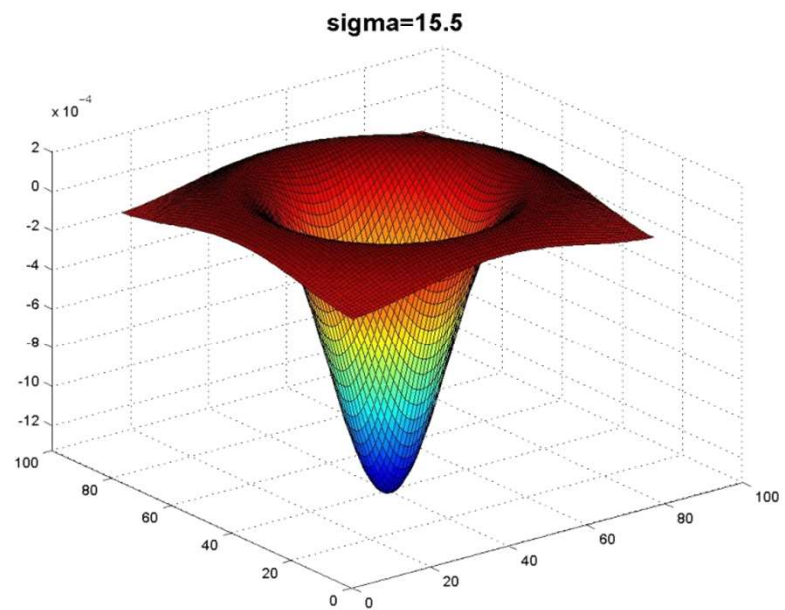
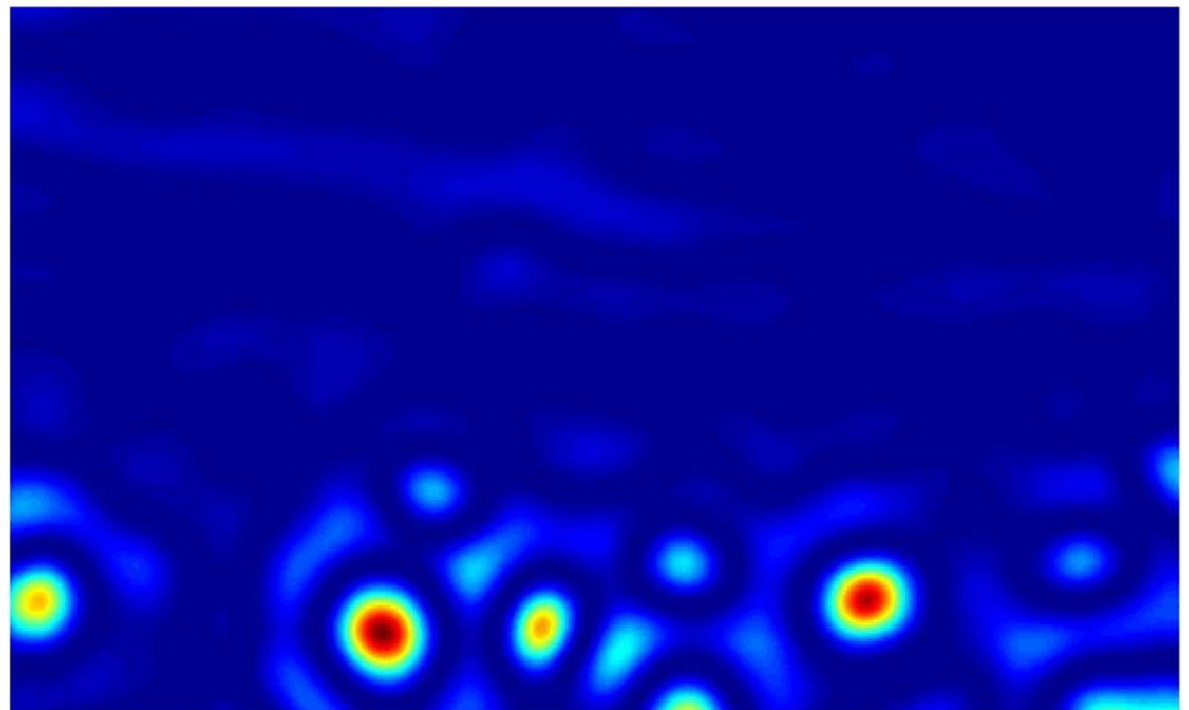
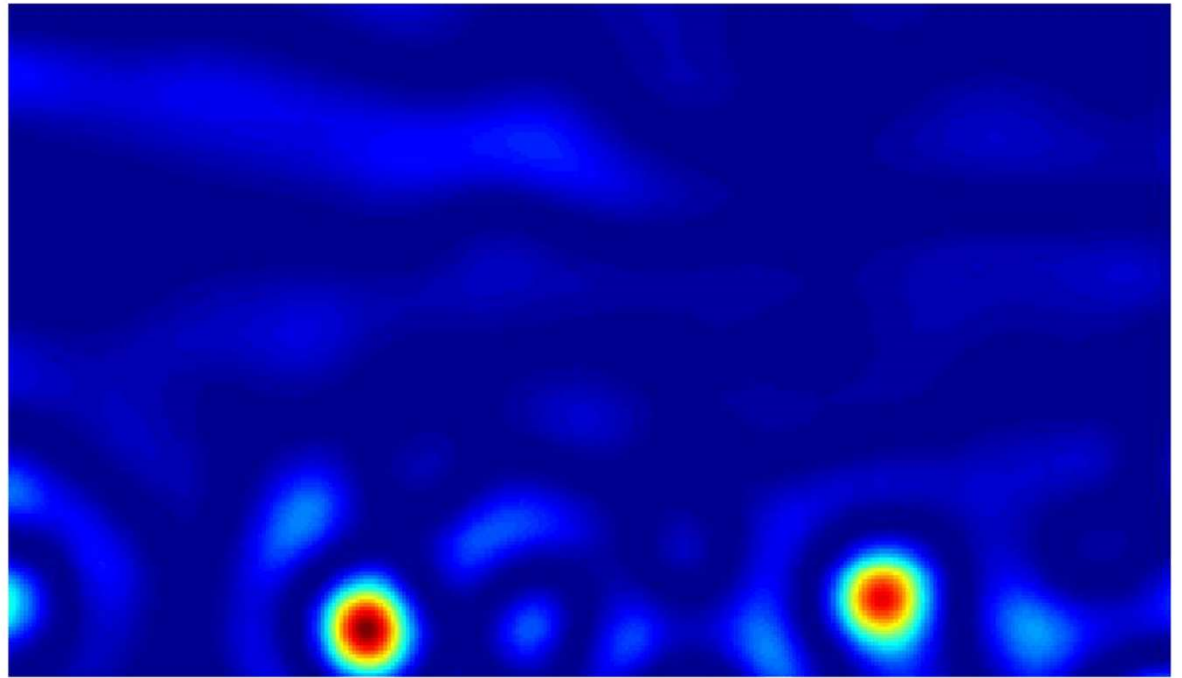
Original image
at $\frac{3}{4}$ the size











Scale Invariant Interest Points

Interest points are local maxima in both position and scale.



$$L_{xx}(\sigma) + L_{yy}(\sigma)$$

Squared filter
response maps

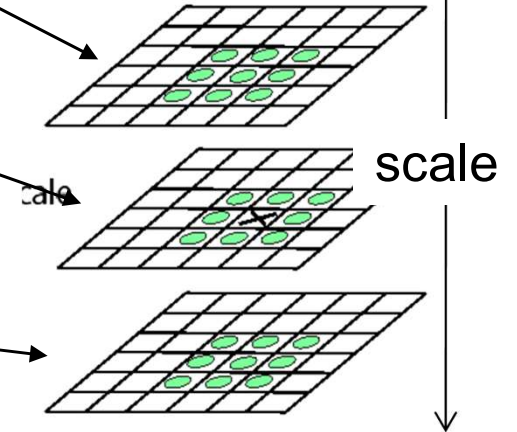
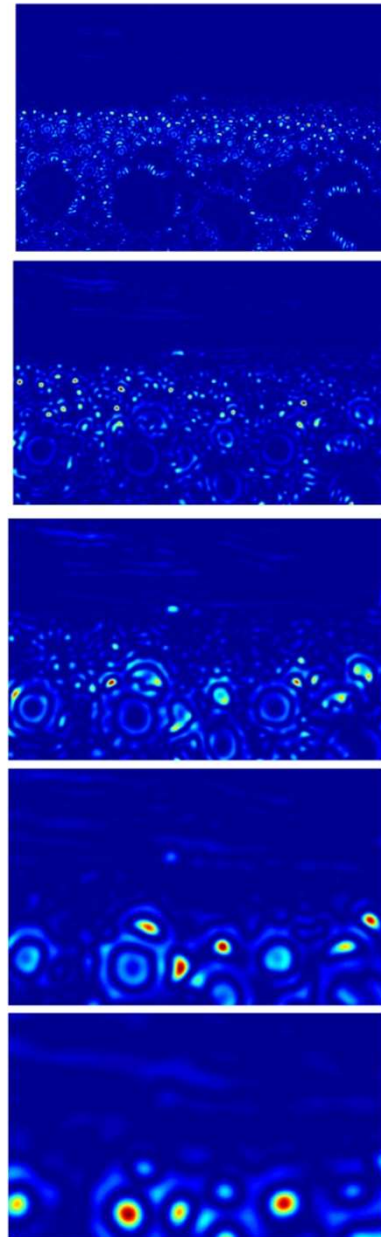
$\sigma 5$

$\sigma 4$

$\sigma 3$

$\sigma 2$

$\sigma 1$



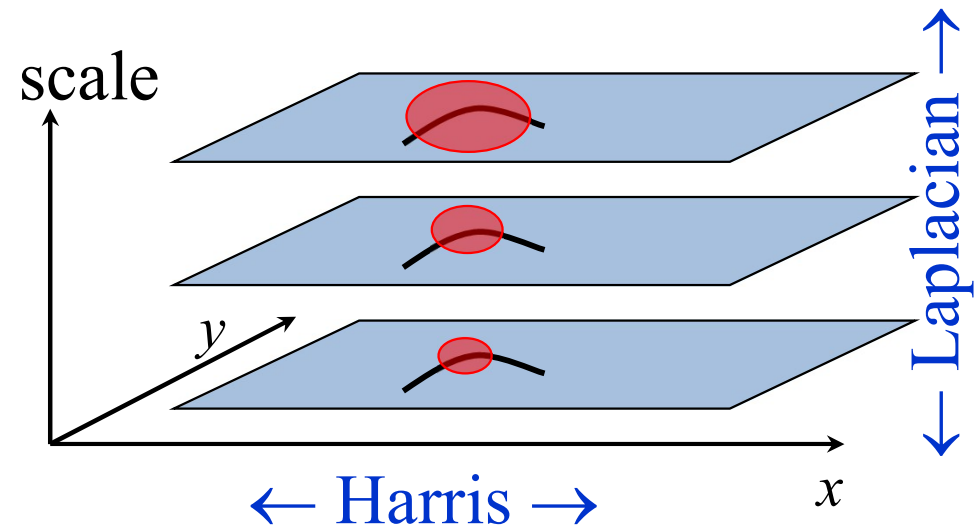
$\Rightarrow$ List of
 (x, y, σ)

Two Popular Scale Invariant Detectors

- **Harris-Laplacian**¹

Find local maximum of:

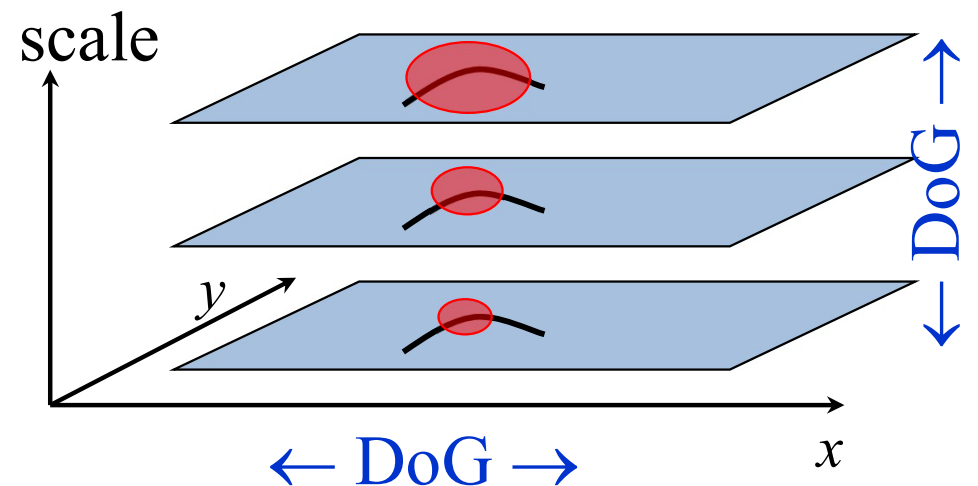
- Harris corner detector in space (image coordinates) + Laplacian in scale



- **SIFT (Lowe)**²

Find local maximum of:

- Difference of Gaussians both in space and scale

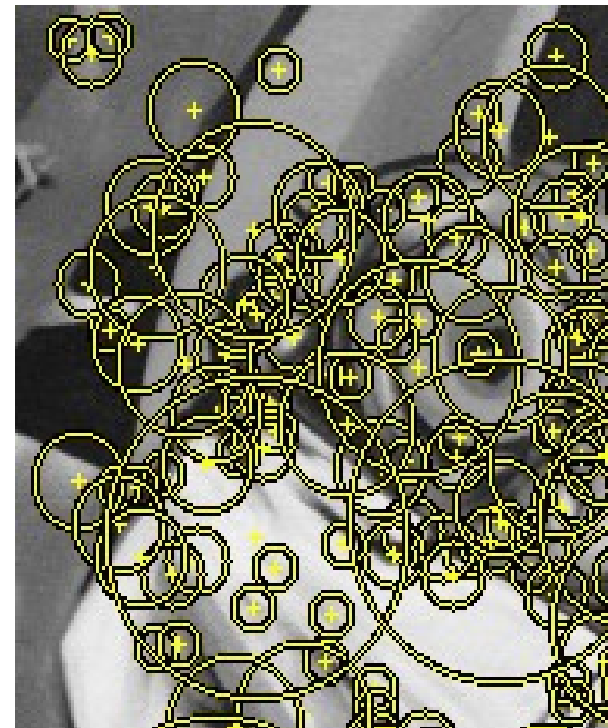


¹ K.Mikolajczyk, C.Schmid. "Indexing Based on Scale Invariant Interest Points". ICCV 2001

² D.Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". Accepted to IJCV 2004

Comparison

Harris



LoG

Scale Invariant Feature Transform

Basic idea:

- Take 16x16 square window around detected feature (8x8 is shown below)
- Compute edge orientation (angle of the gradient - 90°) for each pixel
- Throw out weak edges (threshold gradient magnitude)
- Create histogram of surviving edge orie

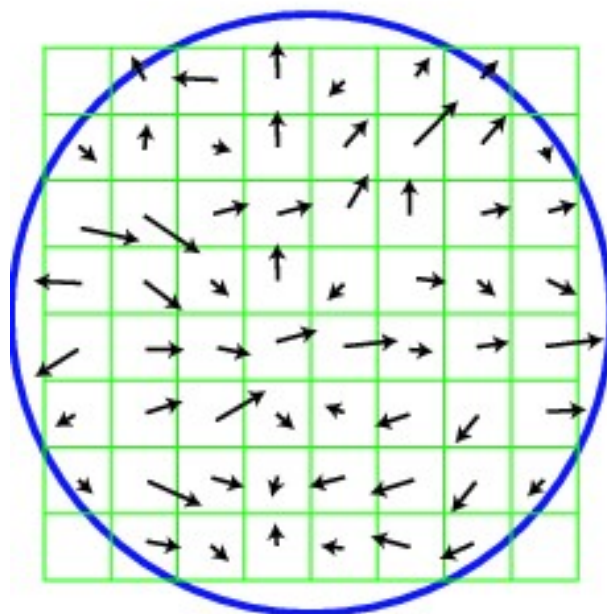
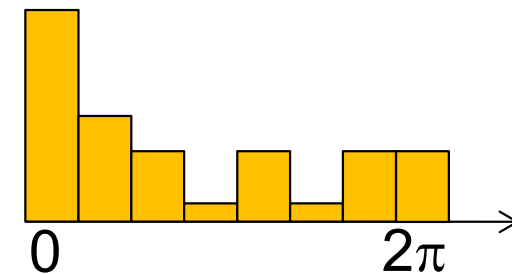
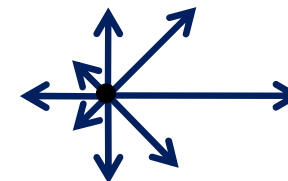


Image gradients



angle histogram

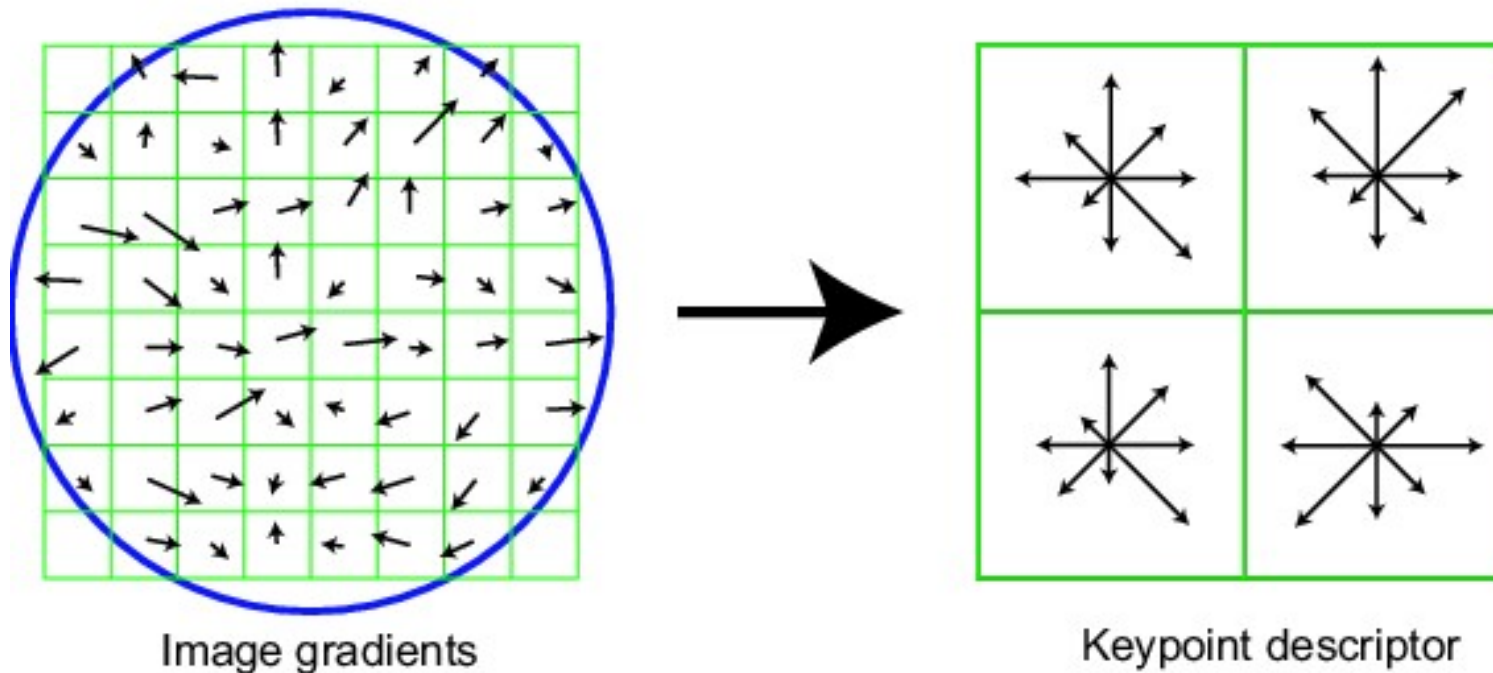


Keypoint descriptor

SIFT descriptor

Full version

- Divide the 16x16 window into a 4x4 grid of cells (2x2 case shown below)
- Compute an orientation histogram for each cell (with 8 orientations)
- 16 cells (from 4x4 grids) * 8 orientations (per cell) = 128 dimensional descriptor

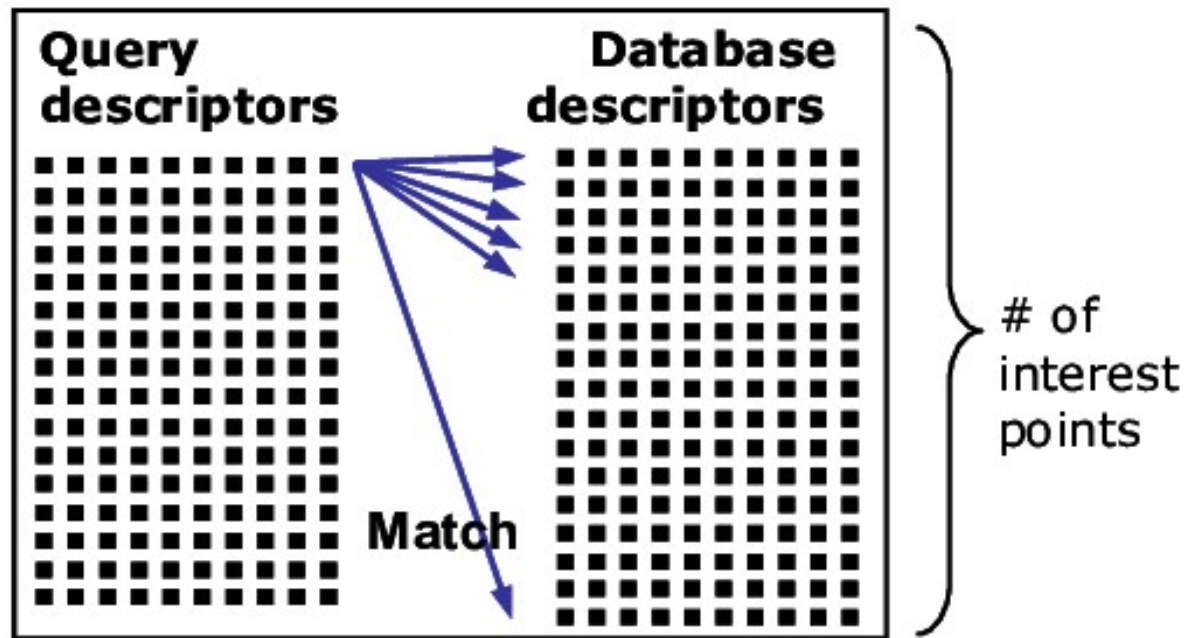
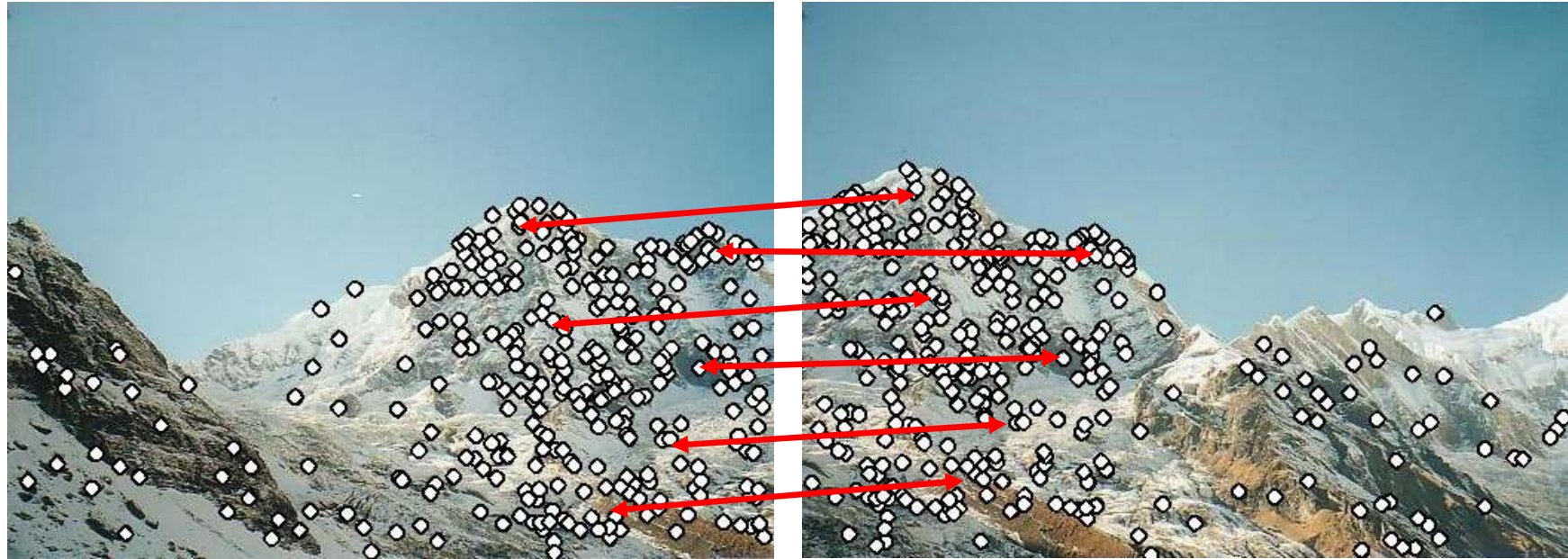


Rotation invariant (robust)



After compute the histogram of gradient directions, rotate the patch to the dominant direction

Match features: brutal force matcher



SIFT properties

- Invariant to
 - Scale: detect scale using DoG
 - Rotation: detect dominant direction using histogram
- Partially invariant to
 - Illumination changes: magnitude of the gradient is thrown away, only use the direction.
 - Camera viewpoint (affine transform)
 - Occlusion, clutter

Outline

- Edge detection
 - Derivatives of Image, Derivatives of Gaussian
 - Canny Edge Detector
- Line fitting
 - Least square
 - RANSAC,
 - Hough transform;
- Corners
 - Harris corner
 - SIFT
 - Applications

Applications

- Feature points are used for:
 - Image alignment
 - 3D reconstruction
 - Motion tracking
 - Robot navigation
 - Indexing and database retrieval
 - Object recognition



Multi-band Blending



P. J. Burt and E. H. Adelson. [A multiresolution spline with application to image mosaics](#). ACM Trans. on Graphics, 2(4), 1983.

Multi-band Blending



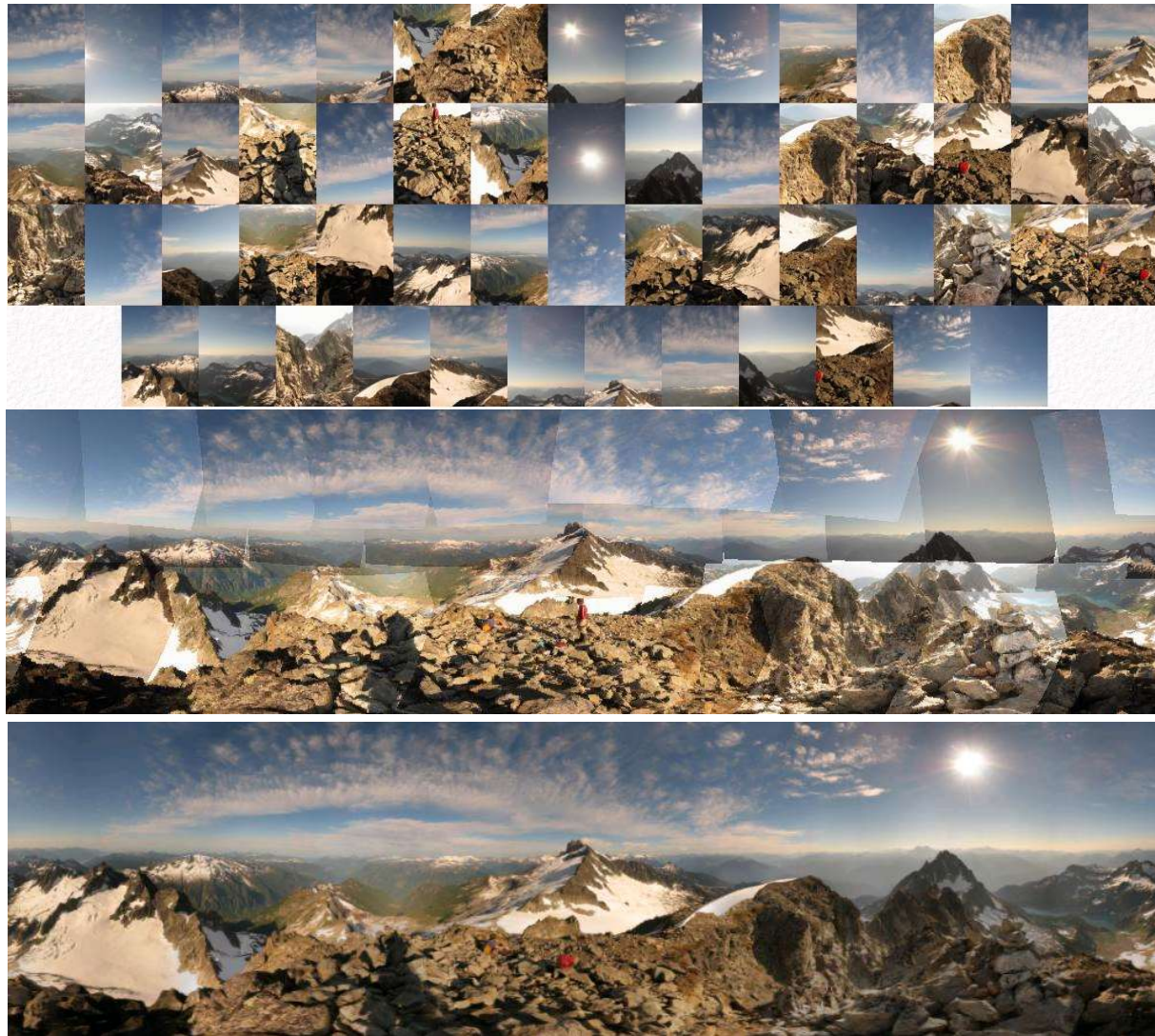
P. J. Burt and E. H. Adelson. [A multiresolution spline with application to image mosaics](#). ACM Trans. on Graphics, 2(4), 1983.

Multi-band Blending



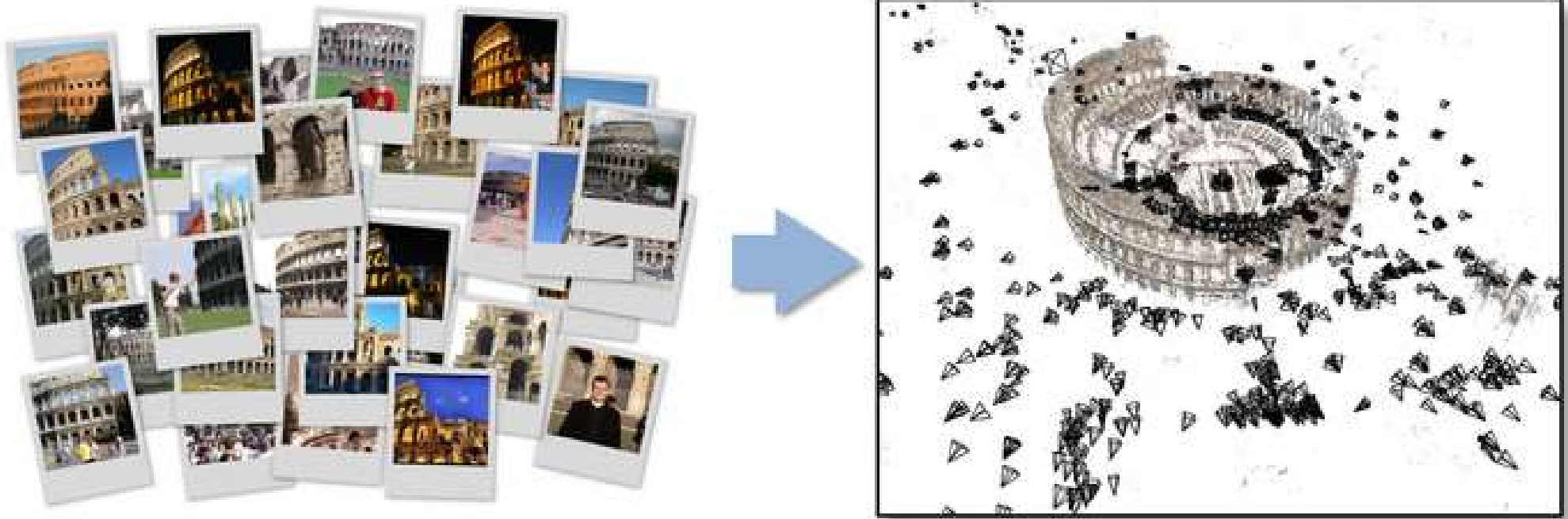
P. J. Burt and E. H. Adelson. [A multiresolution spline with application to image mosaics](#). ACM Trans. on Graphics, 2(4), 1983.

Automatic Mosaicing

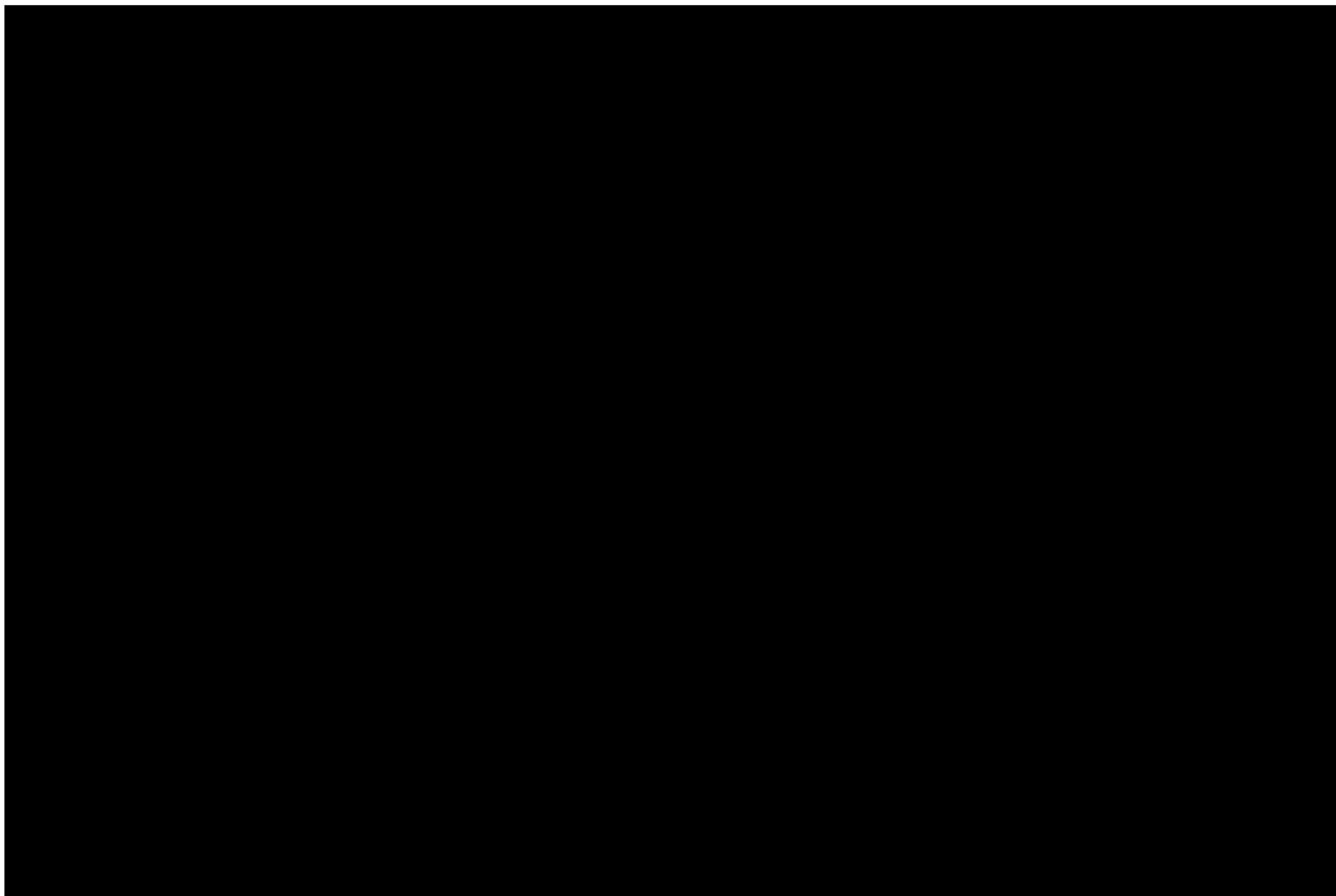


<http://www.cs.ubc.ca/~mbrown/autostitch/autostitch.html>

Example: Structure from Motion



<https://www.youtube.com/watch?v=4cEQZreQ2zQ>



Jan-Michael Frahm et al. "[Building rome on a cloudless day](#)" *11th European Conference on Computer Vision (ECCV)*, 2010.

Action Points

- This week:
 - Thursday Lectures: Motion and optical flow
 - Finish assignment 2.
 - Finish Exercise (1%)

Disclaimer

- Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.